

MATERIALE DI STUDIO · DEEP LEARNING PER L'IMAGING MEDICO

Reti Neurali Convoluzionali

Dalla convoluzione elementare alla segmentazione semantica: fondamenti matematici e concettuali in preparazione a U-Net

Destinatario Studente di Laurea Magistrale in Computer Science & Engineering

Contesto Tesi su segmentazione di immagini mediche (caso di studio: reni, CT addominale)

Prerequisiti Algebra lineare di base, calcolo differenziale, nozioni elementari di reti neurali feed-forward

Obiettivo Comprendere *cosa accade matematicamente* in una CNN, non come si invoca una libreria

Struttura 34 capitoli, in sequenza logica: ogni concetto è prerequisito del successivo

Indice

PARTE I · FONDAMENTA

- 1 Introduzione al Deep Learning
- 2 Perché le CNN
- 3 Struttura di una CNN
- 4 Tensori
- 5 Immagini RGB (e immagini mediche)

PARTE II · L'OPERATORE CONVOLUZIONALE

- 6 Kernel
- 7 Convoluzione
- 8 Padding
- 9 Stride
- 10 Feature Map
- 11 Filtri multipli

PARTE III · BLOCCHI DI RETE

- 12 ReLU e non linearità
- 13 Pooling
- 14 Batch Normalization
- 15 Dropout
- 16 Flatten
- 17 Fully Connected Layer

PARTE IV · DALL'OUTPUT ALL'ERRORE

- 18 Logits
- 19 Softmax
- 20 Loss Function
- 21 Cross Entropy

PARTE V · APPRENDIMENTO

- 22 Gradient Descent

23 Learning Rate

24 Backpropagation

25 Epoch

26 Batch

PARTE VI · GENERALIZZAZIONE

27 Validation

28 Overfitting

29 Underfitting

30 Data Augmentation

PARTE VII · VERSO LA SEGMENTAZIONE

31 Pipeline completa di classificazione

32 Dalla classificazione alla segmentazione

33 Introduzione a U-Net

34 Glossario

COME LEGGERE QUESTA DISPENSA

Ogni capitolo segue lo stesso schema: **definizione tecnica, intuizione, formalizzazione matematica, esempio numerico svolto, figura o schema, errori comuni e riepilogo**. I capitoli sono pensati per essere letti in ordine: ogni concetto usa esplicitamente il vocabolario introdotto nei precedenti. I riferimenti in avanti sono segnalati con la dicitura ($\rightarrow$ Cap. n), così che sia sempre chiaro dove un discorso verrà chiuso.

Il filo conduttore è unico: una rete che classifica immagini e una rete che segmenta immagini condividono **il novanta per cento della matematica**. Cambiano la forma dell'output e la forma del ground truth. Tutto ciò che si costruisce nei capitoli 1–31 serve, senza modifiche concettuali, nei capitoli 32–33.

1 Introduzione al Deep Learning

Prima di parlare di convoluzioni conviene fissare con precisione che cosa sia un modello di apprendimento, che cosa significhi "apprendere" in senso matematico, e perché la profondità di una rete sia una proprietà qualitativamente diversa dalla sua larghezza.

1.1 Il problema dell'apprendimento supervisionato

Si parte sempre dallo stesso oggetto: un insieme di coppie

$$\mathcal{D} = \{(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}), (\mathbf{x}^{(2)}, \mathbf{y}^{(2)}), \dots, (\mathbf{x}^{(N)}, \mathbf{y}^{(N)})\}$$

dove $\mathbf{x}^{(i)}$ è un **input** (per noi: un'immagine, o un volume CT) e $\mathbf{y}^{(i)}$ è il **target**, detto anche *ground truth*: l'etichetta corretta, prodotta da un annotatore umano. Si assume che le coppie siano campionate in modo indipendente da una distribuzione di probabilità congiunta $p(\mathbf{x}, \mathbf{y})$ che noi non conosciamo e non conosceremo mai.

L'obiettivo non è memorizzare $\mathcal{D}$. L'obiettivo è costruire una funzione

$$f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$$

parametrizzata da un vettore di parametri θ , tale che $f_{\theta}(\mathbf{x}) \approx \mathbf{y}$ **anche per coppie mai viste**, purché provenienti dalla stessa distribuzione. Questa distinzione — funzionare sui dati visti contro funzionare sui dati non visti — è il cuore dei capitoli 27–30 e, in pratica, il cuore di qualunque tesi sperimentale.

DEFINIZIONE • APPRENDIMENTO SUPERVISIONATO

Dato un insieme finito di esempi etichettati e una famiglia parametrica di funzioni $\{f_{\theta} : \theta \in \mathbb{R}^P\}$, l'apprendimento supervisionato è il problema di determinare il valore di θ che minimizza il valore atteso di una funzione di costo $\mathcal{L}$ sulla distribuzione generatrice dei dati:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim p} [\mathcal{L}(f_{\theta}(\mathbf{x}), \mathbf{y})]$$

Poiché p è ignota, in pratica si minimizza la media empirica su $\mathcal{D}$ — il cosiddetto *rischio empirico* — e si spera che minimizzarlo faccia scendere anche il rischio vero. Tutto il capitolo sull'overfitting nasce da questa speranza e dalle sue condizioni di validità.

1.2 Che cosa significa "profondo"

Un modello lineare calcola $f(\mathbf{x}) = W\mathbf{x} + \mathbf{b}$. È interpretabile, si stima in forma chiusa, e ha un limite fatale: la sua capacità espressiva è quella di un iperpiano. Nessuna scelta di W e $\mathbf{b}$ permetterà a un modello lineare di separare

due classi disposte a spirale concentrica, né di riconoscere che un rene è un rene indipendentemente da dove si trovi nell'immagine.

Una rete **profonda** è la composizione di più trasformazioni, ciascuna seguita da una funzione non lineare:

$$f_{\theta}(\mathbf{x}) = (\phi_L \circ g_L \circ \phi_{L-1} \circ g_{L-1} \circ \cdots \circ \phi_1 \circ g_1)(\mathbf{x})$$

dove ogni g_{ℓ} è una trasformazione parametrica (tipicamente affine, cioè $W_{\ell}\mathbf{x} + \mathbf{b}_{\ell}$, oppure una convoluzione, → Cap. 7) e ogni ϕ_{ℓ} è una non linearità applicata elemento per elemento (→ Cap. 12).

Il punto essenziale, che va compreso subito perché ritorna in ogni capitolo successivo: **senza le ϕ_{ℓ} , la composizione collassa**. Infatti

$$W_3(W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3 = \underbrace{(W_3W_2W_1)}_{\tilde{W}}\mathbf{x} + \underbrace{(W_3W_2\mathbf{b}_1 + W_3\mathbf{b}_2 + \mathbf{b}_3)}_{\tilde{\mathbf{b}}}$$

e cioè: tre strati lineari sono *esattamente* equivalenti a un solo strato lineare con matrice $\tilde{W}$. Aggiungere strati lineari non aggiunge un solo grado di libertà espressivo; aggiunge soltanto parametri ridondanti e problemi di ottimizzazione. La profondità ha senso solo in presenza di non linearità.

1.3 Feature ingegnerizzate contro feature apprese

La differenza storica fra machine learning classico e deep learning si riassume in una domanda: *chi decide quali caratteristiche dell'input sono rilevanti?*

ASPETTO	ML CLASSICO (PRE-2012)	DEEP LEARNING
Rappresentazione	Feature progettate a mano (SIFT, HOG, filtri di Gabor, wavelet, statistiche di texture)	Feature apprese dai dati, come sottoprodotto della minimizzazione della loss
Ruolo dell'esperto	Progetta gli estrattori di feature; il classificatore è semplice (SVM, random forest)	Progetta l'architettura e la funzione obiettivo; le feature emergono
Dati necessari	Relativamente pochi	Molti (o pochi + <i>transfer learning</i> e augmentation, → Cap. 30)
Comportamento in imaging medico	Radiomica: si estraggono centinaia di descrittori numerici predefiniti	La rete costruisce da sola descrittori gerarchici specifici del compito

Tabella 1.1 — Il deep learning non elimina la progettazione: la sposta dal livello delle feature al livello dell'architettura e della funzione obiettivo.

INTUIZIONE

Un radiologo non identifica un rene misurando l'intensità di un singolo voxel. Usa una gerarchia implicita: bordi → contorni chiusi → forma a fagiolo → posizione retroperitoneale → relazione con fegato, milza, colonna. Una rete profonda costruisce una gerarchia analoga in modo automatico. I primi strati rispondono a variazioni locali di intensità; gli strati intermedi combinano queste risposte in configurazioni geometriche; gli strati profondi rappresentano parti anatomiche intere. Nessuno ha scritto queste regole: sono la conseguenza numerica della discesa del gradiente su milioni di esempi.

1.4 I tre ingredienti indispensabili

Qualunque sistema di deep learning, dalla rete più semplice a nnU-Net, poggia su tre componenti separabili. Tenerli distinti in testa evita gran parte della confusione tipica del principiante.

1. **Architettura** — la forma della funzione f_{θ} : quanti strati, di che tipo, connessi come. È una scelta di progetto, non appresa (o appresa solo in regimi di *neural architecture search*).
2. **Funzione obiettivo** — la loss $\mathcal{L}$ (→ Cap. 20): un numero scalare che quantifica quanto l'output è sbagliato. Definisce *che cosa* la rete considera "bene".
3. **Algoritmo di ottimizzazione** — la procedura che modifica θ per far scendere $\mathcal{L}$: discesa del gradiente e sue varianti (→ Cap. 22), alimentata dalle derivate calcolate con backpropagation (→ Cap. 24).

Un errore di ragionamento estremamente comune consiste nel confondere il primo con il terzo: pensare che l'architettura "cerchi" qualcosa. L'architettura non cerca nulla. È un'espressione matematica fissa con dei buchi (i parametri). È l'ottimizzatore, guidato dalla loss, a riempire quei buchi.

ERRORI COMUNI

«La rete impara le immagini.» No. La rete non conserva alcuna immagine. Conserva solo θ : un vettore di numeri reali la cui dimensione non dipende da N . Un modello addestrato su 10.000 CT e uno addestrato su 45 CT hanno esattamente lo stesso numero di parametri, se l'architettura è la stessa.

«Più strati è sempre meglio.» No. Più strati significa più capacità, quindi maggiore rischio di overfitting (→ Cap. 28), gradienti più difficili da propagare, più memoria. La profondità è utile fino al punto in cui il beneficio rappresentazionale supera il costo di ottimizzazione e di generalizzazione.

RIEPILOGO DEL CAPITOLO 1

- Apprendimento supervisionato = trovare θ che minimizza una loss attesa su una distribuzione ignota, approssimata dalla media empirica sul training set.
- Una rete profonda è una composizione di trasformazioni parametriche e non linearità; senza non linearità la composizione collassa in una singola trasformazione affine.
- Il deep learning sostituisce le feature progettate a mano con feature apprese gerarchicamente.
- Tre componenti indipendenti: architettura, loss, ottimizzatore.

2 Perché le CNN

Una rete completamente connessa può, in linea di principio, approssimare qualunque funzione. Il problema non è la possibilità teorica: è il costo, la statistica e la struttura. Le CNN esistono perché le immagini hanno tre proprietà che una rete densa ignora completamente.

2.1 Il fallimento quantitativo della rete densa

Consideriamo un'immagine RGB di soli 224×224 pixel, dimensione modesta rispetto a una slice CT tipica di 512×512 . Il numero di valori scalari in input è

$$224 \times 224 \times 3 = 150\,528$$

Se il primo strato fully connected avesse 1000 neuroni, la sua matrice dei pesi avrebbe dimensione $1000 \times 150\,528$, cioè **oltre 150 milioni di parametri in un solo strato**. Con una slice CT 512×512 a singolo canale e 1000 neuroni si arriva a 262 144 000 pesi. Tre osservazioni:

- **Memoria e calcolo.** Il costo è proibitivo, e cresce quadraticamente con la risoluzione.
- **Statistica.** Con 45 volumi CT annotati (uno scenario del tutto realistico in ambito clinico) e centinaia di milioni di parametri, il rapporto parametri/esempi rende la memorizzazione inevitabile.
- **Struttura buttata via.** Il primo passo di una rete densa è *appiattire* l'immagine in un vettore. Dopo l'appiattimento, il pixel (10, 10) e il pixel (10, 11) — adiacenti — diventano due componenti qualsiasi del vettore, non più vicine del pixel (200, 3). L'informazione di adiacenza spaziale è distrutta.

2.2 Le tre proprietà strutturali delle immagini

Le CNN non sono un trucco ingegneristico: incorporano tre ipotesi (*inductive bias*) che sono vere per le immagini naturali e mediche, e false per dati tabellari.

Località

Ciò che definisce un bordo, una texture o un piccolo dettaglio anatomico è determinato da un intorno ristretto di pixel. Per capire se in (i, j) c'è un bordo non serve conoscere il pixel a 300 posizioni di distanza. Quindi ogni neurone può guardare solo una finestra locale — il suo **campo recettivo**.

Invarianza per traslazione (condivisione dei pesi)

Un rilevatore di bordo verticale è utile in alto a sinistra tanto quanto in basso a destra. Non ha senso apprendere un rilevatore diverso per ogni posizione: si apprende **un solo** insieme di pesi e lo si applica scorrendo su tutta

l'immagine. È questa condivisione (*weight sharing*) a ridurre i parametri di ordini di grandezza.

Composizionalità gerarchica

Le strutture visive complesse sono composizioni di strutture semplici: bordi → angoli e archi → contorni → parti → oggetti. Impilare strati convoluzionali riproduce esattamente questa gerarchia, perché ogni strato vede l'output del precedente e quindi possiede un campo recettivo effettivo via via più ampio (→ Cap. 11).

ESEMPIO NUMERICO • IL RISPARMIO DI PARAMETRI

Input $512 \times 512 \times 1$ (slice CT). Confrontiamo due modi di produrre 64 mappe di uscita della stessa dimensione spaziale.

Fully connected: uscita $512 \times 512 \times 64 = 16\,777\,216$ valori, ciascuno connesso a tutti i 262 144 ingressi. Pesì: $\approx 4.4 \times 10^{12}$. Impossibile.

Convoluzionale, kernel 3×3 : 64 filtri, ciascuno $3 \times 3 \times 1 = 9$ pesi più 1 bias. Totale: $64 \times (9 + 1) = 640$ parametri.

Il rapporto è di circa **sette miliardi a uno**. E i 640 parametri non sono una versione "impoverita": sono una versione che sfrutta il fatto che lo stesso rilevatore serve ovunque.

2.3 Perché questo conta in ambito medico

In segmentazione renale il vincolo dominante è la scarsità di annotazioni: segmentare manualmente un volume addominale voxel per voxel richiede ore di lavoro di un operatore esperto. Un modello con pochi parametri ben vincolati generalizza da poche decine di volumi; un modello denso non generalizzerebbe mai. Inoltre l'invarianza per traslazione è medicalmente sensata: il tessuto renale ha lo stesso aspetto locale indipendentemente dalla posizione del paziente nel campo di vista, che varia da acquisizione ad acquisizione.

ERRORI COMUNI

«La CNN è invariante alle traslazioni, quindi non le importa dove si trova l'oggetto.» Attenzione alla sfumatura: la convoluzione è *equivariante* alla traslazione, non invariante. Se l'input trasla di k pixel, la feature map trasla di k pixel: l'informazione di posizione è conservata, non cancellata. L'invarianza vera si ottiene solo dopo operazioni di aggregazione globale (pooling globale, → Cap. 13). Questa distinzione è cruciale in segmentazione, dove la posizione è esattamente ciò che si vuole predire.

RIEPILOGO DEL CAPITOLO 2

- Una rete densa su immagini esplode in parametri e distrugge l'adiacenza spaziale con il flatten iniziale.
- Le CNN incorporano tre bias induttivi veri per le immagini: località, condivisione dei pesi, composizionalità gerarchica.
- La condivisione dei pesi riduce i parametri di ordini di grandezza e rende possibile l'addestramento con pochi dati annotati.
- La convoluzione è equivariante, non invariante, alle traslazioni: proprietà indispensabile per la segmentazione.

3 Struttura di una CNN

Prima di scendere nel dettaglio dei singoli operatori conviene avere in testa la mappa completa. Questo capitolo presenta l'anatomia di una CNN per classificazione e definisce il vocabolario che i capitoli 6–19 riempiranno di contenuto matematico.

3.1 Lo schema canonico

Una CNN per classificazione è composta da due macro-blocchi funzionalmente distinti:

1. **Encoder convoluzionale (feature extractor).** Una successione di blocchi *convoluzione* → *normalizzazione* → *non linearità* → (talvolta) *pooling*. Trasforma un tensore ad alta risoluzione spaziale e pochi canali in un tensore a bassa risoluzione spaziale e molti canali. In termini informativi: converte *dove* in *che cosa*.
2. **Testa di classificazione (classifier head).** Un flatten seguito da uno o più strati fully connected, che comprimono la rappresentazione in un vettore di punteggi lunghi quanto il numero di classi.

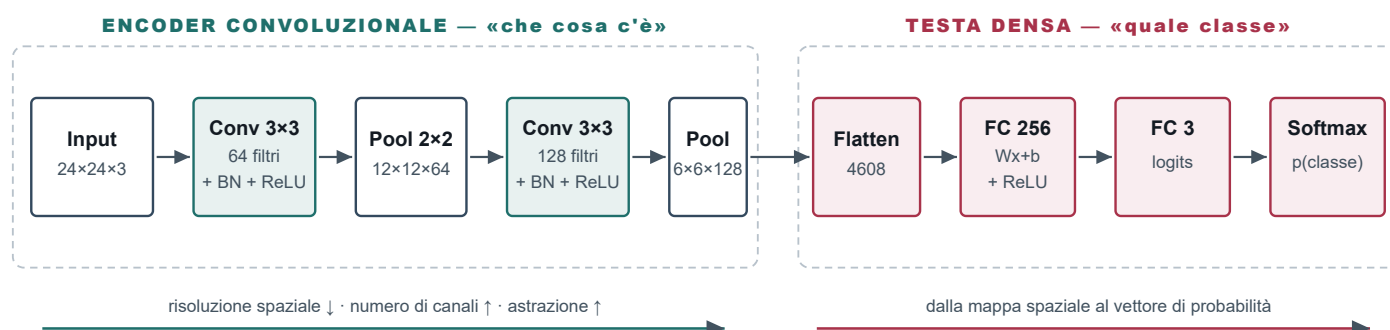


Figura 3.1 — Anatomia canonica di una CNN per classificazione. Il tensore attraversa la rete perdendo progressivamente risoluzione spaziale e guadagnando canali. Nel passaggio al flatten la struttura bidimensionale viene abbandonata definitivamente: è esattamente questo passaggio che la segmentazione dovrà eliminare (→ Cap. 32).

3.2 Il flusso dimensionale come chiave di lettura

Il modo più efficace per leggere qualunque architettura è tracciare la **forma del tensore** a ogni passo. Prendiamo l'esempio della figura, con input $24 \times 24 \times 3$ e padding "same" (→ Cap. 8):

STADIO	OPERAZIONE	FORMA IN USCITA	PARAMETRI
0	Input	$24 \times 24 \times 3$	0
1	Conv 3×3 , 64 filtri, pad 1	$24 \times 24 \times 64$	$64 \cdot (3 \cdot 3 \cdot 3 + 1) = 1\,792$
2	BatchNorm	$24 \times 24 \times 64$	128
3	ReLU	$24 \times 24 \times 64$	0
4	MaxPool 2×2 , stride 2	$12 \times 12 \times 64$	0
5	Conv 3×3 , 128 filtri, pad 1	$12 \times 12 \times 128$	$128 \cdot (3 \cdot 3 \cdot 64 + 1) = 73\,856$
6	BN + ReLU	$12 \times 12 \times 128$	256
7	MaxPool 2×2	$6 \times 6 \times 128$	0
8	Flatten	4 608	0
9	FC 4608 $\rightarrow$ 256	256	1 179 904
10	FC 256 $\rightarrow$ 3	3 (logits)	771
11	Softmax	3 (probabilità)	0

Tabella 3.1 — Tracciamento completo di forme e parametri. Si osservi il dato più istruttivo: **il 94% dei parametri sta nei due strati densi**, che occupano l'ultimo 15% della rete. È il motivo per cui le architetture moderne tendono a ridurre o eliminare la testa densa.

APPROFONDIMENTO • PERCHÉ I CANALI CRESCONO MENTRE LO SPAZIO SI RIDUCE

Esiste un principio di conservazione approssimativa dell'informazione utile. Dimezzando lato e altezza si perde un fattore 4 di risoluzione; raddoppiando i canali si recupera un fattore 2 di capacità rappresentativa per posizione. Il tensore si "assottiglia" spazialmente e si "ispessisce" semanticamente. All'inizio ogni posizione descrive un pixel e serve poca profondità per dirlo; alla fine ogni posizione descrive una regione ampia e serve molta profondità per specificare quale delle centinaia di configurazioni possibili sia presente.

3.3 Che cosa è appreso e che cosa no

COMPONENTE	CONTIENE PARAMETRI APPRESI?	CHE COSA MODIFICA
Convoluzione	Sì: pesi del kernel + bias	Trasformazione lineare locale dell'input
Batch Normalization	Sì: γ, β (+ statistiche non apprese)	Riscalatura e traslazione delle attivazioni
ReLU	No	Solo le attivazioni (azzerà i negativi)
Pooling	No	Solo la dimensione spaziale
Flatten	No	Solo la forma (riordina, non altera i valori)
Fully connected	Sì: matrice W + bias	Combinazione globale di tutte le componenti
Softmax	No	Normalizza i logits in probabilità

Tabella 3.2 — Distinguere gli operatori parametrici da quelli non parametrici è essenziale per capire che cosa la backpropagation aggiorna (→ Cap. 24) e che cosa semplicemente attraversa.

RIEPILOGO DEL CAPITOLO 3

- Una CNN si legge come encoder (estrazione gerarchica di feature) + testa densa (decisione).
- Il tracciamento delle forme del tensore stadio per stadio è lo strumento diagnostico primario.
- La grande maggioranza dei parametri si concentra negli strati densi, non nelle convoluzioni.
- Solo convoluzioni, batch norm e strati densi contengono pesi appresi; ReLU, pooling, flatten e softmax non ne contengono.

4 Tensori

Tutto ciò che entra, attraversa ed esce da una rete neurale è un tensore. Padroneggiare l'ordine degli assi e il significato di ciascuno elimina la quasi totalità degli errori di implementazione e, soprattutto, rende leggibile qualunque architettura.

4.1 Definizione operativa

DEFINIZIONE

Un **tensore** di rango k è un array multidimensionale di numeri, indicizzato da k indici interi. La sua **forma** (*shape*) è la tupla delle dimensioni lungo ciascun asse. Un tensore di forma $(d_1, d_2, \dots, d_k)$ contiene $\prod_i d_i$ valori scalari.

RANGO	NOME	ESEMPIO IN AMBITO CNN	FORMA
0	Scalare	Valore della loss di un batch	()
1	Vettore	Vettore dei logits per 3 classi	(3)
2	Matrice	Immagine in scala di grigi; matrice dei pesi FC	(512, 512)
3	Tensore 3D	Immagine RGB; stack di feature map	(24, 24, 64)
4	Tensore 4D	Mini-batch di immagini; banco di filtri conv 2D	(32, 3, 24, 24)
5	Tensore 5D	Mini-batch di volumi CT (3D)	(2, 1, 128, 128, 128)

4.2 Le due convenzioni di ordinamento degli assi

Esistono due convenzioni incompatibili, ed è indispensabile sapere in quale si sta lavorando:

- **NCHW** (batch, canali, altezza, larghezza) — usata da PyTorch e quindi da nnU-Net.
- **NHWC** (batch, altezza, larghezza, canali) — usata storicamente da TensorFlow.

Il contenuto informativo è identico; cambia solo l'ordine di memorizzazione. Un tensore PyTorch di forma $(32, 3, 24, 24)$ rappresenta 32 immagini RGB 24×24 . Lo stesso dato in NHWC sarebbe $(32, 24, 24, 3)$.

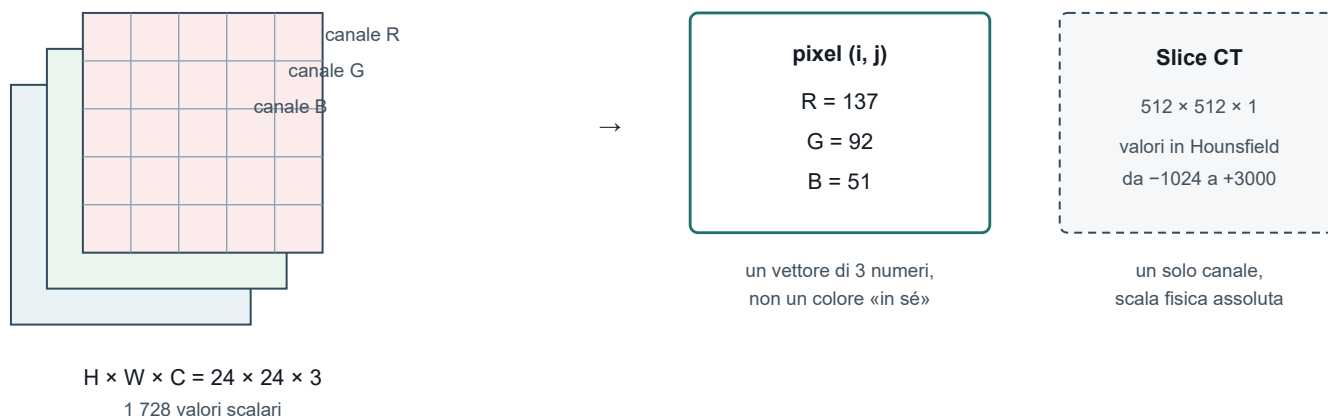


Figura 4.1 — Un'immagine RGB è un tensore di rango 3: tre matrici sovrapposte, una per canale. Ogni pixel è un vettore di tre componenti. Una slice CT ha un solo canale ma, a differenza dell'RGB, i suoi valori hanno significato fisico assoluto.

4.3 Layout in memoria e operazioni di reshape

Un tensore multidimensionale è memorizzato come un blocco contiguo unidimensionale. La corrispondenza è data dagli **stride** di memoria (da non confondere con lo stride convoluzionale del Cap. 9). Per un tensore (C, H, W) in ordine *row-major*, l'elemento (c, h, w) si trova all'offset

$$\text{offset}(c, h, w) = c(H \cdot W) + hW + w$$

Da qui segue una proprietà importante: **reshape e flatten non spostano né alterano dati**, cambiano solo l'interpretazione degli indici. Sono operazioni a costo (quasi) nullo. La *permutazione* degli assi, invece, cambia realmente l'ordine di lettura e può richiedere una copia.

ESEMPIO NUMERICO · CONTARE GLI ELEMENTI

Un mini-batch di 8 volumi CT, singolo canale, ritagliati a 128^3 voxel: forma $(8, 1, 128, 128, 128)$.

$$8 \cdot 1 \cdot 128^3 = 8 \cdot 2\,097\,152 = 16\,777\,216 \text{ valori}$$

In `float32` (4 byte per valore) sono ≈ 64 MB **per il solo tensore di input**. Ogni feature map intermedia con 32 canali alla stessa risoluzione occuperebbe 2 GB. È questa aritmetica — non la teoria — a dettare le dimensioni di patch e batch in segmentazione 3D, ed è il motivo per cui nnU-Net calcola automaticamente la *patch size* in funzione della memoria GPU disponibile.

ERRORI COMUNI

Confondere l'asse dei canali con l'asse del batch. In NCHW l'asse 0 è il batch, l'asse 1 sono i canali. Un errore di permutazione non produce un crash se le dimensioni coincidono per caso: produce silenziosamente risultati privi di senso.

Pensare che "3 canali" significhi "3 dimensioni spaziali". I canali non sono una dimensione spaziale: non c'è alcuna nozione di vicinanza fra il canale R e il canale G. Per questo il kernel scorre solo su H e W , e non lungo C ($\rightarrow$ Cap. 6).

RIEPILOGO DEL CAPITOLO 4

- Un tensore è un array multidimensionale; la forma è la tupla delle dimensioni.
- PyTorch usa NCHW: (batch, canali, altezza, larghezza); i volumi medici aggiungono un asse: NCDHW.
- Reshape e flatten non alterano i valori né l'ordine in memoria: cambiano solo l'indicizzazione.
- Il conteggio degli elementi determina l'occupazione di memoria, che nella pratica 3D è il vincolo progettuale dominante.

5 Immagini RGB e immagini mediche

La convoluzione opera su griglie di numeri, ma la natura di quei numeri cambia radicalmente il modo corretto di pre-processarli. Le differenze fra una fotografia RGB e una CT non sono cosmetiche: hanno conseguenze dirette su normalizzazione, augmentation e interpretazione dei risultati.

5.1 Un'immagine RGB

Un'immagine RGB $24 \times 24 \times 3$ contiene 24 righe, 24 colonne e 3 canali. Ogni pixel (i, j) è la tripla $(I_R(i, j), I_G(i, j), I_B(i, j))$, con valori tipicamente interi in $[0, 255]$ (8 bit per canale).

Punti che vanno chiariti una volta per tutte:

- Il valore numerico è **relativo**: dipende da illuminazione, esposizione, sensore, post-processing. Il pixel di un muro bianco può valere 200 in una foto e 90 in un'altra.
- I tre canali sono **correlati** ma non intercambiabili: la maggior parte dell'informazione strutturale è nella luminanza.
- Non c'è un asse "profondità fisica": i canali sono misure diverse della stessa posizione, non posizioni diverse.

La normalizzazione standard riporta i valori in un intervallo adatto all'ottimizzazione: $x' = (x/255 - \mu)/\sigma$, con μ e σ calcolati sul dataset, canale per canale.

5.2 Un'immagine medica: CT, MRI, e la differenza che conta

	FOTOGRAFIA RGB	CT (TOMOGRAFIA COMPUTERIZZATA)	MRI
Canali	3 (R, G, B)	1 (attenuazione ai raggi X)	1 per sequenza, spesso più sequenze
Scala dei valori	0–255, relativa	Unità Hounsfield, $\approx -1024 \dots +3000$, assoluta	Arbitraria, non comparabile fra scanner
Significato fisico	Nessuno standardizzato	-1000 = aria, 0 = acqua, $+1000$ = osso corticale	Dipende da T1, T2, sequenza, parametri
Struttura	2D	3D (stack di slice) + spaziatura anisotropa	3D, spesso fortemente anisotropa
Metadato critico	—	<i>Spacing</i> in mm/voxel, orientamento, kernel di ricostruzione	Spacing, orientamento, protocollo

Tabella 5.1 — Confronto fra modalità. La colonna CT spiega perché in ambito medico la normalizzazione non è una formalità.

APPROFONDIMENTO • PERCHÉ UNA CT SI NORMALIZZA DIVERSAMENTE

In una CT il valore è una misura fisica: il rene ha tipicamente 30–50 HU, il grasso perirenale circa -100 HU, l'osso oltre $+400$ HU. Questo consente due cose impossibili con l'RGB:

- **Clipping guidato dall'anatomia** (*windowing*): si tronca l'intervallo alla finestra di interesse — per l'addome tipicamente circa $[-200, +300]$ HU — eliminando aria e osso che non aggiungono informazione sul rene e comprimono inutilmente la dinamica.
- **Normalizzazione globale a livello di dataset** anziché per singola immagine: poiché la scala è assoluta, la media e la deviazione standard calcolate sull'intero dataset (o sui soli voxel di primo piano) sono statistiche significative. È esattamente ciò che nnU-Net fa in automatico quando riconosce che la modalità è CT, mentre per la MRI applica una normalizzazione z-score per singolo caso, proprio perché lì la scala non è comparabile fra pazienti.

5.3 Lo spacing: il metadato che non si può ignorare

Un volume CT non è un cubo di numeri privo di unità di misura. Ogni voxel ha una dimensione fisica, per esempio $0.78 \times 0.78 \times 3.0$ mm. Due volumi con lo stesso numero di voxel possono coprire porzioni di corpo molto diverse. Poiché il kernel convoluzionale ragiona in **voxel** e non in **millimetri**, un kernel $3 \times 3 \times 3$ copre $2.3 \times 2.3 \times 9.0$ mm in un caso e magari $1.5 \times 1.5 \times 1.5$ mm in un altro: la stessa rete "vede" strutture di dimensioni fisiche diverse. Per questo il primo passo di ogni pipeline medica seria è il **ricampionamento a spacing comune**. È una forma di normalizzazione geometrica, concettualmente parallela alla normalizzazione delle intensità.

ESEMPIO NUMERICO · DAL DICOM AL TENSORE

Volume renale: $512 \times 512 \times 90$ voxel, spacing (0.78, 0.78, 3.0) mm.

1. **Estensione fisica:** $512 \cdot 0.78 = 399$ mm in x e y; $90 \cdot 3.0 = 270$ mm in z.
2. **Ricampionamento** a spacing isotropo target (1.5, 1.5, 1.5) mm: nuova forma $\lceil 399/1.5 \rceil = 266, 266, \lceil 270/1.5 \rceil = 180 \rightarrow 266 \times 266 \times 180$.
3. **Clipping** a [-200, 300] HU, poi z-score con μ, σ calcolati sui voxel di primo piano del dataset.
4. **Estrazione di patch** 128^3 , perché il volume intero non entra in memoria.
5. **Tensore finale** in ingresso alla rete: $(B, 1, 128, 128, 128)$.

Nessuno di questi passaggi è "preprocessing accessorio": ognuno cambia ciò che la convoluzione vedrà, e quindi cambia il risultato.

ERRORI COMUNI

Trattare una CT come un'immagine a 8 bit. Convertire a [0,255] prima del clipping distrugge la dinamica diagnostica: l'intero intervallo dei tessuti molli, che occupa poche decine di HU, viene schiacciato in due o tre livelli.

Ignorare lo spacing anisotropo. Applicare kernel isotropi 3^3 su un volume con spacing z sei volte maggiore significa fare convoluzione su un campo recettivo fisicamente deformato. Le architetture mediche moderne, anche per questo, usano kernel anisotropi o effettuano pooling solo su alcuni assi.

Copiare il canale in grigi tre volte per usare una rete pre-addestrata su ImageNet. È una pratica diffusa e talvolta accettabile, ma non crea informazione: triplica solo il costo del primo strato.

RIEPILOGO DEL CAPITOLO 5

- RGB: 3 canali, valori relativi, nessun significato fisico assoluto.
- CT: 1 canale, unità Hounsfield con significato fisico; consente clipping anatomico e normalizzazione a livello di dataset.
- MRI: intensità non comparabili fra acquisizioni; richiede normalizzazione per singolo caso.
- Lo spacing traduce voxel in millimetri: senza ricampionamento, il campo recettivo del kernel non è confrontabile fra pazienti.

6 Kernel

Il kernel è l'unità elementare di conoscenza di una CNN. Tutto ciò che la rete "sa" sulla struttura locale delle immagini è codificato in poche decine di numeri per filtro. Questo capitolo definisce con precisione che cosa sono quei numeri, quanti sono, e perché la loro forma non è quella che il nome suggerisce.

6.1 Definizione e forma reale

DEFINIZIONE

Un **kernel** (o filtro) di uno strato convoluzionale 2D è un tensore di rango 3 di forma (k_h, k_w, C_{in}) , accompagnato da un **bias** scalare. Tutti i suoi elementi sono parametri appresi. Il kernel definisce una singola mappa di uscita: uno strato con C_{out} filtri contiene C_{out} kernel distinti, e i suoi parametri formano complessivamente un tensore di rango 4 di forma $(C_{out}, C_{in}, k_h, k_w)$ più un vettore di C_{out} bias.

Quando si dice "filtro 3×3 " si nomina soltanto la sua estensione *spaziale*. La terza dimensione non è opzionale ed è imposta dall'input: un filtro deve attraversare **tutti** i canali in ingresso, perché deve poter combinare informazione proveniente da canali diversi nella stessa posizione.

- Su input RGB ($C_{in} = 3$): un filtro " 3×3 " è in realtà $3 \times 3 \times 3 = 27$ pesi + 1 bias = **28 parametri**.
- Su input CT a singolo canale ($C_{in} = 1$): $3 \times 3 \times 1 = 9$ pesi + 1 bias = **10 parametri**.
- Su una feature map intermedia con $C_{in} = 64$: $3 \times 3 \times 64 = 576$ pesi + 1 bias = **577 parametri**.

Quest'ultimo caso merita attenzione: la profondità del filtro cresce con la profondità della rete. Non è il filtro a diventare "più grande" spazialmente, ma il numero di canali che deve fondere.

$$\# \text{parametri di uno strato conv} = C_{out} \cdot (k_h \cdot k_w \cdot C_{in} + 1)$$

6.2 Che cosa contiene un kernel: dai filtri classici a quelli appresi

Prima del deep learning, i kernel venivano scritti a mano. Alcuni esempi storici, su singolo canale:

NOME	KERNEL	EFFETTO
Sobel verticale	$\begin{bmatrix} -1, 0, 1 \\ -2, 0, 2 \\ -1, 0, 1 \end{bmatrix}$	Risponde a variazioni orizzontali di intensità: rileva bordi verticali
Sobel orizzontale	$\begin{bmatrix} -1, -2, -1 \\ 0, 0, 0 \\ 1, 2, 1 \end{bmatrix}$	Rileva bordi orizzontali
Laplaciano	$\begin{bmatrix} 0, 1, 0 \\ 1, -4, 1 \\ 0, 1, 0 \end{bmatrix}$	Risponde a variazioni del secondo ordine: contorni e punti isolati
Media (box blur)	$(1/9) \cdot \begin{bmatrix} 1, 1, 1 \\ 1, 1, 1 \\ 1, 1, 1 \end{bmatrix}$	Attenua il rumore, sfuoca
Identità	$\begin{bmatrix} 0, 0, 0 \\ 0, 1, 0 \\ 0, 0, 0 \end{bmatrix}$	Copia l'input

Tabella 6.1 — Kernel classici. Una CNN può apprendere questi filtri, e nei primi strati spesso apprende qualcosa di molto simile; ma non è vincolata a farlo, e i filtri che apprende sono ottimizzati per la loss, non per l'estetica.

INTUIZIONE • PERCHÉ UN KERNEL «RILEVA» QUALCOSA

La convoluzione, in ogni posizione, calcola un prodotto scalare fra il kernel e la patch di input. Il prodotto scalare è massimo quando i due vettori sono allineati. Quindi il kernel funziona come un **modello di riferimento**: la risposta è tanto più alta quanto più la patch locale assomiglia (a meno di scala) al pattern codificato nei pesi. Un kernel con valori negativi a sinistra e positivi a destra dà risposta massima dove l'intensità cresce da sinistra a destra, cioè su un bordo verticale con un dato verso. Sul bordo di verso opposto la risposta è fortemente negativa, e sarà poi azzerata da ReLU (→ Cap. 12): è così che una coppia filtro+ReLU si specializza su un verso preciso.

6.3 Inizializzazione: da dove partono i pesi

I pesi non partono da zero (produrrebbe gradienti identici per tutti i filtri, che resterebbero indistinguibili per sempre: il cosiddetto problema della *simmetria*) né da valori arbitrariamente grandi. Si usano schemi che mantengono varianza costante attraverso gli strati:

$$\text{He (adatta a ReLU): } w \sim \mathcal{N}\left(0, \frac{2}{k_h k_w C_{in}}\right) \quad \text{Xavier/Glorot: } w \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

Il denominatore $n_{in} = k_h k_w C_{in}$ è il *fan-in*: il numero di ingressi che concorrono a una singola uscita. L'idea è che la somma di n_{in} termini indipendenti abbia varianza $n_{in} \sigma^2$; scegliendo $\sigma^2 \propto 1/n_{in}$ la varianza dell'attivazione resta stabile strato dopo strato, evitando che il segnale svanisca o esploda in propagazione. Il fattore 2 nella formula di He compensa il fatto che ReLU azzerà metà delle attivazioni.

ERRORI COMUNI

«Un filtro 3×3 ha 9 pesi.» Solo se $C_{in} = 1$. In generale ne ha $9 \cdot C_{in}$.

«Ogni filtro guarda un canale diverso.» No. *Ogni* filtro guarda *tutti* i canali. Ciò che distingue i filtri fra loro non è il canale che osservano, ma i pesi che applicano.

«Il kernel scorre anche lungo i canali.» No. Scorre solo su altezza e larghezza. Lungo l'asse dei canali è esteso per intero e statico, perché i canali non hanno struttura di vicinanza (→ Cap. 4).

RIEPILOGO DEL CAPITOLO 6

- Un kernel ha forma (k_h, k_w, C_{in}) : la profondità è imposta dall'input, non scelta.
- Parametri di uno strato: $C_{out}(k_h k_w C_{in} + 1)$.
- La convoluzione è un prodotto scalare: il kernel è un modello di pattern e la risposta misura la somiglianza.
- L'inizializzazione (He per ReLU) preserva la varianza del segnale e rompe la simmetria fra filtri.

7 Convoluzione

Questo è il capitolo centrale della dispensa. Tutto ciò che precede serve a definirne i termini; tutto ciò che segue ne è una conseguenza o una variazione. L'obiettivo è arrivare a saper calcolare a mano, senza esitazione, il valore di un elemento qualsiasi di una feature map.

7.1 Formulazione matematica

Sia $X \in \mathbb{R}^{H \times W \times C_{in}}$ il tensore di ingresso, $K \in \mathbb{R}^{k_h \times k_w \times C_{in}}$ un kernel e $b \in \mathbb{R}$ il suo bias. L'elemento in posizione (i, j) della mappa di uscita, con stride 1 e senza padding, è

$$Y(i, j) = b + \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u, v, c) \cdot X(i+u, j+v, c)$$

Tre somme annidate: sui canali, sulle righe del kernel, sulle colonne del kernel. Il risultato è **un singolo numero reale**. Facendo variare (i, j) su tutte le posizioni valide si ottiene la mappa completa.

APPROFONDIMENTO • CONVOLUZIONE O CORRELAZIONE INCROCIATA?

In analisi matematica la convoluzione vera prevede il ribaltamento del kernel: $\sum_u K(u) X(i-u)$. Le librerie di deep learning implementano invece la **correlazione incrociata**, cioè la formula scritta sopra, senza ribaltamento. La differenza è irrilevante nell'apprendimento: poiché i pesi sono appresi, la rete può semplicemente imparare il kernel ribaltato. Il nome "convoluzione" è rimasto per ragioni storiche. È utile saperlo per non confondersi leggendo un testo di elaborazione dei segnali.

7.2 Il calcolo passo per passo

L'operazione si svolge in cinque passi, per *ogni* posizione della finestra:

1. **Posizionamento.** La finestra $k_h \times k_w \times C_{in}$ si allinea a un punto dell'input.
2. **Estrazione della patch.** Si legge il sotto-tensore corrispondente.
3. **Prodotto elemento per elemento** fra patch e kernel (stessa forma, moltiplicazione posizione per posizione).
4. **Somma** di tutti i prodotti: $k_h \cdot k_w \cdot C_{in}$ termini ridotti a uno scalare.
5. **Aggiunta del bias.** Il risultato è $Y(i, j)$.

Poi la finestra si sposta secondo lo stride ($\rightarrow$ Cap. 9) e si ripete. È essenziale essere consapevoli che **i pesi non cambiano durante lo scorrimento**: è lo stesso identico kernel applicato in tutte le posizioni. Questa è la condivisione dei pesi del Cap. 2, vista dal basso.

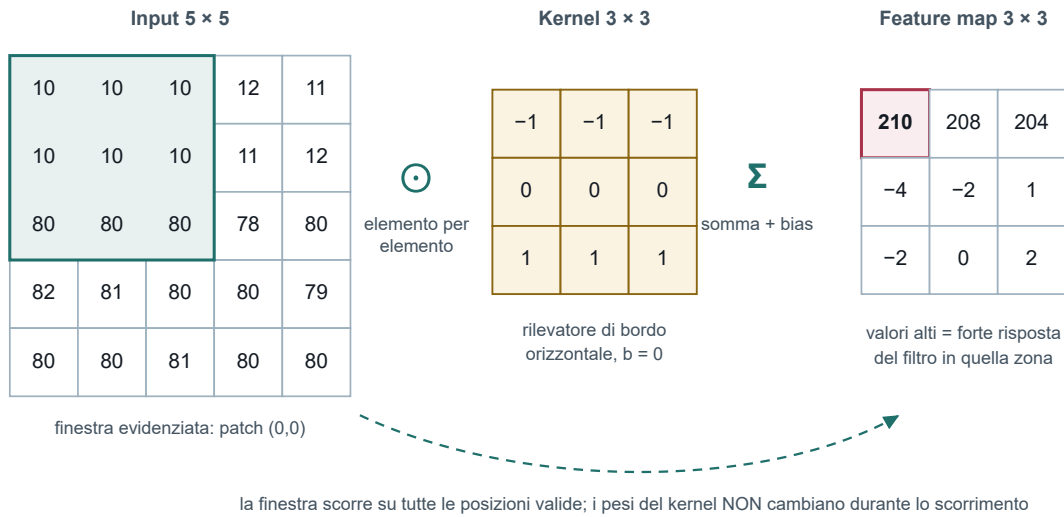


Figura 7.1 — Un passo di convoluzione. La patch 3×3 in alto a sinistra dell'input contiene due righe di valore ≈ 10 e una riga di valore ≈ 80 : è un bordo orizzontale netto. Il kernel risponde con 210, valore molto più alto di tutti gli altri elementi della mappa. Il calcolo esplicito è: $(-1)(10 + 10 + 10) + 0 \cdot (10 + 10 + 10) + (1)(80 + 80 + 80) = -30 + 240 = 210$.

7.3 Esempio numerico completo su input RGB

ESEMPIO NUMERICO · CONVOLUZIONE 3×3×3 CON BIAS

Patch di input (3×3 per ciascuno dei tre canali):

CANALE R	CANALE G	CANALE B
1 2 3	2 2 2	5 5 5
4 5 6	2 2 2	0 0 0
7 8 9	2 2 2	5 5 5

Kernel (3×3 per ciascun canale), bias $b = -4$:

SLICE R DEL KERNEL	SLICE G	SLICE B
1 0 -1	0 1 0	1 1 1
1 0 -1	1 -4 1	1 1 1
1 0 -1	0 1 0	1 1 1

Contributo del canale R. Colonna 1 pesata +1: $1 + 4 + 7 = 12$. Colonna 2 pesata 0: 0. Colonna 3 pesata -1: $-(3 + 6 + 9) = -18$.

$$S_R = 12 + 0 - 18 = -6$$

Contributo del canale G. Tutti i valori sono 2, e la somma dei pesi del kernel laplaciano è $0 + 1 + 0 + 1 - 4 + 1 + 0 + 1 + 0 = 0$:

$$S_G = 2 \cdot 0 = 0$$

Osservazione importante: un kernel a somma nulla è insensibile all'offset costante. Su una regione uniforme risponde zero, per qualunque livello di intensità. È il comportamento tipico dei rilevatori di variazione.

Contributo del canale B. Tutti i pesi valgono 1, quindi si somma la patch: $15 + 0 + 15 = 30$.

$$S_B = 30$$

Somma e bias:

$$Y(0,0) = S_R + S_G + S_B + b = -6 + 0 + 30 - 4 = \mathbf{20}$$

Questo **unico numero** è l'elemento (0,0) della feature map prodotta da questo filtro. Passerà poi attraverso la non linearità: $\text{ReLU}(20) = 20$, quindi sopravvive. Se fosse stato -20 sarebbe stato azzerato.

7.4 Dimensione dell'uscita

La formula generale, con padding p e stride s (capitoli 8 e 9), è

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k_h}{s} \right\rfloor + 1 \quad W_{out} = \left\lfloor \frac{W_{in} + 2p - k_w}{s} \right\rfloor + 1 \quad C_{out} = \# \text{filtri}$$

Va imparata a memoria: è lo strumento con cui si verifica qualunque architettura senza eseguirla. Casi notevoli, con $k = 3$:

H_{IN}	K	P	S	H_{OUT}	COMMENTO
24	3	0	1	22	Senza padding si perdono 2 pixel per lato pari a $k - 1$
24	3	1	1	24	Padding "same": dimensione preservata
24	3	1	2	12	Dimezzamento tramite stride
24	5	2	1	24	Same con $k = 5$ richiede $p = 2$
512	3	1	2	256	Primo downsampling di un encoder su slice CT

Regola pratica: per ottenere padding "same" con stride 1 e kernel dispari, $p = (k - 1)/2$.

7.5 Costo computazionale

Il numero di moltiplicazioni-accumulazioni (MAC) di uno strato convoluzionale è

$$\text{MAC} = H_{out} \cdot W_{out} \cdot C_{out} \cdot (k_h \cdot k_w \cdot C_{in})$$

Per il primo strato del nostro esempio ($24 \times 24 \times 64$ in uscita, kernel $3 \times 3 \times 3$):

$$24 \cdot 24 \cdot 64 \cdot 27 = 995\,328 \approx 10^6 \text{ MAC}$$

con soli 1792 parametri. Ecco il tratto distintivo della convoluzione: **pochi parametri, molte operazioni**. Lo stesso peso viene riutilizzato $H_{out} \cdot W_{out}$ volte. Uno strato denso ha invece tanti parametri quante operazioni. Questa asimmetria spiega perché le CNN sono efficienti in memoria ma pesanti in calcolo, e perché beneficiano così tanto delle GPU.

ERRORI COMUNI

«La convoluzione somma i canali di uscita.» No. Somma i canali di *ingresso*, dentro un singolo filtro. I canali di uscita restano separati: uno per filtro, e non vengono mai sommati fra loro.

«Il bias è uno per peso.» No: uno per filtro, cioè uno per canale di uscita. Uno strato con 64 filtri ha 64 bias, non $64 \cdot 27$.

«La convoluzione è non lineare.» No: è un'operazione perfettamente lineare nei suoi ingressi (affine, considerando il bias). La non linearità arriva soltanto con la funzione di attivazione. Senza di essa, impilare cento convoluzioni equivarrebbe a una sola convoluzione con kernel più grande.

RIEPILOGO DEL CAPITOLO 7

- Convoluzione = prodotto elemento per elemento fra kernel e patch, somma su tutti i canali, più bias → un solo scalare.
- Lo stesso kernel viene applicato in tutte le posizioni: i pesi non variano durante lo scorrimento.
- $H_{out} = \lfloor (H_{in} + 2p - k)/s \rfloor + 1$; il numero di canali in uscita è il numero di filtri.
- L'operazione è lineare; il costo è alto in MAC e basso in parametri, grazie alla condivisione dei pesi.

8 Padding

Il padding è l'operazione più semplice della dispensa e insieme una di quelle su cui si costruiscono più fraintendimenti. Serve a risolvere un problema puramente geometrico: senza di esso il kernel non può essere centrato sui pixel di bordo.

8.1 Il problema

Un kernel 3×3 centrato sul pixel $(0, 0)$ richiederebbe l'esistenza dei pixel $(-1, -1)$, $(-1, 0)$, $(0, -1)$, che non esistono. Le conseguenze, senza padding, sono due:

- **Riduzione dimensionale:** ogni convoluzione 3×3 accorcia il tensore di 2 pixel per asse. Dopo 10 strati un'immagine 24×24 diventa 4×4 ; dopo 12 strati non esiste più.
- **Sottocampionamento del bordo:** il pixel centrale partecipa a 9 finestre diverse, quello d'angolo a una sola. Il bordo contribuisce meno alle attivazioni, e il gradiente che vi ritorna è proporzionalmente più debole.

DEFINIZIONE

Il **padding** è l'aggiunta di p righe e colonne artificiali attorno al tensore di ingresso, su ogni lato, prima di applicare la convoluzione. Con *zero padding* i valori aggiunti sono nulli. Il padding non introduce parametri appresi e non modifica i valori originali: aggiunge soltanto una cornice.

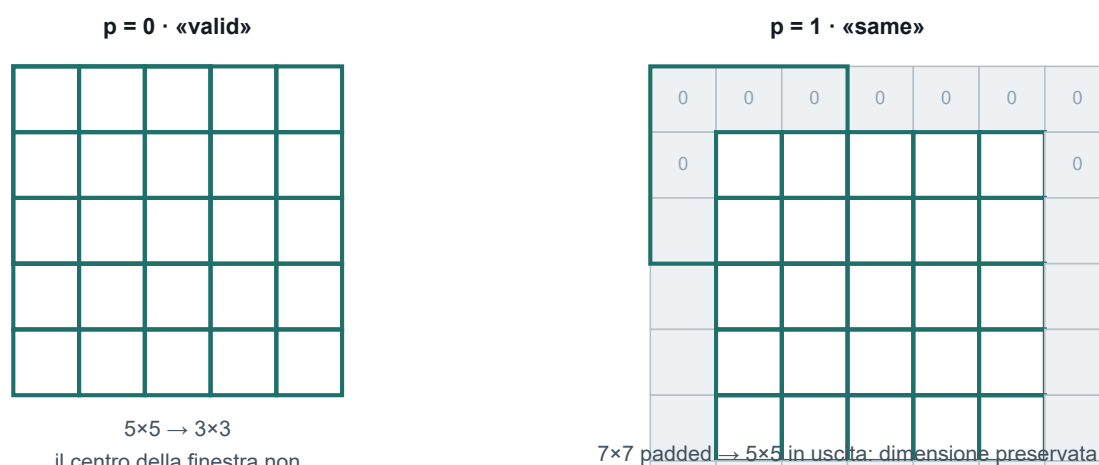


Figura 8.1 — A sinistra, senza padding, la finestra 3×3 può essere centrata solo sui 9 pixel interni: l'uscita è 3×3 . A destra, con una cornice di zeri ($p = 1$), la finestra può essere centrata anche sul pixel d'angolo e l'uscita torna 5×5 .

8.2 Le tre modalità

MODALITÀ	PADDING	USCITA (S = 1)	USO TIPICO
valid	$p = 0$	$H - k + 1$	Quando si vuole scartare esplicitamente il bordo (U-Net originale)
same	$p = (k - 1)/2$	H	Impostazione predefinita nella quasi totalità delle architetture moderne
full	$p = k - 1$	$H + k - 1$	Raro; compare nell'analisi della convoluzione trasposta (→ Cap. 33)

8.3 Che cosa il padding NON fa

ERRORI COMUNI • IL FRAINTENDIMENTO PRINCIPALE

«Il padding serve a dire alla rete dove si trova il bordo dell'immagine.» Non è la sua funzione. Il padding esiste per una ragione geometrica: consentire al kernel di essere applicato anche in posizioni periferiche, preservando la dimensione spaziale. Che poi la rete possa *indirettamente* sfruttare la cornice di zeri come indizio di posizione è un effetto collaterale documentato in letteratura, non l'obiettivo di progetto.

«Gli zeri sono neutri.» Non del tutto. Uno zero è un valore come un altro per il kernel, e su dati normalizzati con z-score lo zero corrisponde alla media del dataset, non a "nulla". Su una CT, dopo la normalizzazione, uno zero di padding rappresenta un tessuto di intensità media: un artefatto. È il motivo per cui esistono alternative come il *reflection padding* (specchia i pixel di bordo) e il *replication padding* (ripete l'ultimo pixel), spesso preferite in ambito medico.

«Il padding aggiunge informazione.» Mai. Aggiunge posizioni, non contenuto. Il contenuto informativo del tensore è invariato.

ESEMPIO NUMERICO • PERCHÉ "SAME" È QUASI OBBLIGATORIO IN SEGMENTAZIONE

Rete profonda 20 strati conv 3×3 , input 256×256 .

Senza padding: ogni strato toglie 2 pixel per asse $\rightarrow 256 - 40 = 216$. L'uscita è 216×216 : non è più allineata all'input, e per produrre una maschera pixel a pixel occorrerebbe ritagliare il ground truth. È esattamente il motivo per cui la U-Net originale del 2015 richiede il *cropping* delle skip connection.

Con padding "same": l'uscita resta 256×256 e ogni pixel di uscita corrisponde a un preciso pixel di ingresso. La corrispondenza uno a uno con il ground truth è automatica. Le implementazioni moderne, nnU-Net compreso, usano padding "same" proprio per questo.

RIEPILOGO DEL CAPITOLO 8

- Il padding aggiunge una cornice artificiale per permettere al kernel di raggiungere il bordo.
- $p = (k - 1)/2$ con stride 1 preserva la dimensione ("same").
- Non aggiunge informazione e non ha parametri appresi; con dati normalizzati gli zeri non sono neutri.
- In segmentazione il padding "same" garantisce corrispondenza pixel-a-pixel fra output e ground truth.

9 Stride

Lo stride è il passo con cui la finestra si sposta. È l'altro meccanismo, insieme al pooling, con cui una rete riduce la risoluzione spaziale — e nelle architetture recenti è quello preferito.

DEFINIZIONE

Lo **stride** s è il numero di pixel di cui la finestra convoluzionale trasla fra due applicazioni successive. Con $s = 1$ la finestra si sposta di un pixel e le finestre adiacenti si sovrappongono per $k - 1$ colonne. Con $s = 2$ si sposta di due pixel, valutando una posizione su due.



Formula unificata: $H_{out} = \lfloor (H + 2p - k)/s \rfloor + 1$

stride 2 ≈ dimezzamento della risoluzione, con costo computazionale ridotto di circa 4×

Figura 9.1 — Stride 1 contro stride 2 su una riga di 7 pixel con kernel 3. Con $s = 2$ si valutano meno posizioni: si perde risoluzione ma si guadagna campo recettivo e velocità.

9.1 Effetti dello stride

GRANDEZZA	S = 1	S = 2
Risoluzione in uscita	Preservata (con padding same)	Circa dimezzata per asse
Costo computazionale	Riferimento	≈ 1/4 (in 2D), ≈ 1/8 (in 3D)
Parametri	Invariati	Invariati — lo stride non cambia il kernel
Campo recettivo per strato successivo	Crescita lineare	Crescita moltiplicata
Rischio	Costo elevato	Aliasing, perdita di dettagli fini

ESEMPIO NUMERICO · UN ENCODER CON STRIDE

Input CT 512×512 , kernel 3×3 , padding 1, stride 2 a ogni blocco:

$$512 \xrightarrow{s=2} 256 \xrightarrow{s=2} 128 \xrightarrow{s=2} 64 \xrightarrow{s=2} 32 \xrightarrow{s=2} 16$$

Verifica del primo passo con la formula: $\lfloor (512 + 2 - 3)/2 \rfloor + 1 = \lfloor 511/2 \rfloor + 1 = 255 + 1 = 256$. ✓

Cinque livelli di downsampling portano da $512^2 = 262\,144$ posizioni a $16^2 = 256$ posizioni: una riduzione di 1024 volte. In parallelo i canali crescono, tipicamente $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 320$. È lo schema esatto dell'encoder di nnU-Net.

APPROFONDIMENTO · STRIDE CONTRO POOLING

Convoluzione con $s = 2$ e max pooling 2×2 riducono entrambi la risoluzione della metà, ma non sono la stessa cosa:

- La convoluzione con stride è **parametrica**: la rete apprende *come* ridurre, cioè quale combinazione lineare della finestra conservare.
- Il pooling è **fisso**: prende sempre il massimo (o la media), qualunque sia il contenuto.

Le architetture recenti — nnU-Net compreso — tendono a usare convoluzioni con stride al posto del pooling, perché la riduzione appresa preserva meglio l'informazione utile al decoder. Il prezzo è un modesto aumento di parametri e la perdita della semplice invarianza locale garantita dal max pooling.

ERRORI COMUNI

«Stride 2 significa saltare due pixel.» No: significa spostarsi *di* due pixel, quindi saltarne uno. Lo stride è la traslazione, non il numero di elementi ignorati.

«Con stride 2 metà dei pixel non viene mai vista.» Falso con $k > s$: con $k = 3$, $s = 2$ le finestre si sovrappongono di un pixel, quindi ogni pixel di input entra in almeno una finestra. Solo se $s \geq k$ si formano zone cieche.

RIEPILOGO DEL CAPITOLO 9

- Lo stride è il passo di traslazione della finestra; non altera il numero di parametri.
- $s = 2$ dimezza la risoluzione e riduce il calcolo di circa $4\times$ in 2D e $8\times$ in 3D.
- Il downsampling con stride è appreso, quello con pooling è fisso.
- Con $k > s$ nessun pixel viene ignorato: le finestre si sovrappongono.

10 Feature Map

La feature map è l'oggetto che una CNN produce, propaga e trasforma. Comprendere che cosa rappresentano i suoi valori — e soprattutto che cosa *non* rappresentano — è il passo che separa l'uso meccanico di una libreria dalla comprensione reale.

DEFINIZIONE

La **feature map** (mappa di attivazione) prodotta da un filtro è la matrice $Y \in \mathbb{R}^{H_{out} \times W_{out}}$ i cui elementi sono le risposte di quel filtro in tutte le posizioni valide dell'input. Uno strato con C_{out} filtri produce C_{out} feature map, impilate lungo l'asse dei canali in un tensore $H_{out} \times W_{out} \times C_{out}$.

10.1 Il significato dei valori

Un elemento $Y(i, j)$ è un numero reale con questa interpretazione, e nessun'altra:

Quanto la regione dell'input coperta dal campo recettivo di (i, j) è allineata con il pattern codificato in questo filtro.

Da qui discendono alcune conseguenze che vale la pena esplicitare:

- **Non sono colori.** Non c'è alcuna relazione con R, G o B. Quando si visualizza una feature map in scala di grigi, si sta usando un artificio grafico, non leggendo un'immagine.
- **Non hanno un'unità di misura assoluta.** Non esiste una tabella per cui "5 = bordo" e "2 = texture". Il valore 5 è alto o basso solo relativamente alla distribuzione dei valori in quella stessa mappa.
- **Sono relative al filtro.** Un valore alto nella mappa 17 e un valore alto nella mappa 42 significano cose diverse, perché i due filtri codificano pattern diversi.
- **Il segno conta.** Un valore fortemente negativo indica anti-allineamento: il pattern opposto. Dopo ReLU questa informazione viene scartata (→ Cap. 12).

INTUIZIONE · UNA MAPPA DI «DOVE», NON DI «CHE COSA»

Un modo utile di pensarci: ogni filtro pone una domanda fissa ("c'è un bordo con questo orientamento?") e la feature map è la risposta, data separatamente per ogni posizione dell'immagine. Le 64 feature map di uno strato sono 64 risposte alla stessa immagine, ottenute con 64 domande diverse. La struttura spaziale è conservata: se un bordo si trova nell'angolo in alto a sinistra dell'immagine, l'attivazione alta comparirà nell'angolo in alto a sinistra della mappa. Questa conservazione della corrispondenza spaziale è ciò che rende possibile la segmentazione.

10.2 Che cosa rappresentano le mappe ai diversi livelli di profondità

LIVELLO	RISOLUZIONE TIPICA	CANALI	CHE COSA ATTIVANO I FILTRI
1 (superficiale)	512 × 512	32	Bordi con orientamento specifico, gradienti di intensità, transizioni aria/tessuto
2	256 × 256	64	Angoli, giunzioni, texture regolari, curvature
3	128 × 128	128	Motivi complessi, bordi curvi di organi, pattern parenchimali
4	64 × 64	256	Parti di organo: polo renale, seno renale, interfaccia con il grasso
5 (profondo)	32 × 32	320	Contesto anatomico: configurazione retroperitoneale, relazione con fegato e milza

Tabella 10.1 — Descrizione qualitativa, ricavata da tecniche di visualizzazione delle attivazioni. Nessuno impone questa gerarchia: emerge dall'ottimizzazione.

10.3 Il campo recettivo

Il **campo recettivo** di un elemento della feature map è la regione dell'immagine di ingresso che può influenzarne il valore. È la grandezza che quantifica "quanto contesto vede" un neurone. Per una pila di strati con kernel k_ℓ e stride s_ℓ , il campo recettivo cresce secondo

$$R_\ell = R_{\ell-1} + (k_\ell - 1) \cdot \prod_{m<\ell} s_m \quad \text{con } R_0 = 1$$

ESEMPIO NUMERICO • CRESCITA DEL CAMPO RECETTIVO

Tre convoluzioni 3×3 con stride 1, senza pooling:

$$R_1 = 1 + 2 \cdot 1 = 3; R_2 = 3 + 2 \cdot 1 = 5; R_3 = 5 + 2 \cdot 1 = 7.$$

Con un pooling di fattore 2 dopo il primo strato:

$$R_1 = 3; \text{dopo il pool } \prod s = 2, \text{ quindi } R_2 = 3 + 2 \cdot 2 = 7; R_3 = 7 + 2 \cdot 2 = 11.$$

Lezione: il downsampling è il moltiplicatore del campo recettivo. Senza ridurre la risoluzione, per vedere un rene intero (diciamo 200 pixel) servirebbero un centinaio di strati 3×3 . Con cinque livelli di downsampling ne bastano una decina. Questa è la vera ragione architeturale per cui esiste l'encoder.

APPROFONDIMENTO · DUE 3×3 VALGONO UN 5×5 , MA COSTANO MENO

Due convoluzioni 3×3 in serie hanno campo recettivo 5×5 , come una singola convoluzione 5×5 .

Confrontiamo i parametri, con $C_{in} = C_{out} = C$:

$$\text{due } 3 \times 3 : 2 \cdot 9C^2 = 18C^2 \quad \text{una } 5 \times 5 : 25C^2$$

Il 28% di parametri in meno, e in più una non linearità aggiuntiva fra le due convoluzioni, che aumenta la capacità espressiva. Questa osservazione, introdotta con VGG nel 2014, è il motivo per cui il kernel 3×3 è oggi lo standard quasi universale.

ERRORI COMUNI

«Le feature map vengono confrontate con il ground truth.» Assolutamente no. Le feature map sono rappresentazioni intermedie; nessuno sa a priori che aspetto "dovrebbero" avere e nessuna etichetta le riguarda. Il confronto con il ground truth avviene una sola volta, sull'output finale della rete (→ Cap. 20).

«Ogni feature map corrisponde a un concetto interpretabile.» Solo approssimativamente e solo nei primi strati. In profondità le rappresentazioni sono distribuite: un concetto è codificato dalla configurazione congiunta di molti canali, non da un canale singolo.

RIEPILOGO DEL CAPITOLO 10

- Una feature map contiene la risposta di un filtro in ogni posizione; conserva la struttura spaziale dell'input.
- I valori non sono colori e non hanno significato assoluto: misurano allineamento con il pattern del filtro.
- Il campo recettivo cresce con la profondità e viene moltiplicato da ogni downsampling.
- Due 3×3 in serie equivalgono a un 5×5 con meno parametri e una non linearità in più.

11 Filtri multipli

Un solo filtro produce una descrizione monodimensionale e povera. La potenza di uno strato convoluzionale nasce dal fatto che decine o centinaia di filtri operano **in parallelo**, ciascuno interrogando l'input su un aspetto diverso.

11.1 Parallelismo, non sequenzialità

Tutti i filtri di uno strato leggono lo **stesso** tensore di ingresso e producono ciascuno una feature map indipendente. Non c'è alcun ordine: il filtro 42 non riceve l'uscita del filtro 41. La sequenzialità esiste solo *fra* strati, mai *dentro* uno strato.

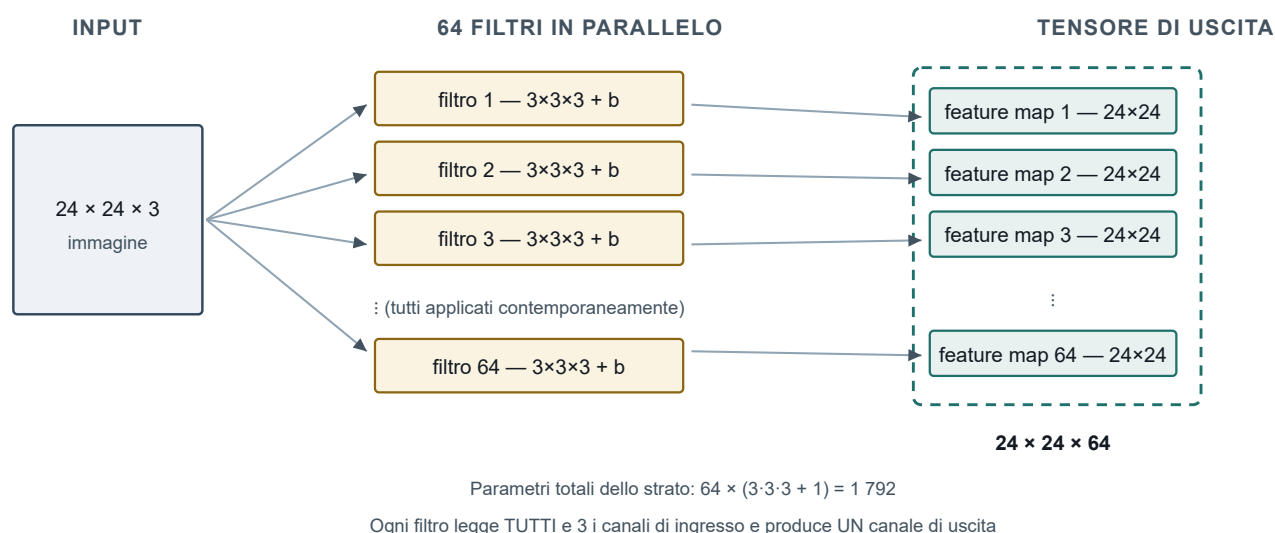


Figura 11.1 — I filtri di uno strato lavorano in parallelo sullo stesso input. Il numero di filtri determina il numero di canali in uscita; nessun filtro vede l'uscita di un altro.

11.2 Come si differenziano i filtri

All'inizio i filtri sono inizializzati con valori casuali indipendenti: sono tutti diversi ma tutti inutili. Durante l'addestramento la differenziazione si accentua per una ragione precisa: **filtri identici ricevono gradienti identici** e non si separerebbero mai, ma filtri leggermente diversi ricevono gradienti diversi, e la loss scende più rapidamente se essi si specializzano su aspetti complementari dell'input. La ridondanza non viene esplicitamente punita, ma non viene nemmeno premiata: emerge naturalmente una copertura diversificata.

INTUIZIONE · LA SQUADRA DI SPECIALISTI

Immaginiamo 64 osservatori che guardano la stessa slice CT, ciascuno con un'unica domanda in mente: "c'è una transizione verticale chiaro-scuro?", "c'è una texture granulare?", "c'è un arco convesso?". Ognuno riempie una griglia con la propria risposta, posizione per posizione. Alla fine si ottengono 64 griglie sovrapposte. Il secondo strato non guarda più l'immagine: guarda le 64 risposte, e pone domande sulle *combinazioni* di risposte — "dove coesistono un arco convesso e una texture granulare?". La gerarchia nasce esattamente così.

11.3 Come si sceglie il numero di filtri

Non esiste una regola derivabile da principi primi; esistono convenzioni consolidate e verificate empiricamente:

- **Raddoppio a ogni downsampling:** $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 320$ (limite superiore in nnU-Net per contenere la memoria). Compensa la perdita di risoluzione con maggiore ricchezza semantica.
- **Pochi filtri nei primi strati:** il numero di pattern locali elementari è limitato; non servono 512 rilevatori di bordo.
- **Vincolo di memoria:** l'attivazione occupa $H \cdot W \cdot C \cdot 4$ byte per immagine del batch, e questa memoria va conservata per tutta la durata del forward, perché serve al backward ($\rightarrow$ Cap. 24).

ESEMPIO NUMERICO · MEMORIA DELLE ATTIVAZIONI

Batch di 8 slice 512×512 , primo strato con 32 canali, float32:

$$8 \cdot 512 \cdot 512 \cdot 32 \cdot 4 \text{ B} = 268\,435\,456 \text{ B} \approx 268 \text{ MB}$$

Per un solo strato. Sommando tutti gli strati e i tensori intermedi conservati per il backward, si arriva rapidamente a diversi gigabyte. È qui che si esaurisce la memoria della GPU, non nei pesi (che per un'intera U-Net 2D sono tipicamente meno di 100 MB).

ERRORI COMUNI

«I filtri si applicano uno dopo l'altro.» No, sono paralleli e indipendenti dentro lo strato.

«Più filtri = rete migliore.» Oltre una certa soglia i filtri diventano ridondanti: si osservano coppie quasi identiche a meno di un fattore di scala. Si pagano memoria e overfitting senza guadagno.

«Ogni filtro produce un'immagine RGB.» No: ogni filtro produce **una** mappa a un solo canale. Sono i 64 canali di uscita, tutti insieme, a formare il tensore.

RIEPILOGO DEL CAPITOLO 11

- I filtri di uno strato operano in parallelo sullo stesso input; ogni filtro produce esattamente un canale di uscita.
- La specializzazione dei filtri emerge dall'ottimizzazione, non da un vincolo esplicito.
- Convenzione standard: raddoppiare i canali a ogni dimezzamento della risoluzione.
- Il collo di bottiglia in memoria sono le attivazioni, non i pesi.

12 ReLU e non linearità

La funzione di attivazione è l'unico ingrediente che rende la profondità significativa. Senza di essa una rete di cento strati è algebricamente identica a uno strato solo. ReLU è la scelta dominante, per ragioni che vale la pena capire fino in fondo.

12.1 Definizione

DEFINIZIONE

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{se } z > 0 \\ 0 & \text{se } z \leq 0 \end{cases}$$

Si applica **elemento per elemento** a ogni valore del tensore. Non ha parametri appresi, non altera la forma del tensore, non mescola posizioni né canali.

La derivata, che serve alla backpropagation, è altrettanto semplice:

$$\frac{d}{dz}\text{ReLU}(z) = \begin{cases} 1 & \text{se } z > 0 \\ 0 & \text{se } z < 0 \end{cases}$$

In $z = 0$ la derivata non esiste; per convenzione le librerie usano 0. Il costo computazionale è quello di un confronto: nessuna esponenziale, nessuna divisione.

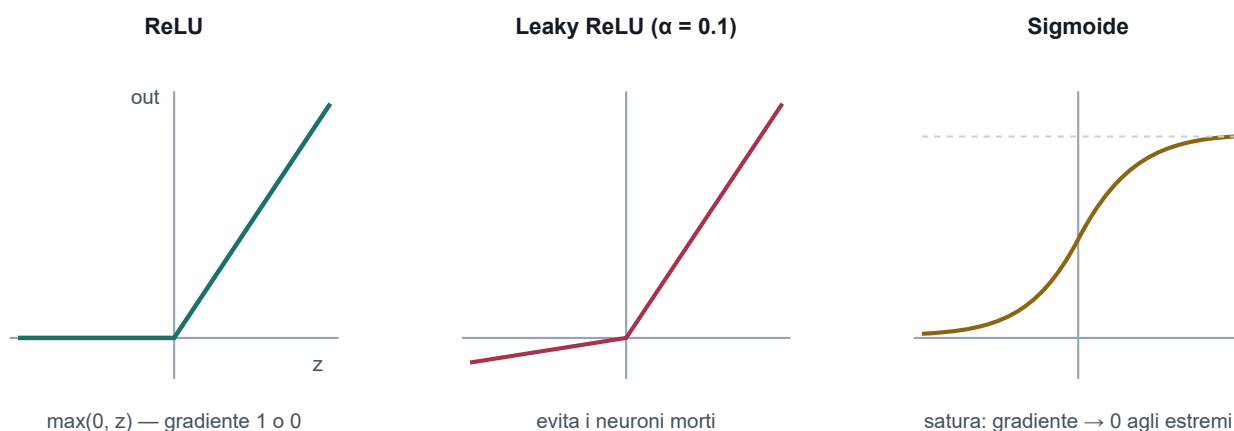


Figura 12.1 — Tre funzioni di attivazione. Il difetto della sigmoide, storicamente fatale per le reti profonde, è visibile nel grafico: agli estremi la curva è piatta, quindi la derivata tende a zero e il gradiente svanisce moltiplicandosi strato dopo strato.

12.2 Perché serve una non linearità

Riprendiamo il calcolo del capitolo 1 in forma convoluzionale. La convoluzione è un'operazione lineare, quindi rappresentabile come prodotto matrice-vettore $A\mathbf{x}$ (con una matrice A sparsa e a struttura di Toeplitz a blocchi). Componendo due convoluzioni senza attivazione:

$$A_2(A_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (A_2A_1)\mathbf{x} + (A_2\mathbf{b}_1 + \mathbf{b}_2)$$

e A_2A_1 è ancora una matrice di convoluzione (con kernel più grande). Cento strati convoluzionali lineari equivalgono a **un solo** strato convoluzionale con kernel di dimensione maggiore. Le famiglie di funzioni rappresentabili sono le stesse. Il guadagno rappresentazionale della profondità è esattamente zero.

Con ReLU interposta, invece, ogni strato può attivare o disattivare selettivamente parti dell'input, e la funzione risultante è lineare a tratti su un numero di regioni che cresce **esponenzialmente** con la profondità. È questo il fondamento della potenza espressiva delle reti profonde.

ESEMPIO NUMERICO · EFFETTO DI RELU SU UNA FEATURE MAP

Feature map 3×3 all'uscita di una convoluzione:

$$\begin{array}{ccc} 20 & -5 & 3 \\ -12 & 8 & -1 \\ 0 & 15 & -7 \end{array} \rightarrow \begin{array}{ccc} 20 & 0 & 3 \\ 0 & 8 & 0 \\ 0 & 15 & 0 \end{array}$$

Cinque valori su nove azzerati: il 56% di **sparsità**. La rete lavora d'ora in poi solo sulle posizioni in cui il filtro ha risposto positivamente. Si noti che i pesi del filtro non sono cambiati di un millesimo: è cambiata solo l'attivazione.

12.3 Vantaggi, e il prezzo da pagare

PROPRIETÀ	CONSEGUENZA
Gradiente esattamente 1 per $z > 0$	Nessuna attenuazione del gradiente lungo il cammino attivo: risolve in gran parte il <i>vanishing gradient</i>
Calcolo banale	Un confronto; molto più veloce di exp o tanh
Sparsità delle attivazioni	Rappresentazioni più selettive, effetto regolarizzante lieve
Non limitata superiormente	Le attivazioni possono crescere: rende importante la normalizzazione (→ Cap. 14)
Gradiente 0 per $z < 0$	Neuroni morti : se un filtro produce sempre valori negativi, non riceve mai gradiente e resta bloccato per sempre

APPROFONDIMENTO • IL PROBLEMA DEL NEURONE MORTO E LE VARIANTI DI RELU

Se, per esempio a causa di un learning rate troppo alto ($\rightarrow$ Cap. 23), il bias di un filtro scende molto sotto zero, l'uscita del filtro può risultare negativa per *ogni* input del dataset. Allora ReLU la azzerava sempre, la derivata è sempre 0, il gradiente che arriva al filtro è sempre 0, e i suoi pesi non cambiano più: il filtro è morto. Rimedi comuni:

- **Leaky ReLU:** $\max(\alpha z, z)$ con $\alpha \approx 0.01-0.1$. Il ramo negativo ha pendenza piccola ma non nulla, quindi un gradiente arriva sempre. È l'attivazione usata da nnU-Net, con $\alpha = 0.01$.
- **PReLU:** come Leaky, ma α è appreso.
- **ELU, GELU, SiLU:** versioni lisce, con derivata continua; GELU è standard nei Transformer.

ERRORI COMUNI

«**ReLU modifica i pesi.**» No. ReLU non contiene pesi e non ne modifica alcuno. Modifica le *attivazioni*, cioè i valori che scorrono nella rete. Indirettamente influenza l'aggiornamento dei pesi, perché determina quale gradiente passa e quale viene azzerato — ma è un effetto sul percorso del gradiente, non un'operazione sui pesi.

«**ReLU serve a normalizzare.**» No, ReLU non normalizza nulla: non limita superiormente e non centra la distribuzione. La normalizzazione è compito della batch normalization.

«**Mettere ReLU dopo l'ultimo strato.**» Errore serio: azzererebbe tutti i logits negativi, distruggendo l'informazione che softmax deve elaborare ($\rightarrow$ Cap. 19). L'ultimo strato lineare non va seguito da ReLU.

RIEPILOGO DEL CAPITOLO 12

- ReLU azzerava i valori negativi e lascia inalterati i positivi; agisce sulle attivazioni, mai sui pesi.
- Senza non linearità la profondità è algebricamente inutile.
- Gradiente 1 sul ramo positivo: cammino privilegiato per la retropropagazione.
- Rischio di neuroni morti sul ramo negativo; Leaky ReLU lo mitiga ed è lo standard in ambito medico.

13 Pooling

Il pooling riduce la risoluzione spaziale aggregando finestre di attivazioni. Non estrae nuove feature, non ha parametri: sintetizza. È l'operazione che rende computazionalmente sostenibile la crescita del campo recettivo.

DEFINIZIONE

Il **pooling** partiziona (o fa scorrere su) la feature map con finestre di dimensione $k \times k$ e stride s , sostituendo ciascuna finestra con un singolo valore riassuntivo. Le due varianti principali:

$$\text{MaxPool}(i, j) = \max_{0 \leq u, v < k} Y(si + u, sj + v) \quad \text{AvgPool}(i, j) = \frac{1}{k^2} \sum_{u, v} Y(si + u, sj + v)$$

Il pooling si applica **indipendentemente a ogni canale**: il numero di canali non cambia mai.

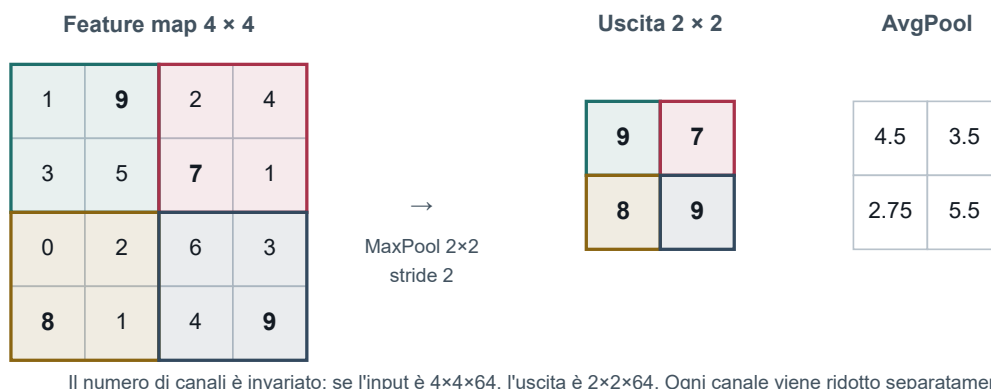


Figura 13.1 — Max pooling e average pooling 2×2 con stride 2. Il max pooling conserva l'attivazione più forte di ogni quadrante: risponde alla domanda "il pattern è presente da qualche parte in questa regione?", scartando l'esatta posizione all'interno della finestra.

13.1 Che cosa fa e che cosa non fa

- **Riduce** la dimensione spaziale di un fattore s per asse, quindi le attivazioni di s^2 in 2D e s^3 in 3D.
- **Amplia** il campo recettivo degli strati successivi (→ Cap. 10).
- **Introduce invarianza locale alle traslazioni**: se il pattern si sposta di un pixel dentro la finestra, il massimo non cambia.
- **Non** introduce parametri, **non** mescola canali, **non** estrae nuove feature: opera su ciò che la convoluzione ha già calcolato.

13.2 Max o average?

	MAX POOLING	AVERAGE POOLING
Semantica	"Il pattern c'è, in qualche punto della finestra"	"Quanto in media risponde il filtro qui"
Effetto sul segnale	Preserva i picchi, elimina le risposte deboli	Attenua i picchi, riduce il rumore
Gradiente	Fluisce solo verso l'elemento massimo (gli altri ricevono 0)	Distribuito equamente su tutti gli elementi ($1/k^2$ ciascuno)
Uso tipico	Encoder di reti di riconoscimento	Global average pooling in coda alle reti moderne

Il **global average pooling** merita una menzione: riduce ogni feature map $H \times W$ a un solo numero, producendo un vettore lungo C . Sostituisce il flatten seguito da FC ed elimina milioni di parametri, rendendo la rete indipendente dalla dimensione dell'input.

ESEMPIO NUMERICO • RIDUZIONE LUNGO UN ENCODER

Partenza $24 \times 24 \times 64$, due blocchi con MaxPool 2×2 :

$$24 \times 24 \times 64 \xrightarrow{\text{pool}} 12 \times 12 \times 64 \xrightarrow{\text{conv } 128} 12 \times 12 \times 128 \xrightarrow{\text{pool}} 6 \times 6 \times 128$$

Attivazioni: $36\,864 \rightarrow 9\,216 \rightarrow 18\,432 \rightarrow 4\,608$. Ogni pooling divide per 4; ogni raddoppio di canali moltiplica per 2. Il risultato netto è una compressione progressiva.

ERRORI COMUNI

«Il pooling estrae feature.» No. Le feature le estrae la convoluzione. Il pooling seleziona o media valori già calcolati.

«Il pooling riduce anche i canali.» Mai. $24 \times 24 \times 64 \rightarrow 12 \times 12 \times 64$: il 64 resta 64.

«Il pooling non ha gradiente perché non ha parametri.» Confusione fra due cose diverse. Il pooling non ha parametri *da aggiornare*, ma deve comunque *propagare* il gradiente verso gli strati precedenti: nel max pooling il gradiente va interamente all'elemento che era massimo (la libreria memorizza l'indice durante il forward), negli altri va zero.

APPROFONDIMENTO • PERCHÉ LA SEGMENTAZIONE HA UN RAPPORTO COMPLICATO CON IL POOLING

Il pooling scarta deliberatamente l'informazione sulla posizione precisa: dice "c'è un bordo in questo quadrante" senza dire in quale dei quattro pixel. Per la classificazione è un vantaggio (invarianza). Per la segmentazione è un danno diretto: il compito è proprio decidere se *quel* pixel appartiene al rene. Questa tensione è il problema che U-Net risolve con le skip connection (→ Cap. 33): l'informazione ad alta risoluzione, persa nel pooling, viene reiniettata nel decoder.

RIEPILOGO DEL CAPITOLO 13

- Il pooling riduce la risoluzione spaziale aggregando finestre; canali invariati, parametri zero.
- Max pooling preserva i picchi e propaga il gradiente solo all'elemento massimo.
- Amplia il campo recettivo e introduce invarianza locale alle traslazioni.
- L'invarianza è utile in classificazione e problematica in segmentazione: da qui le skip connection.

14 Batch Normalization

La batch normalization normalizza le attivazioni all'interno della rete. È l'intervento che, più di ogni altro, ha reso addestrabili le reti molto profonde: consente learning rate più alti, riduce la sensibilità all'inizializzazione e ha un modesto effetto regolarizzante.

14.1 Il problema

Durante l'addestramento la distribuzione degli ingressi di ogni strato cambia continuamente, perché dipende dai pesi degli strati precedenti, che a loro volta vengono aggiornati a ogni passo. Ogni strato insegue quindi un bersaglio mobile — fenomeno storicamente battezzato *internal covariate shift*. Le conseguenze pratiche sono attivazioni che possono crescere o svanire di strato in strato e la necessità di learning rate molto piccoli.

14.2 La formulazione

Per uno strato convoluzionale, la BN normalizza **per canale**, calcolando media e varianza su tutte le posizioni spaziali i e su tutti gli esempi del mini-batch. Per il canale c , dato un batch di m esempi:

$$\mu_c = \frac{1}{m H W} \sum_{n,i,j} x_{n,c,i,j} \quad \sigma_c^2 = \frac{1}{m H W} \sum_{n,i,j} (x_{n,c,i,j} - \mu_c)^2$$

$$\hat{x}_{n,c,i,j} = \frac{x_{n,c,i,j} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} \quad y_{n,c,i,j} = \gamma_c \hat{x}_{n,c,i,j} + \beta_c$$

La prima riga standardizza (media 0, varianza 1). La seconda riga è cruciale: γ_c e β_c sono **parametri appresi**, uno per canale, che permettono alla rete di ripristinare una scala e uno spostamento diversi se questo riduce la loss. Senza di essi la BN sarebbe un vincolo rigido; con essi è una riparametrizzazione. Il termine $\epsilon \approx 10^{-5}$ evita la divisione per zero.

Uno strato BN su 64 canali ha quindi $2 \times 64 = 128$ parametri appresi (esattamente il valore riportato nella Tabella 3.1), più 128 statistiche **non** apprese, di cui parliamo subito.

14.3 Training e inference: la asimmetria fondamentale

APPROFONDIMENTO • DUE COMPORTAMENTI DIVERSI NELLO STESSO MODULO

In **training** si usano μ e σ^2 calcolati sul mini-batch corrente. L'uscita di un esempio dipende quindi dagli altri esempi del batch: una forma di rumore stocastico che agisce da regolarizzatore.

In **inference** questo è inaccettabile: la predizione su un paziente non può dipendere da quali altri pazienti si trovano nello stesso batch. Si usano perciò stime globali accumulate durante il training con una media mobile esponenziale:

$$\mu_{\text{run}} \leftarrow (1 - \alpha) \mu_{\text{run}} + \alpha \mu_{\text{batch}}$$

Dimenticare di mettere il modello in modalità di valutazione è uno degli errori più frequenti e più insidiosi in assoluto: il modello continua a normalizzare con le statistiche del batch di test, e le metriche risultano incoerenti — a volte inspiegabilmente migliori, a volte disastrose, sempre non riproducibili.

14.4 Ordine dei blocchi e alternative

L'ordine convenzionale è **Conv** → **BN** → **ReLU**. La ragione: la BN centra la distribuzione attorno allo zero, che è il punto di ginocchio di ReLU; ciò garantisce che circa metà delle attivazioni sopravviva, evitando la morte di massa dei neuroni. Inoltre, poiché la BN sottrae la media, **il bias della convoluzione precedente diventa ridondante** e viene tipicamente omissso: qualunque costante additiva è cancellata dalla sottrazione di μ , e il ruolo di offset è svolto da β .

TECNICA	STATISTICHE CALCOLATE SU	DIPENDE DAL BATCH?	USO TIPICO
Batch Norm	(N, H, W) per canale	Sì	CNN 2D con batch grandi
Instance Norm	(H, W) per canale e per esempio	No	Segmentazione medica 3D , style transfer
Group Norm	(gruppo di canali, H, W) per esempio	No	Batch molto piccoli
Layer Norm	(C, H, W) per esempio	No	Transformer

PERCHÉ nnU-NET USA INSTANCE NORMALIZATION

In segmentazione 3D le patch sono enormi e la memoria GPU limita il batch a 2 o addirittura 1 volume. Con $m = 2$, la media e la varianza del batch sono stime rumorosissime: la BN diventa instabile e le statistiche accumulate sono inaffidabili. L'Instance Normalization calcola le statistiche *dentro il singolo esempio*, quindi è del tutto indipendente dalla dimensione del batch e si comporta in modo identico in training e in inference. È per questo motivo — non per una preferenza estetica — che nnU-Net adotta Instance Norm come impostazione predefinita.

ERRORI COMUNI

«La BN normalizza l'input della rete.» No: normalizza le attivazioni *interne*, strato per strato. La normalizzazione dell'input è un'altra cosa e va fatta comunque, nel preprocessing.

«La BN non ha parametri.» Ne ha: γ e β , appresi per backpropagation come tutti gli altri.

«Con la BN posso usare qualsiasi learning rate.» La BN aumenta il learning rate tollerabile, non lo rende irrilevante.

RIEPILOGO DEL CAPITOLO 14

- La BN standardizza le attivazioni per canale usando le statistiche del mini-batch, poi riscalda con γ, β appresi.
- Comportamento diverso in training (statistiche del batch) e in inference (medie mobili accumulate).
- Ordine standard Conv $\rightarrow$ BN $\rightarrow$ ReLU; il bias della convoluzione diventa ridondante.
- Con batch piccoli, tipici della segmentazione 3D, si preferisce Instance Normalization.

15 Dropout

Il dropout è una tecnica di regolarizzazione: non serve a rendere la rete più potente, ma a impedirle di memorizzare il training set. Consiste nel disattivare casualmente una frazione di neuroni a ogni passo di addestramento.

DEFINIZIONE

Durante il training, ogni unità di uno strato con dropout viene azzerata con probabilità p , in modo indipendente e ricampionato a ogni forward pass. Formalmente, con $r_i \sim \text{Bernoulli}(1 - p)$:

$$\tilde{y}_i = \frac{r_i}{1 - p} y_i$$

Il fattore $1/(1 - p)$ è l'**inverted dropout**: riscalda le unità sopravvissute in modo che il valore atteso dell'attivazione resti invariato. Grazie a questo accorgimento, in inference il dropout viene semplicemente disattivato e non serve alcuna correzione.

15.1 Perché funziona

- **Impedisce la co-adattazione.** Un neurone non può fare affidamento sulla presenza di un particolare compagno, perché quel compagno potrebbe essere assente al prossimo passo. Ogni unità è costretta a produrre un contributo utile per conto proprio.
- **È un ensemble implicito.** Ogni maschera casuale definisce una sotto-rete diversa; con n unità esistono 2^n sotto-reti che condividono i pesi. La rete finale, con dropout disattivato, approssima la media geometrica delle predizioni di tutte le sotto-reti.
- **Aggiunge rumore.** Il rumore sulle attivazioni rende difficile la memorizzazione di dettagli idiosincratici dei singoli esempi.

ESEMPIO NUMERICO • DROPOUT CON $p = 0.5$

Attivazioni di uno strato denso: $[2.0, 4.0, 1.0, 3.0, 5.0, 0.5]$. Maschera campionata: $[1, 0, 1, 1, 0, 0]$. Fattore di scala $1/(1 - 0.5) = 2$.

$$[2.0 \cdot 2, 0, 1.0 \cdot 2, 3.0 \cdot 2, 0, 0] = [4.0, 0, 2.0, 6.0, 0, 0]$$

Somma originale: 15.5. Somma dopo dropout: 12.0. Su un singolo campione i due valori differiscono, ma **in media su molte maschere** coincidono: è questo che la riscalatura garantisce. Al passo successivo la maschera sarà diversa, e diversa sarà la sotto-rete addestrata.

15.2 Dove si mette, e dove no

POSIZIONE	VALORE TIPICO DI P	NOTE
Dopo strati fully connected	0.3 – 0.5	Uso classico e più efficace: è lì che stanno quasi tutti i parametri
Fra blocchi convoluzionali	0.0 – 0.2	Meno efficace: le attivazioni vicine sono fortemente correlate, azzerarne una singola informa poco
Prima dell'ultimo strato (logits)	Sconsigliato	Introduce rumore diretto sulla decisione finale
In inference	Disattivato	Eccezione importante: MC dropout, vedi sotto

Nelle CNN moderne il dropout classico è in parte soppiantato da batch normalization, data augmentation e weight decay. Esiste però una variante specifica per le convoluzioni, lo **spatial dropout** (o dropout 2D), che azzerà *interi canali* anziché singoli valori: dato che i valori dentro una feature map sono spazialmente correlati, azzerare l'intera mappa è un intervento molto più significativo.

APPROFONDIMENTO • MC DROPOUT E QUANTIFICAZIONE DELL'INCERTEZZA

Se si **lascia attivo** il dropout in fase di inference e si eseguono T forward pass sullo stesso input, si ottengono T predizioni diverse $\{\hat{y}^{(t)}\}$. Media e varianza di questo campione forniscono, rispettivamente, una predizione e una stima di incertezza:

$$\bar{y} = \frac{1}{T} \sum_t \hat{y}^{(t)} \quad \text{Var}[y] = \frac{1}{T} \sum_t (\hat{y}^{(t)} - \bar{y})^2$$

È il *Monte Carlo dropout*, interpretabile come approssimazione variazionale di un'inferenza bayesiana. In segmentazione medica produce una **mappa di incertezza voxel-wise**: tipicamente l'incertezza è alta lungo i contorni dell'organo e nelle regioni di transizione, bassa nell'interno del parenchima. È uno dei due metodi standard per l'uncertainty quantification, insieme ai deep ensemble (che addestrano M modelli indipendenti e sono generalmente più affidabili, al costo di M addestramenti completi).

ERRORI COMUNI

«**Il dropout elimina neuroni definitivamente.**» No: li disattiva per un solo passo. Al passo successivo la maschera è ricampionata e quel neurone può tornare attivo. Nessun peso viene cancellato.

«**Dropout alto = generalizzazione migliore.**» Con p eccessivo (0.7–0.8) la rete non riesce nemmeno ad adattarsi al training set: si passa da overfitting a underfitting (→ Cap. 29).

«**Il dropout va tenuto attivo in test.**» Solo se si sta facendo MC dropout intenzionalmente. Altrimenti è un bug che rende le predizioni non deterministiche.

RIEPILOGO DEL CAPITOLO 15

- Dropout = azzeramento casuale di unità durante il training, con riscalatura $1/(1 - p)$.
- Riduce la co-adattazione e agisce come ensemble implicito di sotto-reti.
- Disattivato in inference, salvo l'uso deliberato del MC dropout per stimare l'incertezza.
- Nelle CNN è più efficace nella variante spaziale, che azzerava interi canali.

16 Flatten

Il flatten è l'operazione più semplice di tutte, e insieme quella con la conseguenza architetturale più pesante: è il punto in cui la rete abbandona definitivamente la nozione di spazio.

DEFINIZIONE

Il **flatten** riorganizza un tensore di rango k in un vettore di rango 1, concatenando gli elementi secondo l'ordine di memoria. Un tensore (H, W, C) diventa un vettore di lunghezza $H \cdot W \cdot C$. Non modifica alcun valore, non ha parametri, non compie alcuna operazione aritmetica.

Esempio: $4 \times 4 \times 64 \rightarrow 1024$ componenti. Con ordinamento NCHW, l'elemento (c, h, w) finisce nella posizione $c(H \cdot W) + hW + w$ del vettore, esattamente la formula dell'offset di memoria vista al capitolo 4. In pratica il flatten **non muove nemmeno un byte**: reinterpreta soltanto gli indici.

16.1 Che cosa si perde

Prima del flatten, il valore in posizione $(3, 7)$ di un canale è adiacente a quello in $(3, 8)$: la rete può sfruttare questa adiacenza con una convoluzione. Dopo il flatten, i due valori sono le componenti n e $n + 1$ di un vettore, e lo strato successivo tratterà ogni componente come indipendente, assegnandole un peso separato. **L'informazione geometrica non è distrutta nei numeri, ma è resa inaccessibile all'architettura**: nessun operatore successivo sa più che quelle due componenti erano vicine.

INTUIZIONE

È come sciogliere una griglia di scacchi in una fila di 64 caselle numerate. I pezzi ci sono ancora tutti, ma la nozione di "diagonale" e "colonna" non esiste più per chi guarda solo la fila. Se il compito è contare i pezzi, la fila basta; se il compito è dire quali caselle sono minacciate, la fila è inservibile. La classificazione è il primo caso, la segmentazione il secondo.

16.2 La conseguenza sui parametri e sulla dimensione dell'input

Il flatten produce una lunghezza che dipende dalla dimensione spaziale, e quella lunghezza determina la dimensione della matrice del primo strato denso:

INPUT ALLA RETE	TENSORE PRIMA DEL FLATTEN	LUNGHEZZA VETTORE	PESI VERSO FC-256
$24 \times 24 \times 3$	$6 \times 6 \times 128$	4 608	1 179 648
$48 \times 48 \times 3$	$12 \times 12 \times 128$	18 432	4 718 592
$96 \times 96 \times 3$	$24 \times 24 \times 128$	73 728	18 874 368

Tabella 16.1 — Il costo del flatten cresce quadraticamente con la risoluzione di ingresso.

Da qui deriva un vincolo pratico ben noto: **una rete con flatten + FC accetta solo input di dimensione fissa.**

Cambiare la risoluzione richiede di cambiare la matrice dei pesi, cioè un modello diverso. Le due soluzioni moderne sono il global average pooling (→ Cap. 13), che rende la lunghezza indipendente da H , W , e le reti *fully convolutional*, che eliminano del tutto il flatten (→ Cap. 32).

ERRORI COMUNI

«**Il flatten perde informazione.**» Nessun valore viene perso: il vettore contiene esattamente gli stessi numeri, nello stesso ordine. Ciò che si perde è la *struttura*, cioè la possibilità di sfruttare l'adiacenza.

«**Il flatten è un layer con pesi.**» No, è una pura operazione di reshape.

«**In segmentazione si fa flatten e poi si rimodella l'output.**» Tecnicamente si può, ma è un pessimo progetto: il numero di parametri esploderebbe e si perderebbe l'equivarianza traslazionale. La soluzione corretta è non fare mai il flatten (→ Cap. 32).

RIEPILOGO DEL CAPITOLO 16

- Flatten = reshape da tensore a vettore; nessun parametro, nessun valore modificato.
- Rende inaccessibile all'architettura la struttura spaziale.
- Vincola la rete a una dimensione di input fissa e genera la maggior parte dei parametri del modello.
- Le architetture di segmentazione lo eliminano.

17 Fully Connected Layer

Lo strato completamente connesso è il classificatore vero e proprio: prende la descrizione globale prodotta dall'encoder e la trasforma, attraverso una o più trasformazioni affini intervallate da non linearità, in un punteggio per ciascuna classe.

17.1 Definizione e calcolo

DEFINIZIONE

Uno strato **fully connected** (o denso, o lineare) con n ingressi e m uscite calcola

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \quad W \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m$$

seguito, in tutti gli strati tranne l'ultimo, da una funzione di attivazione: $\mathbf{a} = \phi(\mathbf{z})$. Ogni uscita dipende da **tutti** gli ingressi: da qui il nome. Parametri: $m \cdot n + m$.

Componente per componente:

$$z_i = \sum_{j=1}^n W_{ij} x_j + b_i$$

È lo stesso prodotto scalare della convoluzione, con due differenze decisive: (i) il "kernel" $W_{i,:}$ copre l'intero ingresso, non una finestra locale; (ii) ogni uscita ha il proprio insieme di pesi, cioè **non** c'è condivisione. Sono esattamente i due bias induttivi delle CNN, rimossi.

ESEMPIO NUMERICO · UNO STRATO DENS0 4 → 3

Ingresso $\mathbf{x} = [1, 2, 0, 3]^\top$. Matrice e bias:

$$W = \begin{pmatrix} 0.5 & -1.0 & 2.0 & 0.0 \\ 1.0 & 0.0 & -0.5 & 1.5 \\ -2.0 & 1.0 & 0.0 & 0.5 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1.0 \\ -0.5 \\ 0.0 \end{pmatrix}$$

Calcolo riga per riga:

$$z_1 = 0.5(1) + (-1.0)(2) + 2.0(0) + 0(3) + 1.0 = 0.5 - 2.0 + 1.0 = -0.5$$

$$z_2 = 1.0(1) + 0(2) + (-0.5)(0) + 1.5(3) - 0.5 = 1.0 + 4.5 - 0.5 = 5.0$$

$$z_3 = -2.0(1) + 1.0(2) + 0(0) + 0.5(3) + 0.0 = -2.0 + 2.0 + 1.5 = 1.5$$

Uscita: $\mathbf{z} = [-0.5, 5.0, 1.5]^\top$. Con ReLU diventerebbe $[0, 5.0, 1.5]$; se però questo è l'ultimo strato, **non** si applica ReLU e $\mathbf{z}$ è il vettore dei logits (→ Cap. 18). Parametri usati: $3 \times 4 + 3 = 15$.

17.2 Il ruolo funzionale nella CNN

L'encoder risponde alla domanda "quali pattern sono presenti e dove". La testa densa risponde a "che cosa significa questa combinazione di pattern". Il primo strato denso combina globalmente le attivazioni: può, in linea di principio, apprendere una regola del tipo "se il canale 12 è acceso in alto e il canale 40 in basso, allora questa è la classe 2". Il secondo strato denso riduce a K valori, uno per classe.

17.3 Confronto diretto con la convoluzione

	CONVOLUZIONE	FULLY CONNECTED
Connettività	Locale (campo recettivo)	Globale
Condivisione dei pesi	Sì, stesso kernel ovunque	No, pesi indipendenti per ogni uscita
Parametri	$C_{out}(k^2 C_{in} + 1)$, indipendente da H, W	$m \cdot n + m$, dipendente dalla dimensione dell'input
Struttura spaziale	Preservata (equivarianza)	Distrutta
Input di dimensione variabile	Accettato	Non accettato

APPROFONDIMENTO · UNO STRATO DENSO È UNA CONVOLUZIONE 1×1

Osservazione tanto elegante quanto operativamente utile: applicare uno strato denso da C a K unità a ogni posizione spaziale di un tensore $H \times W \times C$ è **esattamente** una convoluzione con kernel 1×1 , $C_{in} = C$, $C_{out} = K$. Il kernel 1×1 non guarda alcun intorno spaziale: mescola solo i canali nella stessa posizione.

Ne segue la costruzione centrale del capitolo 32: si può sostituire la testa densa con una conv 1×1 e ottenere una rete che produce una decisione **per ogni pixel** anziché una sola decisione globale. È la trasformazione da rete di classificazione a *fully convolutional network*, e il passo concettuale che apre la segmentazione.

ERRORI COMUNI

«Il fully connected estrae feature.» Le combina; l'estrazione avviene nell'encoder.

«Più strati densi migliorano il classificatore.» Aggiungono moltissimi parametri e sono la fonte principale di overfitting. Le architetture moderne usano tipicamente un solo strato lineare finale dopo il global average pooling.

«L'ultimo strato ha una funzione di attivazione.» No: l'ultimo strato lineare produce logits grezzi. La trasformazione in probabilità è compito di softmax, e nelle implementazioni è spesso fusa nella loss (→ Cap. 21).

RIEPILOGO DEL CAPITOLO 17

- FC: $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, ogni uscita connessa a ogni ingresso, nessuna condivisione di pesi.
- Combina globalmente le feature dell'encoder per prendere la decisione.
- Contiene la maggior parte dei parametri e impone dimensione di input fissa.
- Uno strato denso applicato posizione per posizione è una convoluzione 1×1 : è il ponte verso la segmentazione.

18 Logits

I logits sono l'uscita grezza della rete: numeri reali non normalizzati, uno per classe. Sono un oggetto tecnico preciso, e confonderli con le probabilità è all'origine di molti errori sottili, incluso il tema della calibrazione.

DEFINIZIONE

I **logits** $\mathbf{z} \in \mathbb{R}^K$ sono l'uscita dell'ultimo strato lineare, prima di qualunque normalizzazione. Non hanno vincoli: possono essere negativi, positivi, arbitrariamente grandi, e la loro somma non è fissata.

Il nome deriva dalla funzione *logit*, inversa della sigmoide: $\text{logit}(p) = \log \frac{p}{1-p}$. Nel caso binario, il logit è il logaritmo del rapporto di verosimiglianza; il legame con le probabilità è quindi logaritmico, e questo spiega la forma esponenziale di softmax.

18.1 Proprietà

- **Solo le differenze contano.** Aggiungere una costante c a tutti i logits non cambia la distribuzione di probabilità prodotta da softmax: $\text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z})$. I logits sono definiti a meno di una costante additiva.
- **La scala assoluta conta però per la confidenza.** $[1, 2]$ e $[10, 20]$ hanno la stessa classe vincente ma producono probabilità molto diverse: $[0.27, 0.73]$ contro $[0.00005, 0.99995]$. Moltiplicare i logits per un fattore è esattamente ciò che fa il *temperature scaling*.
- **Per la predizione non servono le probabilità.** $\arg \max_k z_k = \arg \max_k p_k$, perché softmax è monotona crescente. Se serve solo l'etichetta, si può leggere direttamente il logit massimo.

ESEMPIO NUMERICO · LOGITS E LORO INTERPRETAZIONE

Rete a 3 classi (0 = sfondo, 1 = rene sinistro, 2 = rene destro). Output finale: $\mathbf{z} = [-1.2, 3.5, 0.8]$.

- Classe predetta: indice 2 nel vettore, cioè "rene sinistro" (logit 3.5, il massimo).
- Margine rispetto alla seconda classe: $3.5 - 0.8 = 2.7$: un margine ampio, la rete è netta.
- Se i logits fossero stati $[-1.2, 0.9, 0.8]$, il margine sarebbe 0.1: stessa decisione, ma sostanzialmente un pareggio.

La sola etichetta predetta non distingue i due casi. La loss, invece, sì: il secondo caso produce un errore molto maggiore, e quindi un gradiente maggiore.

APPROFONDIMENTO · LOGITS, CONFIDENZA E CALIBRAZIONE

Le reti neurali profonde moderne sono note per essere **mal calibrate**: producono sistematicamente probabilità troppo elevate rispetto alla loro accuratezza reale. Un modello che dichiara 0.99 di confidenza su un insieme di predizioni ne azzecca magari l'88%. La causa è, in larga parte, che la minimizzazione della cross entropy spinge i logits a divergere: per far tendere la loss a zero occorre far tendere il logit corretto a $+\infty$.

Il **temperature scaling** corregge questo comportamento a posteriori, dividendo i logits per uno scalare $T > 0$ appreso sul validation set:

$$p_k = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}}$$

Con $T > 1$ la distribuzione si appiattisce (meno confidenza), con $T < 1$ si acuisce. È l'intervento minimo possibile: un solo parametro, che **non cambia mai la classe predetta** (l'ordinamento dei logits è preservato) ma cambia la probabilità associata, e quindi metriche come l'Expected Calibration Error e il Brier score. In segmentazione medica, dove la probabilità per voxel viene usata per costruire mappe di incertezza e per scegliere le soglie, la calibrazione non è un dettaglio accademico: è ciò che rende la mappa di confidenza interpretabile da un clinico.

ERRORI COMUNI

«Un logit alto significa alta probabilità.» Solo *relativamente* agli altri logits dello stesso vettore. Un logit di 8 è schiacciante se gli altri valgono 1, irrilevante se un altro vale 15.

«Applicare softmax e poi passare il risultato alla cross entropy.» Numericamente pericoloso, e nella maggior parte delle librerie una doppia applicazione: le funzioni di loss standard accettano i logits e integrano il softmax al loro interno in forma numericamente stabile (→ Cap. 21).

RIEPILOGO DEL CAPITOLO 18

- I logits sono l'uscita lineare non normalizzata, uno per classe, senza vincoli di segno o somma.
- Invarianti per traslazione additiva; la loro scala determina la confidenza.
- La classe predetta è l'argmax dei logits: softmax non è necessario per decidere.
- Reti profonde tendono a logits sovradimensionati, da cui la miscalibrazione e il rimedio del temperature scaling.

19 Softmax

Softmax converte i logits in una distribuzione di probabilità. È l'ultimo anello del forward pass e il punto in cui l'output della rete diventa confrontabile con un'etichetta.

DEFINIZIONE

$$p_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K$$

Proprietà garantite: $p_k > 0$ per ogni k (l'esponenziale è sempre positivo) e $\sum_k p_k = 1$ (per costruzione del denominatore). L'uscita è quindi una distribuzione di probabilità sulle K classi. Softmax non ha parametri appresi.

19.1 Perché l'esponenziale

Servono tre proprietà simultaneamente: positività, somma unitaria, e conservazione dell'ordinamento. Una semplice normalizzazione $p_k = z_k / \sum_j z_j$ fallisce con logits negativi (produrrebbe probabilità negative). L'esponenziale risolve la positività, è monotona (quindi preserva l'ordine) e ha una proprietà aggiuntiva molto comoda: **trasforma le differenze in rapporti**.

$$\frac{p_a}{p_b} = \frac{e^{z_a}}{e^{z_b}} = e^{z_a - z_b}$$

Il rapporto fra due probabilità dipende solo dalla differenza dei rispettivi logits. Da qui l'invarianza per traslazione additiva vista al capitolo 18, e la stabilizzazione numerica che ne consegue.

ESEMPIO NUMERICO • SOFTMAX PASSO PER PASSO

Logits $\mathbf{z} = [-1.2, 3.5, 0.8]$.

1. **Esponenziali:** $e^{-1.2} = 0.3012$, $e^{3.5} = 33.115$, $e^{0.8} = 2.2255$.

2. **Somma:** $0.3012 + 33.115 + 2.2255 = 35.642$.

3. **Divisione:**

$$p_1 = \frac{0.3012}{35.642} = 0.0085 \quad p_2 = \frac{33.115}{35.642} = 0.9291 \quad p_3 = \frac{2.2255}{35.642} = 0.0624$$

Verifica: $0.0085 + 0.9291 + 0.0624 = 1.0000$. ✓

La rete assegna il 92.9% alla classe 2. Si osservi l'effetto dell'esponenziale: la differenza di logit $3.5 - 0.8 = 2.7$ diventa un rapporto di probabilità $e^{2.7} \approx 14.9$. Softmax **amplifica** le differenze.

19.2 Stabilità numerica

Con logits grandi, e^z va in overflow: già e^{800} supera il massimo rappresentabile in doppia precisione. Si sfrutta l'invarianza additiva sottraendo il massimo:

$$p_k = \frac{e^{z_k - \max_j z_j}}{\sum_i e^{z_i - \max_j z_j}}$$

Il risultato matematico è identico, ma ora l'esponente massimo è 0, quindi $e^0 = 1$ e tutti gli altri termini stanno in $(0, 1]$. Ogni implementazione seria di softmax fa questo internamente.

19.3 Softmax, sigmoide e la scelta in segmentazione

	SOFTMAX	SIGMOIDE (PER CLASSE)
Vincolo	Le probabilità sommano a 1	Ogni uscita è indipendente in $[0, 1]$
Ipotesi	Classi mutuamente esclusive	Classi non esclusive, possibile sovrapposizione
Uso	Classificazione multi-classe; segmentazione con etichette esclusive	Multi-label; segmentazione di regioni annidate

In segmentazione renale con etichette {sfondo, rene, tumore} mutuamente esclusive si usa softmax, applicato **voxel per voxel** lungo l'asse dei canali. Se invece le regioni fossero annidate — per esempio "rene intero" e "tumore dentro il rene", entrambe vere sullo stesso voxel — la scelta corretta sarebbe la sigmoide per canale, come avviene nelle formulazioni a "regioni" adottate da nnU-Net per alcune sfide.

ERRORI COMUNI

«**Softmax rende la rete non lineare.**» Contribuisce, ma non è quello il suo scopo, ed è collocato dopo l'ultimo strato: non aggiunge capacità rappresentazionale agli strati interni.

«**La probabilità di softmax è la probabilità reale che la classe sia corretta.**» Solo se il modello è calibrato, cosa che in generale non è vera (→ Cap. 18).

«**Softmax si applica sulla dimensione sbagliata.**» Errore concreto e frequente in segmentazione: va applicato lungo l'asse delle *classi* (i canali), non lungo gli assi spaziali. Applicarlo sulle dimensioni spaziali farebbe sommare a 1 le probabilità di tutti i pixel, il che non ha alcun senso.

RIEPILOGO DEL CAPITOLO 19

- Softmax normalizza i logits in una distribuzione di probabilità positiva a somma 1.
- L'esponenziale garantisce positività e trasforma differenze di logit in rapporti di probabilità.
- Si stabilizza sottraendo il logit massimo; il risultato è invariato.
- Softmax per classi esclusive, sigmoide per classi indipendenti; in segmentazione si applica lungo l'asse dei canali.

20 Loss Function

La funzione di perdita è la definizione operativa di "errore". È l'unico canale attraverso cui il compito da risolvere entra nella matematica dell'ottimizzazione: cambiare la loss significa cambiare che cosa la rete considera un buon risultato.

DEFINIZIONE

Una **funzione di perdita** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$ associa a una coppia (predizione, ground truth) un numero reale non negativo che misura la discrepanza. La loss di un mini-batch è la media delle loss dei singoli esempi:

$$\mathcal{L}_{\text{batch}} = \frac{1}{m} \sum_{n=1}^m \mathcal{L}(\hat{\mathbf{y}}^{(n)}, \mathbf{y}^{(n)})$$

Requisito indispensabile: la loss deve essere **differenziabile** rispetto ai parametri (almeno quasi ovunque), perché l'ottimizzazione procede per gradiente.

20.1 Il punto di contatto con il ground truth

Questo è il punto più importante da fissare, e vale la pena enunciarlo senza ambiguità:

PRINCIPIO

Il ground truth entra nella rete **una volta sola**, alla fine, nel calcolo della loss. Nessuna feature map intermedia viene mai confrontata con un'etichetta. Le rappresentazioni interne non hanno un target: si formano indirettamente, perché *quella* configurazione di pesi rende piccola la loss finale.

Una loss scalare per l'intero batch è dunque l'unico segnale di supervisione. Da quel numero, la backpropagation ricava una derivata per ciascuno dei milioni di parametri della rete. È un fatto notevole, ed è il tema del capitolo 24.

20.2 Le loss principali

LOSS	FORMULA	COMPITO
Errore quadratico medio (MSE)	$\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$	Regressione
Errore assoluto medio (MAE)	$\frac{1}{n} \sum_i \hat{y}_i - y_i $	Regressione robusta agli outlier
Cross entropy binaria	$-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$	Classificazione binaria
Cross entropy categorica	$-\sum_k y_k \log \hat{y}_k$	Classificazione multi-classe (→ Cap. 21)
Dice loss	$1 - \frac{2 \sum_i \hat{y}_i y_i + \varepsilon}{\sum_i \hat{y}_i + \sum_i y_i + \varepsilon}$	Segmentazione con classi sbilanciate

APPROFONDIMENTO • PERCHÉ LA SEGMENTAZIONE MEDICA USA DICE + CROSS ENTROPY

In un volume addominale i voxel di rene sono forse l'1-2% del totale. Una rete che predicesse "sfondo" ovunque otterrebbe un'accuratezza del 98-99% e una cross entropy molto bassa: la loss non la punirebbe abbastanza. La **Dice loss** deriva dal coefficiente di Dice, che misura la sovrapposizione fra due insiemi:

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|} \in [0, 1]$$

ed è per costruzione insensibile alla dimensione dello sfondo, perché nel numeratore e nel denominatore compaiono solo i voxel di primo piano. La versione "soft" sostituisce l'intersezione con il prodotto delle probabilità, rendendola differenziabile.

La combinazione $\mathcal{L} = \mathcal{L}_{\text{Dice}} + \mathcal{L}_{\text{CE}}$, adottata come impostazione predefinita da nnU-Net, unisce due comportamenti complementari: la cross entropy fornisce gradienti stabili e ben condizionati per ogni voxel, la Dice si concentra sulla qualità della sovrapposizione globale della struttura. È una scelta empirica, validata su decine di dataset, non un teorema.

ESEMPIO NUMERICO • LA STESSA PREDIZIONE, DUE LOSS DIVERSE

Immagine 100 voxel, di cui 5 di rene. Predizione: la rete trova 3 dei 5 voxel di rene e non produce falsi positivi.

- **Accuratezza:** 98 voxel corretti su 100 = 0.98. Sembra eccellente.
- **Dice:** $2 \cdot 3 / (3 + 5) = 6/8 = 0.75$. Descrive molto meglio la realtà: manca il 40% dell'organo.

Ecco perché in segmentazione non si riporta mai l'accuratezza, ma Dice, IoU e distanza di Hausdorff.

ERRORI COMUNI

«**La loss è la metrica di valutazione.**» Sono cose distinte. La loss deve essere differenziabile e serve all'ottimizzazione; la metrica serve a valutare e può essere non differenziabile (Dice binario, Hausdorff, accuratezza). Spesso si ottimizza un surrogato differenziabile della metrica che interessa davvero.

«**Una loss più bassa in training significa un modello migliore.**» Solo sul training set. La domanda rilevante è che cosa faccia la loss di validazione (→ Cap. 27).

RIEPILOGO DEL CAPITOLO 20

- La loss traduce il compito in un numero differenziabile da minimizzare.
- Il ground truth è confrontato solo con l'output finale, mai con le rappresentazioni intermedie.
- La scelta della loss dipende dal compito e dallo sbilanciamento delle classi.
- In segmentazione medica la combinazione Dice + Cross Entropy è lo standard.

21 Cross Entropy

La cross entropy è la loss canonica della classificazione. Ha una giustificazione teorica solida (massima verosimiglianza), una forma finale sorprendentemente semplice e un gradiente di eleganza notevole.

21.1 Formulazione

Dato il vettore di ground truth in codifica one-hot $\mathbf{y}$ (tutti zeri tranne un 1 in corrispondenza della classe corretta c) e le probabilità predette $\hat{\mathbf{y}}$:

$$\mathcal{L}_{CE} = - \sum_{k=1}^K y_k \log \hat{y}_k = - \log \hat{y}_c$$

La semplificazione è immediata: tutti i termini con $y_k = 0$ si annullano, e resta soltanto il logaritmo negativo della probabilità assegnata alla classe corretta. **La cross entropy guarda esclusivamente alla probabilità della classe giusta**; come si distribuisce la probabilità residua fra le classi sbagliate non la riguarda direttamente (anche se la influenza attraverso il vincolo di normalizzazione di softmax).

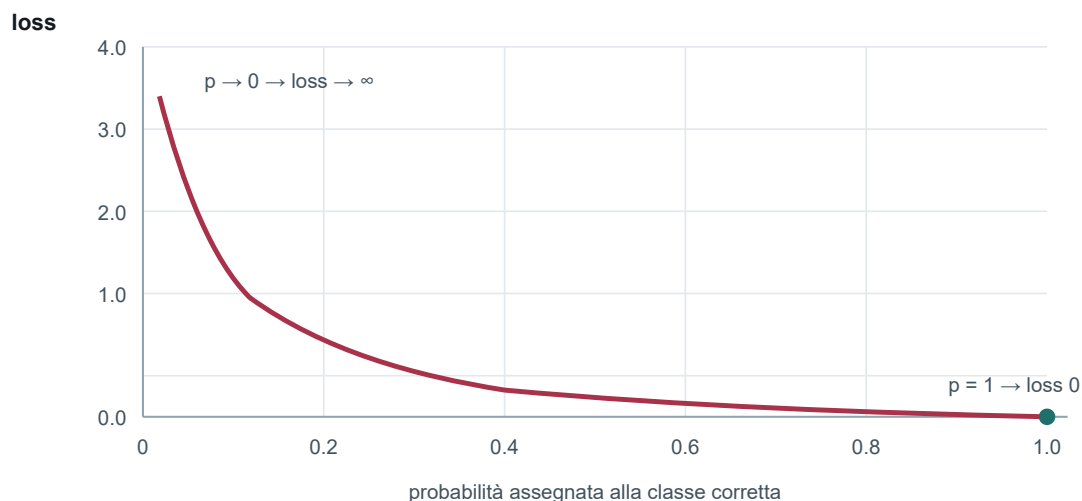


Figura 21.1 — La curva $-\log p$. La penalità è nulla quando la rete assegna probabilità 1 alla classe corretta e diverge quando la probabilità tende a zero. L'asimmetria è deliberata: essere sicuri e sbagliare è punito in modo severissimo.

21.2 Interpretazione

Y_C	LOSS – $\text{IN}Y_C$	LETTURA
0.99	0.010	Predizione corretta e sicura: penalità trascurabile
0.90	0.105	Corretta, ragionevolmente sicura
0.50	0.693	Incertezza massima in un problema binario
0.10	2.303	Errore: la classe giusta è stata considerata improbabile
0.01	4.605	Errore grave e sicuro di sé

Confronto istruttivo: passare da 0.99 a 0.90 costa 0.095 di loss; passare da 0.10 a 0.01 costa 2.30. La cross entropy investe la quasi totalità della sua pressione sugli esempi mal classificati, il che è esattamente il comportamento desiderabile in un ottimizzatore.

APPROFONDIMENTO • DA DOVE VIENE: MASSIMA VEROSIMIGLIANZA

Interpretiamo l'uscita della rete come una distribuzione condizionata $p_{\theta}(y \mid \mathbf{x})$. La verosimiglianza del dataset, assunta l'indipendenza degli esempi, è $\prod_n p_{\theta}(y^{(n)} \mid \mathbf{x}^{(n)})$. Massimizzarla equivale a minimizzarne il logaritmo negativo:

$$-\log \prod_n p_{\theta}(y^{(n)} \mid \mathbf{x}^{(n)}) = -\sum_n \log p_{\theta}(y^{(n)} \mid \mathbf{x}^{(n)})$$

che è precisamente la somma delle cross entropy. La cross entropy non è quindi una scelta arbitraria: è la stima di massima verosimiglianza per un modello categorico. Equivalentemente, minimizza la divergenza di Kullback-Leibler fra la distribuzione empirica (one-hot) e quella predetta.

21.3 Il gradiente: il risultato più elegante della dispensa

Componendo softmax e cross entropy e derivando rispetto ai **logits**, si ottiene un risultato notevolmente semplice. Ponendo $\mathcal{L} = -\log \hat{y}_c$ con $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$:

$$\frac{\partial \mathcal{L}}{\partial z_k} = \hat{y}_k - y_k$$

Il gradiente è semplicemente **predizione meno verità**. Tutte le esponenziali e i logaritmi si cancellano. Conseguenze pratiche:

- Se la predizione è perfetta ($\hat{\mathbf{y}} = \mathbf{y}$), il gradiente è nullo: nessun aggiornamento.
- Il gradiente è limitato in $[-1, 1]$: niente esplosioni dovute alla loss.
- L'entità della correzione è proporzionale all'errore: più si sbaglia, più forte è la spinta.

- Per motivi di stabilità numerica, le librerie fondono softmax e cross entropy in un'unica funzione che opera direttamente sui logits — evitando di calcolare esplicitamente probabilità prossime a zero e poi il loro logaritmo.

ESEMPIO NUMERICO • LOSS E GRADIENTE COMPLETI

Riprendiamo i logits del capitolo 19: $\mathbf{z} = [-1.2, 3.5, 0.8]$, probabilità $\hat{\mathbf{y}} = [0.0085, 0.9291, 0.0624]$.

Caso A — la classe corretta è la 2 ($\mathbf{y} = [0, 1, 0]$):

$$\mathcal{L} = -\ln(0.9291) = 0.0735$$

$$\nabla_{\mathbf{z}}\mathcal{L} = [0.0085 - 0, 0.9291 - 1, 0.0624 - 0] = [+0.0085, -0.0709, +0.0624]$$

Loss piccola, gradienti piccoli: la rete è già quasi corretta e verrà modificata poco.

Caso B — la classe corretta è la 3 ($\mathbf{y} = [0, 0, 1]$):

$$\mathcal{L} = -\ln(0.0624) = 2.7739$$

$$\nabla_{\mathbf{z}}\mathcal{L} = [+0.0085, +0.9291, -0.9376]$$

Loss quasi 38 volte maggiore, e gradienti di modulo quasi massimo. Il segno indica la direzione: il logit della classe corretta (il terzo) ha gradiente negativo, quindi la discesa del gradiente lo **aumenterà**; quello della classe erroneamente favorita ha gradiente positivo e verrà **diminuito**. Il meccanismo è del tutto trasparente.

ERRORI COMUNI

«**Applicare softmax e poi la cross entropy della libreria.**» Nella maggior parte dei framework la funzione di cross entropy si aspetta i *logits*: passarle probabilità significa applicare softmax due volte, con il risultato di appiattire i gradienti e ottenere un addestramento lentissimo e apparentemente inspiegabile.

«**La cross entropy vale solo per la classificazione di immagini intere.**» In segmentazione si applica identica, ma *per ogni pixel*, e poi si media su tutti i pixel (→ Cap. 32).

RIEPILOGO DEL CAPITOLO 21

- $\mathcal{L}_{CE} = -\log \hat{y}_c$: dipende solo dalla probabilità assegnata alla classe corretta.
- Penalizza in modo divergente gli errori sicuri; deriva dalla massima verosimiglianza.
- Il gradiente rispetto ai logits è $\hat{\mathbf{y}} - \mathbf{y}$: predizione meno verità.
- Softmax e cross entropy vanno usate fuse per stabilità numerica.

22 Gradient Descent

La discesa del gradiente è l'algoritmo con cui i parametri vengono modificati. L'idea è elementare: muoversi nella direzione in cui la loss decresce più rapidamente. Le complicazioni nascono tutte dal fatto che la superficie da percorrere è non convessa e vive in uno spazio a milioni di dimensioni.

22.1 L'idea e la regola di aggiornamento

Il gradiente $\nabla_{\theta}\mathcal{L}$ è il vettore delle derivate parziali rispetto a ogni parametro. Punta nella direzione di massima *crescita* della loss; per minimizzare ci si muove nel verso opposto:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta}\mathcal{L}(\theta_t)$$

dove η è il learning rate ($\rightarrow$ Cap. 23). Componente per componente, per un singolo peso:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}}$$

Ogni peso viene aggiornato indipendentemente, ma tutti gli aggiornamenti sono calcolati a partire dallo **stesso** valore di loss.

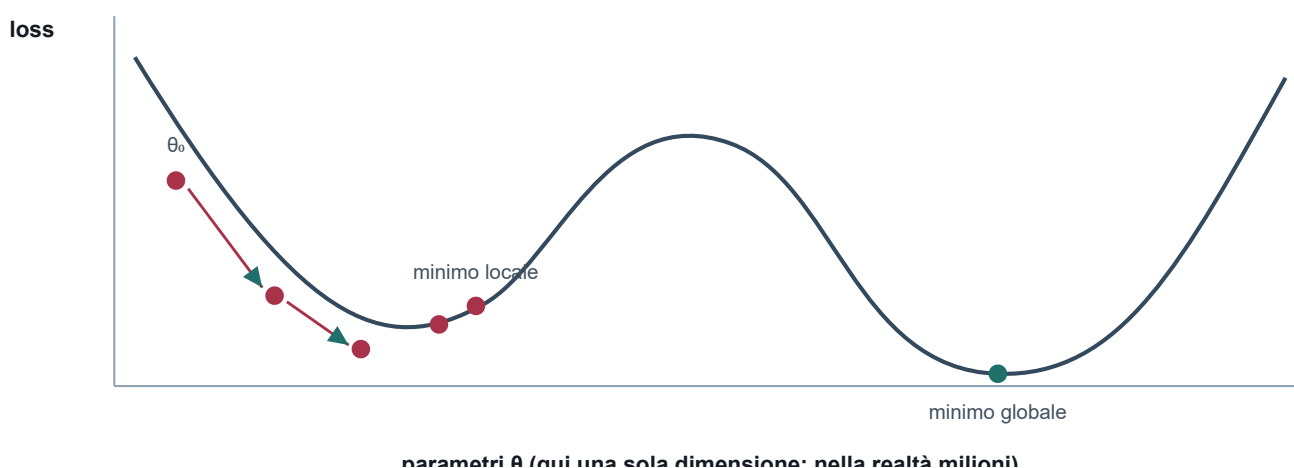


Figura 22.1 — Discesa del gradiente. Il passo è proporzionale alla pendenza: grande dove la superficie è ripida, piccolo in prossimità di un punto stazionario. In alta dimensione i minimi locali stretti sono rari; il problema dominante sono i punti di sella e le regioni piatte.

22.2 Batch, stocastico, mini-batch

VARIANTE	ESEMPI PER AGGIORNAMENTO	PRO	CONTRO
Batch (full)	Tutti (N)	Gradiente esatto, traiettoria liscia	Un solo aggiornamento per epoca; impossibile con dataset grandi
Stocastico (SGD puro)	1	Aggiornamenti frequentissimi, il rumore aiuta a uscire dai minimi locali	Traiettoria molto rumorosa, nessun vantaggio dal parallelismo
Mini-batch	m (8–256, o 2 in 3D)	Compromesso ottimale, sfrutta la GPU	m è un iperparametro da scegliere

Il mini-batch è lo standard universale. Il gradiente calcolato su m esempi è una stima non distorta del gradiente vero, con varianza proporzionale a $1/m$: il rumore residuo non è solo tollerato, è utile, perché aiuta a sfuggire ai punti di sella.

22.3 Momentum e ottimizzatori adattivi

La discesa pura ha due debolezze note: oscilla nelle valli strette e rallenta a morte negli altipiani. Il **momentum** introduce inerzia accumulando una media mobile dei gradienti passati:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla \mathcal{L}(\boldsymbol{\theta}_t) \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_{t+1}$$

con $\beta \approx 0.9\text{--}0.99$. Le componenti oscillanti si cancellano fra loro, quelle coerenti si sommano.

Adam aggiunge un secondo ingrediente: un passo diverso per ogni parametro, normalizzato dalla stima della varianza dei gradienti.

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

(con $\hat{\mathbf{m}}$, $\hat{\mathbf{v}}$ corretti per il bias iniziale). Adam converge rapidamente ed è robusto alla scelta iniziale di η ; tuttavia in diversi contesti — segmentazione medica inclusa — **SGD con momentum di Nesterov** e un buon schedule produce modelli che generalizzano meglio. nnU-Net usa SGD con momentum 0.99 e decadimento polinomiale del learning rate, una scelta controcorrente ma validata su moltissimi dataset.

ESEMPIO NUMERICO · UN PASSO DI AGGIORNAMENTO

Peso $w = 0.500$, gradiente $\partial\mathcal{L}/\partial w = +2.0$, learning rate $\eta = 0.01$:

$$w \leftarrow 0.500 - 0.01 \cdot 2.0 = 0.480$$

Il gradiente è positivo, quindi aumentare w farebbe crescere la loss: si diminuisce. Con momentum $\beta = 0.9$ e velocità precedente $v = 1.5$:

$$v \leftarrow 0.9(1.5) + 2.0 = 3.35 \quad w \leftarrow 0.500 - 0.01(3.35) = 0.4665$$

Il passo è più che raddoppiato perché il gradiente conferma la direzione già intrapresa: è l'accelerazione tipica del momentum lungo direzioni coerenti.

ERRORI COMUNI

«**La discesa del gradiente trova il minimo globale.**» No, e non è nemmeno l'obiettivo. La superficie è non convessa; si converge a un minimo locale o a una regione piatta. Empiricamente, in reti sovrapparametrizzate, la gran parte dei minimi raggiungibili ha qualità simile.

«**Dimenticare di azzerare i gradienti.**» Nei framework a grafo dinamico i gradienti si *accumulano*. Non azzerarli a ogni iterazione significa sommare i gradienti di tutti i batch precedenti: l'addestramento diverge in modo apparentemente misterioso.

RIEPILOGO DEL CAPITOLO 22

- Aggiornamento: $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$, verso opposto al gradiente.
- Il mini-batch è il compromesso standard fra accuratezza del gradiente ed efficienza.
- Momentum accumula inerzia; Adam adatta il passo a ogni parametro.
- L'obiettivo non è il minimo globale ma un minimo che generalizzi bene.

23 Learning Rate

Il learning rate è, per opinione praticamente unanime, l'iperparametro più importante. Determina l'ampiezza di ogni passo di aggiornamento e da solo può decidere se un addestramento converge, diverge o si blocca.

DEFINIZIONE

Il **learning rate** $\eta > 0$ è il fattore moltiplicativo applicato al gradiente nell'aggiornamento dei parametri: $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$. È un iperparametro: non viene appreso dal gradiente, ma scelto (o programmato) dall'esterno.

23.1 I tre regimi



Figura 23.1 — Il learning rate regola l'ampiezza del passo. Troppo piccolo: convergenza lentissima e rischio di restare intrappolati. Troppo grande: si scavalca il minimo, l'errore cresce, i valori esplodono numericamente.

SINTOMO OSSERVATO	DIAGNOSI PROBABILE	INTERVENTO
Loss costante o quasi piatta	η troppo piccolo, oppure gradienti nulli (neuroni morti)	Aumentare η di un fattore 3–10
Loss che oscilla senza scendere	η leggermente troppo grande	Ridurre η , attivare uno schedule
Loss che esplode o diventa NaN	η molto troppo grande	Ridurre di un ordine di grandezza, aggiungere gradient clipping
Loss che scende e poi si ferma su un plateau	Passo ormai troppo grande per la fase finale	Decadimento del learning rate

23.2 Schedule

Un valore fisso è raramente ottimale: all'inizio conviene un passo ampio per coprire terreno, alla fine un passo fine per assestarsi. Gli schemi più diffusi:

- **Step decay:** $\eta \leftarrow \eta \cdot \gamma$ ogni s epoche (per esempio $\gamma = 0.1$ ogni 30 epoche).
- **Cosine annealing:** $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right)$; discesa liscia fino a quasi zero.
- **Polinomiale** (usato da nnU-Net): $\eta_t = \eta_0 \left(1 - \frac{t}{T}\right)^{0.9}$, con $\eta_0 = 0.01$ e $T = 1000$ epoche.
- **Warmup:** nelle prime iterazioni si parte da un valore molto basso e si sale linearmente, per evitare che i primi aggiornamenti, calcolati su pesi casuali, destabilizzino la rete.

ESEMPIO NUMERICO · DECADIMENTO POLINOMIALE

Con $\eta_0 = 0.01$, $T = 1000$, esponente 0.9:

EPOCA	$(1 - t/T)^{0.9}$	η_t
0	1.000	0.01000
250	0.771	0.00771
500	0.536	0.00536
750	0.287	0.00287
990	0.0126	0.00013

Il passo si riduce di quasi due ordini di grandezza nell'ultima frazione dell'addestramento: la rete smette di esplorare e si assesta.

APPROFONDIMENTO • IL LEARNING RATE RANGE TEST

Un metodo pratico per scegliere η senza tentativi alla cieca: si esegue un addestramento di poche centinaia di iterazioni aumentando η esponenzialmente a ogni passo, da 10^{-7} a 10, e si traccia la loss contro $\log \eta$. Il grafico mostra tipicamente una zona piatta iniziale (passo troppo piccolo), una discesa netta, un minimo e poi un'esplosione. Il valore consigliato è quello nel punto di massima pendenza discendente, circa un ordine di grandezza sotto il minimo. È un investimento di pochi minuti che ne fa risparmiare molti.

ERRORI COMUNI

«Il learning rate si adatta da solo con Adam.» Adam adatta il passo *relativo* per parametro, ma il fattore globale η resta da scegliere. Un Adam con $\eta = 1.0$ diverge come qualunque altro ottimizzatore.

«Se la loss è NaN c'è un bug nel codice.» Nella maggior parte dei casi è semplicemente il learning rate. Vale la pena provare a dividerlo per 10 prima di iniziare a cercare il bug.

RIEPILOGO DEL CAPITOLO 23

- η controlla l'ampiezza del passo; è l'iperparametro più influente.
- Troppo piccolo: lentezza; troppo grande: oscillazione e divergenza.
- Gli schedule riducono η nel tempo; il warmup protegge le prime iterazioni.
- Il range test individua rapidamente un valore ragionevole.

24 Backpropagation

La backpropagation è l'algoritmo che calcola, a partire da un unico numero — la loss — la derivata parziale rispetto a ciascuno dei milioni di parametri della rete. Non è un algoritmo di apprendimento: è un modo efficiente di applicare la regola della catena.

24.1 La regola della catena

Per una composizione $y = f(g(x))$ vale

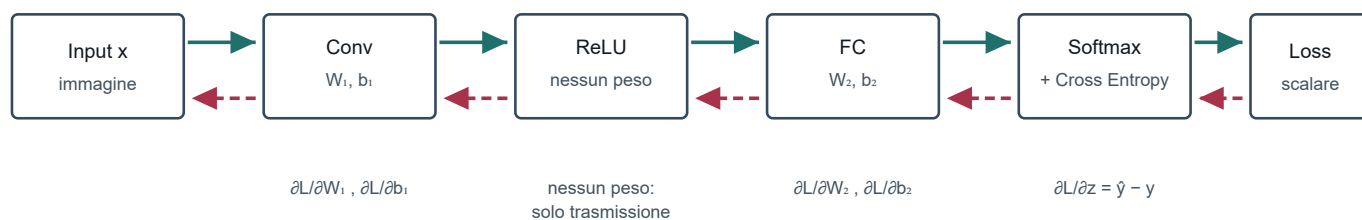
$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$$

Una rete neurale è precisamente una lunga composizione di funzioni. La derivata della loss rispetto a un peso dello strato ℓ si ottiene moltiplicando le derivate locali di tutti gli strati che stanno fra quel peso e la loss:

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \cdot \frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{a}^{(L-1)}} \cdots \frac{\partial \mathbf{a}^{(\ell+1)}}{\partial \mathbf{a}^{(\ell)}} \cdot \frac{\partial \mathbf{a}^{(\ell)}}{\partial W^{(\ell)}}$$

L'idea che rende il tutto praticabile è il **riuso**: il prodotto dei primi fattori, calcolato per lo strato $\ell + 1$, serve identico allo strato ℓ . Si procede quindi all'indietro, propagando un solo vettore di derivate — il gradiente rispetto alle attivazioni — e derivando a ogni tappa i gradienti rispetto ai pesi locali. Il costo totale del backward è circa il doppio di quello del forward, **indipendentemente** dal numero di parametri.

FORWARD →



Il gradiente attraversa TUTTA la rete: aggiorna sia i pesi densi sia i kernel convoluzionali.

← BACKWARD

Figura 24.1 — Forward e backward. Nel forward i dati fluiscono verso destra e ogni modulo memorizza ciò che gli servirà; nel backward il gradiente fluisce verso sinistra. I moduli senza parametri (ReLU, pooling, flatten) non hanno nulla da aggiornare ma **devono** comunque trasmettere il gradiente.

24.2 Esempio numerico completo

ESEMPIO NUMERICO • BACKPROPAGATION SU UNA RETE MINIMA

Rete scalare: $x \rightarrow z_1 = w_1 x + b_1 \rightarrow a_1 = \text{ReLU}(z_1) \rightarrow z_2 = w_2 a_1 + b_2$, loss $\mathcal{L} = (z_2 - y)^2$.

Valori iniziali: $x = 1.0$, $w_1 = 0.5$, $b_1 = 0$, $w_2 = 2.0$, $b_2 = -0.3$, target $y = 1.0$.

Forward:

$$z_1 = 0.5(1.0) + 0 = 0.5 \quad \Rightarrow \quad a_1 = \max(0, 0.5) = 0.5$$

$$z_2 = 2.0(0.5) - 0.3 = 0.7 \quad \Rightarrow \quad \mathcal{L} = (0.7 - 1.0)^2 = 0.09$$

Backward, procedendo a ritroso:

$$\frac{\partial \mathcal{L}}{\partial z_2} = 2(z_2 - y) = 2(-0.3) = -0.6$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial z_2} \cdot a_1 = (-0.6)(0.5) = -0.30 \quad \frac{\partial \mathcal{L}}{\partial b_2} = -0.60$$

$$\frac{\partial \mathcal{L}}{\partial a_1} = \frac{\partial \mathcal{L}}{\partial z_2} \cdot w_2 = (-0.6)(2.0) = -1.2$$

$$\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial a_1} \cdot 1[z_1 > 0] = (-1.2)(1) = -1.2$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial z_1} \cdot x = -1.2 \quad \frac{\partial \mathcal{L}}{\partial b_1} = -1.2$$

Aggiornamento con $\eta = 0.05$:

$$w_2 \leftarrow 2.0 - 0.05(-0.30) = 2.015 \quad b_2 \leftarrow -0.3 - 0.05(-0.60) = -0.27$$

$$w_1 \leftarrow 0.5 - 0.05(-1.2) = 0.56 \quad b_1 \leftarrow 0 - 0.05(-1.2) = 0.06$$

Verifica — nuovo forward: $z_1 = 0.62$, $a_1 = 0.62$, $z_2 = 2.015(0.62) - 0.27 = 0.9793$.

$$\mathcal{L}_{\text{nuova}} = (0.9793 - 1.0)^2 = 0.00043$$

La loss è scesa da 0.09 a 0.0004: un solo passo di aggiornamento. Si osservi che **anche w_1 , il peso "più lontano" dalla loss, è stato aggiornato**: il gradiente ha attraversato ReLU e lo strato successivo. In una CNN reale w_1 sarebbe un peso di un kernel convoluzionale, e il meccanismo è identico.

24.3 Il gradiente attraverso i moduli tipici

MODULO	CHE COSA FA NEL BACKWARD
Convoluzione	Calcola $\partial\mathcal{L}/\partial K$ (correlazione fra input e gradiente di uscita) e propaga $\partial\mathcal{L}/\partial X$ (convoluzione con il kernel ribaltato). Poiché il kernel è condiviso, i contributi di tutte le posizioni si sommano nello stesso gradiente
ReLU	Moltiplica il gradiente per 1 dove l'attivazione era positiva, per 0 altrove
Max pooling	Instrada tutto il gradiente all'elemento che era massimo; zero agli altri
Flatten	Rimodella il gradiente nella forma originale; nessun calcolo
Fully connected	$\partial\mathcal{L}/\partial W = \delta \mathbf{x}^\top$, propaga $W^\top \delta$
Softmax + CE	Restituisce direttamente $\hat{\mathbf{y}} - \mathbf{y}$

APPROFONDIMENTO • PERCHÉ SERVE MEMORIZZARE LE ATTIVAZIONI

Il gradiente rispetto ai pesi di uno strato dipende dall'*input* di quello strato: $\partial\mathcal{L}/\partial W = \delta \mathbf{x}^\top$. Perciò tutte le attivazioni intermedie calcolate nel forward vanno conservate fino al backward. È questa la ragione per cui l'occupazione di memoria durante il training è molto superiore a quella dell'inference (dove le attivazioni si possono scartare subito), ed è per questo che si può fare inference con batch molto più grandi. Il *gradient checkpointing* baratta memoria con calcolo: scarta parte delle attivazioni e le ricalcola nel backward.

ERRORI COMUNI

«La **backpropagation** aggiorna solo la parte densa.» Falso, e va corretto con decisione: il gradiente attraversa **tutta** la rete. I kernel convoluzionali del primo strato vengono aggiornati esattamente come i pesi dell'ultimo strato. È l'intero motivo per cui le feature sono *apprese* e non progettate.

«La **backpropagation** è l'algoritmo di apprendimento.» No: calcola i gradienti. L'apprendimento è compito dell'ottimizzatore, che quei gradienti li usa.

«Il gradiente viene calcolato per ogni esempio separatamente e poi si sceglie il migliore.» No: i gradienti degli esempi del mini-batch vengono *mediati* in un unico gradiente, che produce un solo aggiornamento.

RIEPILOGO DEL CAPITOLO 24

- La backpropagation applica la regola della catena all'indietro, riusando i calcoli intermedi.
- Costo circa doppio del forward, indipendente dal numero di parametri.
- Aggiorna tutti i parametri, kernel convoluzionali compresi.
- Richiede la memorizzazione delle attivazioni del forward: da qui l'elevato consumo di memoria in training.

25 Epoch

Epoca, iterazione e campione sono tre unità di conto diverse. Confonderle rende impossibile leggere una curva di addestramento o confrontare due esperimenti.

DEFINIZIONI

- **Sample (campione):** un singolo esempio del dataset — un'immagine, una patch, un volume.
- **Mini-batch:** un gruppo di m campioni processati insieme, che produce **un** aggiornamento dei pesi.
- **Iterazione (o step):** un ciclo completo forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update su un mini-batch.
- **Epoca:** un passaggio completo su tutto il training set, cioè $\lceil N/m \rceil$ iterazioni.

ESEMPIO NUMERICO • CONTEGGIO ESATTO

Training set $N = 1000$ immagini, batch $m = 32$, addestramento per 50 epoche.

$$\text{iterazioni per epoca} = \lceil 1000/32 \rceil = \lceil 31.25 \rceil = 32$$

(31 batch completi da 32 immagini e un ultimo batch da 8, salvo che si scelga di scartarlo)

$$\text{aggiornamenti totali} = 32 \times 50 = 1600$$

$$\text{forward pass totali} = 1000 \times 50 = 50\,000$$

Ogni immagine viene vista 50 volte, **una volta per epoca**, in un ordine diverso a ogni epoca e — se c'è data augmentation — in una versione trasformata diversa. Non esiste alcun momento in cui la stessa immagine venga mostrata 50 volte di seguito.

25.1 Il mescolamento

All'inizio di ogni epoca il dataset viene rimescolato. Senza mescolamento i mini-batch sarebbero sempre gli stessi e nello stesso ordine, introducendo una correlazione sistematica negli aggiornamenti: se il dataset fosse ordinato per classe, la rete vedrebbe centinaia di esempi consecutivi della stessa classe e i pesi oscillerebbero seguendo l'ordinamento. Il mescolamento rende i gradienti dei batch stime più indipendenti del gradiente vero.

25.2 Quante epoche

Non c'è una risposta universale. Il criterio operativo è la curva di loss di validazione ($\rightarrow$ Cap. 27): si addestra finché essa migliora, e si conserva il modello con il valore migliore (*early stopping*). Alcuni riferimenti pratici:

- Dataset piccoli con forte augmentation: molte epoche (centinaia), perché ogni epoca mostra dati diversi.
- Fine-tuning di un modello pre-addestrato: poche epoche (5–20).
- nnU-Net: 1000 epoche di 250 iterazioni ciascuna, con una definizione *fissa* di epoca svincolata da N — scelta che rende comparabili dataset di dimensioni molto diverse.

ERRORI COMUNI

«1000 epoche significa vedere 1000 volte la stessa immagine di fila.» No: significa 1000 passaggi completi sull'intero training set, ciascuno in ordine rimescolato.

«Confrontare due esperimenti a parità di epoche.» Se i batch hanno dimensione diversa, a parità di epoche il numero di *aggiornamenti* è diverso. Spesso il confronto sensato è a parità di iterazioni.

RIEPILOGO DEL CAPITOLO 25

- Epoca = un passaggio completo sul training set = $\lceil N/m \rceil$ iterazioni.
- Un aggiornamento dei pesi per mini-batch, non per campione.
- Il mescolamento a ogni epoca decorrela i gradienti.
- Il numero di epoche si decide osservando la validazione, non a priori.

26 Batch

La dimensione del mini-batch è un iperparametro con effetti che vanno ben oltre la velocità: influenza la qualità del gradiente, la stabilità della batch normalization e persino le proprietà di generalizzazione del minimo raggiunto.

26.1 Effetti della dimensione

ASPETTO	BATCH PICCOLO (2–8)	BATCH GRANDE (128–512)
Rumore del gradiente	Alto (varianza $\propto 1/m$)	Basso, gradiente vicino a quello vero
Aggiornamenti per epoca	Molti	Pochi
Uso della GPU	Inefficiente	Efficiente (parallelismo saturo)
Memoria	Bassa	Alta: è il vincolo pratico principale
Batch Normalization	Statistiche instabili → preferire Instance/Group Norm	Statistiche affidabili
Generalizzazione	Spesso migliore: il rumore favorisce minimi "piatti"	Rischio di minimi "aguzzi", peggiore generalizzazione

APPROFONDIMENTO • LA REGOLA DI SCALA LINEARE

Se si moltiplica per k la dimensione del batch, il gradiente medio ha varianza k volte minore: si può quindi permettere un passo più lungo. La regola empirica, ben verificata su ampia scala, è di moltiplicare anche il learning rate per k , accompagnandolo con qualche epoca di warmup. Passare da $m = 32$, $\eta = 0.01$ a $m = 256$ suggerisce $\eta \approx 0.08$. La regola perde validità per batch estremamente grandi.

26.2 Il caso della segmentazione 3D

Qui la scelta non è libera. Un volume 128^3 a 32 canali occupa già centinaia di megabyte di attivazioni; il batch è tipicamente 2, talvolta 1. Le conseguenze si concatenano:

1. Batch 2 $\Rightarrow$ le statistiche della batch norm sarebbero inutilizzabili $\Rightarrow$ si usa **Instance Normalization** ($\rightarrow$ Cap. 14).

2. Batch 2 $\Rightarrow$ gradiente molto rumoroso $\Rightarrow$ si usa **momentum alto** (0.99 in nnU-Net) per mediare implicitamente su molti passi.
3. Batch 2 $\Rightarrow$ poca varietà per aggiornamento $\Rightarrow$ si usa **augmentation aggressiva** ($\rightarrow$ Cap. 30) per aumentare la diversità effettiva.

Nessuna di queste scelte è arbitraria: sono conseguenze concatenate di un vincolo di memoria. Riconoscere questa catena causale è un buon esempio di come si legge criticamente un'architettura.

ESEMPIO NUMERICO · GRADIENT ACCUMULATION

Si desidera un batch effettivo di 16, ma in memoria ne entrano solo 4. Si esegue il forward e il backward su 4 micro-batch consecutivi **senza** azzerare i gradienti, poi si applica un solo aggiornamento:

$$\nabla_{\text{eff}} = \frac{1}{4} \sum_{i=1}^4 \nabla_i$$

Il risultato è matematicamente equivalente a un batch di 16 (con l'unica eccezione delle statistiche della batch norm, che restano calcolate su 4). Il costo è temporale, non di memoria.

ERRORI COMUNI

«**Batch grande = addestramento migliore.**» Più veloce per epoca, non necessariamente migliore: a parità di epoche gli aggiornamenti sono meno numerosi, e la letteratura documenta un peggioramento della generalizzazione con batch molto grandi non compensati.

«**Batch size e numero di classi sono collegati.**» Non lo sono in alcun modo.

RIEPILOGO DEL CAPITOLO 26

- Batch grande: gradiente più preciso, meno aggiornamenti, più memoria.
- Batch piccolo: gradiente rumoroso, spesso migliore generalizzazione.
- Regola di scala lineare fra batch size e learning rate.
- In 3D il batch è imposto dalla memoria e determina a cascata normalizzazione, momentum e augmentation.

27 Validation

Il training set serve ad adattare i pesi, il validation set a prendere decisioni sul modello, il test set a stimare la prestazione finale. Tenere separati questi tre ruoli è ciò che distingue un risultato credibile da un numero senza valore.

27.1 La partizione e il suo significato

INSIEME	QUOTA TIPICA	USATO PER	QUANTE VOLTE LO SI GUARDA
Training	70–80%	Calcolare la loss e aggiornare i pesi	A ogni iterazione
Validation	10–15%	Scegliere iperparametri, architettura, epoca di arresto	Una volta per epoca
Test	10–15%	Stimare la prestazione su dati nuovi	Una volta sola, alla fine

Il punto sottile riguarda il validation set. I suoi dati non entrano mai nel calcolo del gradiente, quindi non influenzano *direttamente* i pesi. Ma influenzano le decisioni: quale learning rate, quante epoche, quale architettura. Ripetendo molte volte questo ciclo si finisce per adattare le **scelte progettuali** al validation set, che perde progressivamente la sua natura di dato mai visto. È il motivo per cui esiste un terzo insieme, il test, da toccare una volta sola.

27.2 Come si legge una curva di validazione

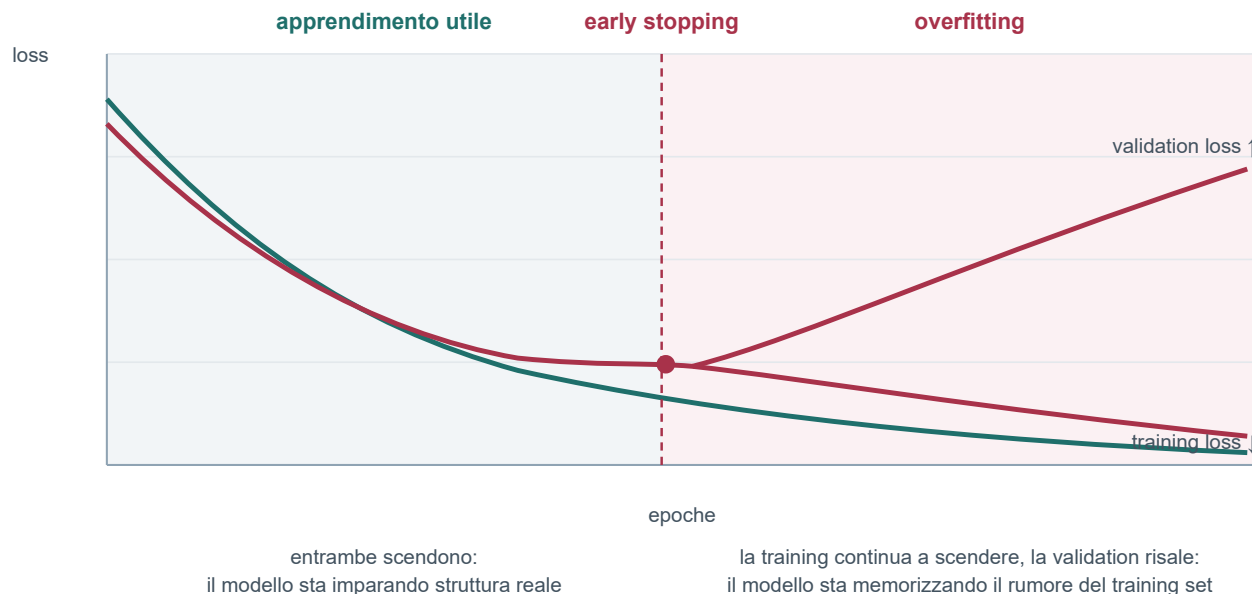


Figura 27.1 — Curve tipiche. Il punto di minimo della validation loss individua il modello da conservare. Oltre quel punto ogni epoca aggiuntiva peggiora la prestazione sui dati nuovi, pur migliorando quella sui dati visti.

TRAINING LOSS	VALIDATION LOSS	DIAGNOSI
Scende	Scende	Tutto regolare, continuare
Scende	Sale	Overfitting (→ Cap. 28)
Alta e piatta	Alta e piatta	Underfitting (→ Cap. 29) o learning rate inadeguato
Scende	Molto rumorosa	Validation set troppo piccolo, o learning rate alto
Alta	Più bassa del training	Sospetto: tipicamente dropout o augmentation attivi solo in training

27.3 Cross-validation

Con pochi dati — 45 volumi, per dire un numero realistico in ambito clinico — una singola partizione è instabile: il risultato dipende pesantemente da quali casi sono finiti in validazione. La **K-fold cross-validation** divide il dataset in K parti, addestra K modelli usando ogni volta una parte diversa come validazione, e media le metriche.

Costo: K addestramenti completi. Beneficio: una stima molto più stabile, con la sua deviazione standard, e — vantaggio spesso trascurato — K modelli che possono essere usati come **ensemble** in inference. È esattamente ciò che nnU-Net fa in modo predefinito con $K = 5$.

ERRORI COMUNI - LA CONTAMINAZIONE DEI DATI

Split a livello di slice invece che di paziente. È l'errore più grave e più frequente in medical imaging. Se le slice di uno stesso paziente finiscono sia in training sia in validation, il modello ha già visto quell'anatomia: le metriche risultano ottime e crollano su pazienti nuovi. **La partizione va sempre fatta a livello di paziente.**

Normalizzare usando statistiche calcolate su tutto il dataset. Se μ e σ includono il test set, un'informazione sul test è entrata nel preprocessing. Le statistiche vanno calcolate solo sul training.

Selezionare la soglia ottimale sul test set. A quel punto il test non stima più nulla: è diventato un secondo validation set.

RIEPILOGO DEL CAPITOLO 27

- Training per i pesi, validation per le decisioni, test per la stima finale.
- Il validation set si consuma con l'uso ripetuto: da qui la necessità di un test intoccato.
- La divergenza fra le due curve è il segnale operativo di overfitting.
- Con pochi dati la K-fold cross-validation è la scelta corretta; la partizione deve sempre essere a livello di paziente.

28 Overfitting

L'overfitting è la condizione in cui il modello ha imparato regolarità che esistono solo nel training set. Non è un difetto di implementazione: è la conseguenza naturale di un modello abbastanza espressivo da poter memorizzare.

DEFINIZIONE

Si ha **overfitting** quando l'errore sul training continua a diminuire mentre l'errore su dati non visti aumenta. Formalmente, il divario di generalizzazione $\mathcal{L}_{\text{test}} - \mathcal{L}_{\text{train}}$ cresce.

28.1 Perché accade

Ogni dataset finito contiene due componenti: il **segnale** (le regolarità che valgono anche fuori dal campione: un rene ha una forma caratteristica) e il **rumore** (specificità del campione: il particolare pattern di rumore di ricostruzione di quello scanner, la particolare posa di quel paziente). Un modello con capacità sufficiente non ha modo di distinguere i due, e minimizzando la loss di training li adatta entrambi. Il rumore non si ripete sui dati nuovi: da lì la caduta di prestazione.

La probabilità di overfitting cresce con: pochi dati, molti parametri, addestramento lungo, assenza di regolarizzazione, classi sbilanciate, e — nel caso medico — bassa varietà di scanner e protocolli nel training set.

28.2 I rimedi, in ordine di efficacia

RIMEDIO	MECCANISMO	NOTE
Più dati	Rende il rumore meno memorizzabile e il segnale più evidente	Il rimedio più efficace in assoluto, e spesso il meno praticabile
Data augmentation	Genera varianti plausibili, simulando più dati	Prima scelta quando il dataset non può crescere (→ Cap. 30)
Early stopping	Ferma l'addestramento al minimo della validation loss	Gratuito; da usare sempre
Weight decay (L2)	Aggiunge $\lambda \ \theta\ ^2$ alla loss, penalizzando pesi grandi	Valori tipici 10^{-4} – 10^{-5}
Dropout	Impedisce la co-adattazione fra unità	Molto efficace sugli strati densi (→ Cap. 15)
Modello più piccolo	Riduce la capacità di memorizzazione	Da valutare, ma spesso preferibile regolarizzare un modello grande
Transfer learning	Parte da feature già valide, riducendo i gradi di libertà effettivi	Utilità limitata in 3D per mancanza di modelli pre-addestrati adeguati

ESEMPIO NUMERICO · RICONOSCERE L'OVERFITTING DAI NUMERI

EPOCA	TRAIN LOSS	VAL LOSS	TRAIN DICE	VAL DICE
10	0.412	0.438	0.71	0.69
30	0.198	0.223	0.86	0.84
60	0.089	0.201	0.94	0.86
90	0.031	0.267	0.98	0.83
120	0.012	0.341	0.99	0.79

Il minimo della validation loss è all'epoca 60 (0.201): è quello il modello da conservare. Dopo, la training loss scende ancora di un fattore sette, ma la validation peggiora del 70% e il Dice di validazione scende da 0.86 a 0.79. La rete sta memorizzando. Si noti che **il divario fra train Dice e val Dice** (0.94 vs 0.86, poi 0.99 vs 0.79) è un indicatore altrettanto leggibile della sola loss.

APPROFONDIMENTO • LA MEMORIZZAZIONE NON È SEMPRE UN MALE

Le reti moderne sono spesso *sovrapparametrizzate*: hanno abbastanza capacità da azzerare la training loss anche con etichette casuali. Eppure generalizzano bene. La teoria classica (varianza-bias, dimensione VC) non spiega adeguatamente il fenomeno, e il tema è tuttora oggetto di ricerca attiva (fenomeni come il *double descent* mostrano che, aumentando ancora la capacità oltre il punto di interpolazione, l'errore di test può tornare a scendere). La lezione operativa non cambia: **si misura sempre su dati non visti**, e non si presume nulla dalla sola dimensione del modello.

ERRORI COMUNI

«**Il modello ha training accuracy del 99%, funziona.**» Un'informazione priva di valore da sola. Un modello che memorizzasse tutte le immagini otterrebbe il 100%.

«**L'overfitting si vede solo dalla loss.**» A volte la loss di validazione risale mentre la metrica clinica (Dice) resta stabile: dipende dal fatto che la cross entropy è sensibile alla confidenza, non solo alla decisione. È il caso in cui il modello diventa *miscalibrato* prima di diventare impreciso — situazione particolarmente rilevante quando si lavora su calibrazione e incertezza.

RIEPILOGO DEL CAPITOLO 28

- Overfitting = adattamento al rumore del training set; si diagnostica confrontando train e validation.
- Favorito da pochi dati, molti parametri, addestramento prolungato.
- Rimedi principali: più dati, augmentation, early stopping, weight decay, dropout.
- La loss di validazione può degradare per miscalibrazione prima ancora che per errori di predizione.

29 Underfitting

L'underfitting è il problema opposto e, in pratica, più facile da diagnosticare: il modello non riesce ad adattarsi nemmeno ai dati che ha visto. Si riconosce perché entrambe le loss restano alte.

DEFINIZIONE

Si ha **underfitting** quando il modello ha errore elevato sia sul training sia sulla validazione: non ha catturato nemmeno le regolarità principali dei dati. In termini di scomposizione dell'errore, è un regime dominato dal **bias** anziché dalla varianza.

29.1 Cause e rimedi

CAUSA	COME SI RICONOSCE	RIMEDIO
Modello troppo semplice	Loss di training alta e stabile anche con molte epoche	Più strati, più filtri, campo recettivo maggiore
Learning rate inadeguato	Loss piatta (troppo basso) o oscillante (troppo alto)	Range test (→ Cap. 23)
Addestramento troppo breve	Loss ancora in discesa quando si interrompe	Continuare
Regolarizzazione eccessiva	Dropout ≥ 0.7 , weight decay molto alto, augmentation estrema	Ridurre gradualmente
Preprocessing sbagliato	Loss che non scende affatto dall'inizio	Verificare normalizzazione, orientamento, allineamento con le etichette
Neuroni morti	Gradienti nulli in molti strati	Leaky ReLU, inizializzazione He, learning rate più basso

APPROFONDIMENTO • IL TEST DIAGNOSTICO DELL'OVERFIT DELIBERATO

Prima di lanciare un addestramento lungo, conviene sempre eseguire questo controllo: si prendono 2–4 esempi e si addestra la rete solo su quelli, senza augmentation e senza regolarizzazione, per qualche centinaio di iterazioni. **Una rete corretta deve riuscire ad azzerare la loss su quattro esempi.** Se non ci riesce, il problema non è nei dati né nella difficoltà del compito: è un bug — etichette disallineate, learning rate impossibile, gradienti che non fluiscono, canale sbagliato, loss calcolata sulla dimensione sbagliata. È il singolo test più utile della pratica quotidiana e costa pochi minuti.

ESEMPIO NUMERICO • TRE SCENARI A CONFRONTO

SCENARIO	TRAIN DICE	VAL DICE	DIAGNOSI
A	0.52	0.50	Underfitting: entrambi bassi, divario nullo
B	0.99	0.71	Overfitting: divario di 0.28
C	0.93	0.90	Buon equilibrio

Regola di lettura: il **livello** della metrica di training indica se c'è capacità sufficiente; il **divario** fra training e validation indica se c'è generalizzazione. Sono due diagnosi indipendenti e vanno lette insieme.

ERRORI COMUNI

«**La rete non impara, aggiungo strati.**» Prima verificare learning rate, normalizzazione dei dati e correttezza delle etichette. L'aumento di capacità è l'ultima cosa da provare, non la prima: nella grande maggioranza dei casi l'underfitting apparente è un bug di pipeline.

«**Underfitting e overfitting sono i due estremi di un unico asse.**» Vero come schema didattico, ma nella pratica si possono avere entrambi contemporaneamente su sottopopolazioni diverse: il modello sovra-adatta i casi frequenti e sotto-adatta quelli rari.

RIEPILOGO DEL CAPITOLO 29

- Underfitting = errore alto ovunque, divario train/val nullo; regime dominato dal bias.
- Cause frequenti: bug di pipeline, learning rate sbagliato, regolarizzazione eccessiva, capacità insufficiente.
- Il test dell'overfit deliberato su pochi esempi distingue un bug da una difficoltà reale.
- Livello della metrica → capacità; divario train/val → generalizzazione.

30 Data Augmentation

La data augmentation genera varianti plausibili degli esempi di addestramento. In ambito medico, dove i dati annotati sono scarsi e costosi, non è un accessorio: è uno dei fattori che più determinano la qualità finale del modello.

DEFINIZIONE

La **data augmentation** applica agli esempi di training trasformazioni casuali che ne preservano l'etichetta, esponendo il modello a una varietà di input molto maggiore di quella del dataset originale. Le trasformazioni sono ricampionate a ogni epoca: la rete non vede mai due volte esattamente la stessa immagine.

30.1 Il principio: insegnare le invarianze

Ogni augmentation comunica alla rete un'invarianza desiderata: "una CT ruotata di 8 gradi contiene lo stesso rene", "una CT con un po' più di rumore contiene lo stesso rene". Il modello è quindi spinto a costruire rappresentazioni robuste a quelle variazioni. Ne segue il criterio di scelta fondamentale: **si applicano solo trasformazioni che potrebbero realisticamente presentarsi al momento del test, e che non alterano l'etichetta corretta.**

30.2 Trasformazioni tipiche

TRASFORMAZIONE	PARAMETRI TIPICI	ADATTA IN AMBITO MEDICO?
Rotazione	$\pm 15^\circ$ (2D), $\pm 30^\circ$ (3D)	Sì — la posa del paziente varia
Scala	0.85 – 1.25	Sì — corporature e campi di vista diversi
Flip	Assi spaziali	Con cautela: il flip sinistra-destra scambia gli organi e può essere anatomicamente scorretto (situs inversus a parte, il fegato è a destra)
Deformazione elastica	Campo di spostamento morbido	Sì — molto efficace: simula la variabilità anatomica reale
Rumore gaussiano	$\sigma = 0 - 0.1$	Sì — simula dosi e ricostruzioni diverse
Sfocatura gaussiana	$\sigma = 0.5 - 1.5$	Sì — simula kernel di ricostruzione più morbidi
Variazione di luminosità/contrasto	$\pm 25\%$	Sì, con misura: su CT la scala HU è fisica e stravolgerla riduce il realismo
Simulazione di bassa risoluzione	Downsampling e ritorno	Sì — robustezza a spacing diversi
Correzione gamma	$\gamma = 0.7 - 1.5$	Sì
Ritaglio casuale	Patch	Necessario per motivi di memoria, e agisce anche da augmentation

Tabella 30.1 — La pipeline di nnU-Net applica in sequenza rotazione e scala, rumore gaussiano, sfocatura, variazione di luminosità e contrasto, simulazione di bassa risoluzione, correzione gamma e mirroring, ciascuna con una propria probabilità di attivazione.

LA REGOLA NON NEGOZIABILE DELLA SEGMENTAZIONE

Le trasformazioni **geometriche** vanno applicate **identiche** all'immagine e alla maschera di ground truth. Se si ruota l'immagine di 12° e non la maschera, si è appena insegnato alla rete un'associazione sbagliata, e il danno è tanto più insidioso quanto più è silenzioso: nessun errore viene sollevato, semplicemente il modello impara peggio.

Con una differenza tecnica importante: l'immagine si interpola in modo continuo (lineare o spline), la maschera **obbligatoriamente** con nearest neighbour. Interpolare linearmente delle etichette intere produrrebbe valori come 0.5 fra la classe 0 e la classe 1 — un'etichetta inesistente.

Le trasformazioni di **intensità** (rumore, luminosità, gamma) si applicano invece solo all'immagine: la maschera non ha intensità.

ESEMPIO NUMERICO • LA VARIETÀ EFFETTIVA

Dataset di 45 volumi. Con rotazione (continua in $\pm 30^\circ$), scala (continua in 0.85–1.25), flip su 3 assi, rumore e gamma casuali, e ritaglio della patch in posizione casuale, ogni volume genera un numero di varianti distinte praticamente illimitato.

Su 1000 epoche da 250 iterazioni con batch 2, la rete vede $1000 \times 250 \times 2 = 500\,000$ patch. Nessuna coppia identica. Il dataset "effettivo" non è di 45 elementi ma di mezzo milione di osservazioni correlate — che è ben diverso da mezzo milione di pazienti indipendenti, ma è ciò che rende possibile addestrare un modello con centinaia di migliaia di parametri su 45 casi.

APPROFONDIMENTO • TEST-TIME AUGMENTATION

L'augmentation può essere usata anche in inference: si predice sull'immagine originale e su alcune sue trasformazioni (tipicamente i mirroring lungo i vari assi), si riportano le predizioni al sistema di riferimento originale e si mediano. Il risultato è tipicamente un guadagno di Dice modesto ma consistente, e una stima di incertezza aggiuntiva a costo quasi nullo. nnU-Net lo attiva di default con il mirroring.

ERRORI COMUNI

«Applicare augmentation anche al validation set.» No. La validazione deve misurare la prestazione su dati realistici e non alterati (salvo la TTA, che è una scelta esplicita e va applicata coerentemente anche in test).

«Più augmentation è sempre meglio.» Trasformazioni troppo aggressive producono immagini non realistiche e spostano la distribuzione di training lontano da quella di test: si passa a underfitting.

«L'augmentation sostituisce i dati veri.» No: non introduce nuova informazione anatomica. Insegna invarianze, non nuovi pazienti.

RIEPILOGO DEL CAPITOLO 30

- L'augmentation genera varianti plausibili e insegna invarianze; è il principale antidoto all'overfitting quando i dati sono pochi.
- Le trasformazioni geometriche vanno applicate identiche a immagine e maschera, con nearest neighbour sulla maschera.
- Le trasformazioni di intensità si applicano solo all'immagine.
- Mai in validazione, salvo la test-time augmentation applicata deliberatamente.

31 Pipeline completa della classificazione

Questo capitolo ricompone tutti i pezzi in un unico percorso, seguendo una singola immagine dall'ingresso all'aggiornamento dei pesi. Serve da verifica: se ogni passaggio risulta chiaro, i capitoli precedenti sono stati assimilati.

31.1 Impostazione dell'esempio

Compito: classificare una patch 24×24 estratta da una slice CT in tre categorie — 0 = solo tessuto circostante, 1 = contiene parenchima renale, 2 = contiene lesione. Architettura: quella della Tabella 3.1. Batch $m = 32$, $\eta = 0.01$, loss cross entropy, ottimizzatore SGD con momentum.

31.2 Il percorso, passo per passo

#	FASE	CHE COSA ACCADE	FORMA DEL TENSORE
1	Caricamento	Lettura dal disco, ricampionamento a spacing comune, clipping HU, z-score	24 × 24 × 3
2	Augmentation	Rotazione casuale, rumore, gamma — solo in training	24 × 24 × 3
3	Batching	32 patch impilate lungo un nuovo asse	32 × 3 × 24 × 24
4	Conv 1	64 filtri 3×3×3, padding 1: prodotti scalari locali in ogni posizione	32 × 64 × 24 × 24
5	BatchNorm	Standardizzazione per canale, poi γ, β	invariata
6	ReLU	Azzeramento dei valori negativi ($\approx$ 50% di sparsità)	invariata
7	MaxPool 2×2	Massimo per quadrante, canali invariati	32 × 64 × 12 × 12
8	Conv 2 + BN + ReLU	128 filtri 3×3×64: combinano le risposte del primo strato	32 × 128 × 12 × 12
9	MaxPool 2×2	Seconda riduzione	32 × 128 × 6 × 6
10	Flatten	Reshape: la struttura spaziale diventa inaccessibile	32 × 4608
11	FC 256 + ReLU	Combinazione globale di tutte le feature	32 × 256
12	Dropout 0.5	Metà delle unità azzerata, riscalatura ×2 (solo training)	32 × 256
13	FC 3	Ultimo strato lineare: nessuna attivazione	32 × 3 (logits)
14	Softmax + CE	Normalizzazione e confronto con il ground truth	32 valori → 1 scalare
15	Backward	Regola della catena a ritroso: gradiente per ogni parametro	un gradiente per parametro
16	Update	$\theta \leftarrow \theta - \eta \mathbf{v}$	—
17	Zero grad	Azzeramento dei gradienti accumulati	—

Tabella 31.1 — Una iterazione completa. I passi 4–14 sono il forward, 15 il backward, 16–17 l'ottimizzazione. Si ripete per ogni mini-batch, per ogni epoca.

ESEMPIO NUMERICO · UN SINGOLO ESEMPIO DEL BATCH, DALL'INIZIO ALLA FINE

Ground truth: classe 1 (parenchima renale) $\rightarrow$ one-hot $\mathbf{y} = [0, 1, 0]$.

Passo 4. Il filtro 17 del primo strato, nella posizione (5,5), calcola $\sum K \odot X + b = 20.0$ (l'esempio svolto al capitolo 7).

Passo 5. La BN, per il canale 17, ha $\mu = 12.4$, $\sigma = 8.1$, $\gamma = 1.05$, $\beta = -0.2$:

$$\hat{x} = \frac{20.0 - 12.4}{8.1} = 0.938 \quad \Rightarrow \quad y = 1.05(0.938) - 0.2 = 0.785$$

Passo 6. $\text{ReLU}(0.785) = 0.785$: sopravvive.

Passi 7–9. Il valore compete nel pooling con i tre vicini del quadrante; supponiamo che sia il massimo e passi. Prosegue attraverso il secondo blocco.

Passo 13. Logits finali: $\mathbf{z} = [0.4, 2.1, -0.8]$.

Passo 14. Softmax: $e^{0.4} = 1.492$, $e^{2.1} = 8.166$, $e^{-0.8} = 0.449$; somma = 10.107.

$$\hat{\mathbf{y}} = [0.1476, 0.8079, 0.0444] \quad \mathcal{L} = -\ln(0.8079) = 0.2133$$

Passo 15. Gradiente sui logits: $\hat{\mathbf{y}} - \mathbf{y} = [+0.1476, -0.1921, +0.0444]$. Da qui il gradiente risale attraverso FC, dropout, flatten, pooling, ReLU, BN, fino ai 27 pesi del filtro 17 del primo strato, che riceve la sua correzione insieme a tutti gli altri.

Passo 16. Tutti i parametri — kernel, bias, γ , β , matrici dense — vengono aggiornati simultaneamente, usando il gradiente **mediato** sui 32 esempi del batch.

31.3 Che cosa cambia in inference

COMPONENTE	TRAINING	INFERENCE
Augmentation	Attiva	Disattivata (salvo TTA)
Dropout	Attivo	Disattivato (salvo MC dropout)
BatchNorm	Statistiche del batch	Medie mobili accumulate
Backward e update	Eseguiti	Non eseguiti
Memoria attivazioni	Conservate per il backward	Scartate subito
Loss	Calcolata	Calcolata solo se sono disponibili le etichette

RIEPILOGO DEL CAPITOLO 31

- Una iterazione = forward (dati $\rightarrow$ logits $\rightarrow$ loss), backward (loss $\rightarrow$ gradienti), update (gradienti $\rightarrow$ pesi).
- Il ground truth interviene solo al passo 14; tutto il resto è propagazione di tensori.
- L'aggiornamento è unico per mini-batch e coinvolge simultaneamente tutti i parametri.
- Training e inference differiscono in modo sostanziale: dropout, batch norm e augmentation cambiano comportamento.

32 Dalla classificazione alla segmentazione

Il passaggio alla segmentazione richiede una sola idea nuova e nessuna matematica nuova. Tutto quello che serve — convoluzione, ReLU, softmax, cross entropy, backpropagation — è già stato costruito. Cambiano la forma dell'output e la forma del ground truth.

32.1 Il confronto strutturale

	CLASSIFICAZIONE	SEGMENTAZIONE SEMANTICA
Domanda	Che cosa c'è in questa immagine?	A quale classe appartiene ciascun pixel?
Output	Vettore di K probabilità	Tensore $H \times W \times K$ di probabilità
Ground truth	Un intero (o un one-hot di K)	Una maschera $H \times W$ di interi
Numero di decisioni	1 per immagine	$H \cdot W$ per immagine (262 144 su una slice 512 ²)
Loss	Cross entropy sul vettore	Cross entropy media su tutti i pixel, spesso + Dice
Proprietà richiesta	Invarianza alla traslazione	Equivarianza alla traslazione
Testa della rete	Flatten + FC	Conv 1×1, nessun flatten

32.2 La trasformazione in rete fully convolutional

Riprendiamo l'osservazione del capitolo 17: uno strato denso applicato posizione per posizione è una convoluzione 1×1 . Costruiamo allora la testa in questo modo: dato il tensore di feature $H \times W \times C$ prodotto dall'encoder, applichiamo una convoluzione 1×1 con $C_{in} = C$ e $C_{out} = K$.

$$H \times W \times C \xrightarrow{\text{conv } 1 \times 1, K \text{ filtri}} H \times W \times K$$

Il risultato è un tensore di logits **per pixel**: in ogni posizione (i, j) c'è un vettore di K logits. Applicando softmax lungo l'asse dei canali si ottiene, per ogni pixel, una distribuzione di probabilità sulle K classi. La predizione finale è $\arg \max_k$ per pixel.

INTUIZIONE · UN CLASSIFICATORE CHE SCORRE

Si può pensare alla rete fully convolutional come a un classificatore applicato simultaneamente a tutte le posizioni possibili — con l'enorme differenza che i calcoli condivisi vengono eseguiti una volta sola, invece di essere ripetuti per ogni finestra. È lo stesso guadagno di efficienza che la convoluzione offre rispetto alla rete densa, applicato al livello dell'intera architettura.

32.3 La loss per pixel

La cross entropy si estende nel modo più naturale possibile: si calcola per ogni pixel e si media.

$$\mathcal{L}_{CE} = -\frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W \sum_{k=1}^K y_{ijk} \log \hat{y}_{ijk} = -\frac{1}{HW} \sum_{i,j} \log \hat{y}_{ij,c_{ij}}$$

dove c_{ij} è la classe corretta del pixel (i, j) . Il gradiente rispetto ai logits resta $\hat{y}_{ijk} - y_{ijk}$, **identico** alla formula del capitolo 21, applicata in ogni posizione. Non c'è nulla di nuovo da imparare: cambia solo il numero di termini nella somma.

ESEMPIO NUMERICO · LOSS SU UNA MICRO-IMMAGINE 2×2

Due classi (0 = sfondo, 1 = rene). Ground truth: $\begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}$. Probabilità predette per la classe 1:

$$\hat{p}_1 = \begin{pmatrix} 0.10 & 0.85 \\ 0.30 & 0.60 \end{pmatrix}$$

Cross entropy pixel per pixel (usando la probabilità della classe *corretta*):

$$-\ln(0.90) = 0.105 \quad -\ln(0.85) = 0.163 \quad -\ln(0.70) = 0.357 \quad -\ln(0.60) = 0.511$$

$$\mathcal{L}_{CE} = \frac{0.105 + 0.163 + 0.357 + 0.511}{4} = 0.284$$

Il pixel in basso a sinistra (0.30 di probabilità per la classe sbagliata) e quello in basso a destra (solo 0.60 di confidenza sulla classe corretta) contribuiscono la maggior parte della loss, e quindi la maggior parte del gradiente. La rete verrà corretta soprattutto lì.

Dice soft sulla stessa predizione, con $\hat{p}$ probabilità e y maschera:

$$\text{Dice} = \frac{2(0.10 \cdot 0 + 0.85 \cdot 1 + 0.30 \cdot 0 + 0.60 \cdot 1)}{(0.10 + 0.85 + 0.30 + 0.60) + (0 + 1 + 0 + 1)} = \frac{2(1.45)}{1.85 + 2} = \frac{2.90}{3.85} = 0.753$$

$$\mathcal{L}_{\text{Dice}} = 1 - 0.753 = 0.247 \quad \mathcal{L}_{\text{tot}} = 0.284 + 0.247 = 0.531$$

32.4 Il problema che resta aperto

La costruzione appena descritta funziona, ma ha un difetto grave. L'encoder ha ridotto la risoluzione, diciamo di 16 volte: da 512×512 a 32×32 . La conv 1×1 produce quindi una mappa di logits $32 \times 32 \times K$, che va riportata a 512×512 per essere confrontata con il ground truth. Un semplice ingrandimento per interpolazione produce contorni grossolani: ogni pixel della mappa a bassa risoluzione si espande in un blocco di 16×16 pixel, e la precisione del bordo — che in segmentazione medica è *la* quantità che conta — va perduta.

LA TENSIONE FONDAMENTALE

Per **riconoscere** serve contesto ampio, cioè un campo recettivo grande, cioè downsampling. Per **localizzare con precisione** serve alta risoluzione, cioè assenza di downsampling. Le due esigenze sono in conflitto diretto.

La soluzione — un percorso discendente che costruisce il contesto, un percorso ascendente che recupera la risoluzione, e connessioni dirette fra i due che reiniettano il dettaglio — è l'architettura U-Net.

RIEPILOGO DEL CAPITOLO 32

- La segmentazione è classificazione ripetuta per pixel; la matematica è la stessa.
- Si elimina il flatten e si sostituisce la testa densa con una convoluzione 1×1 a K canali.
- Softmax si applica lungo l'asse dei canali, la loss si media su tutti i pixel.
- Resta il conflitto fra contesto (downsampling) e precisione dei bordi (alta risoluzione).

33 Introduzione a U-Net

U-Net risolve il conflitto del capitolo precedente con una struttura simmetrica: un encoder che comprime, un decoder che ricostruisce, e skip connection che trasferiscono direttamente il dettaglio ad alta risoluzione da un lato all'altro.

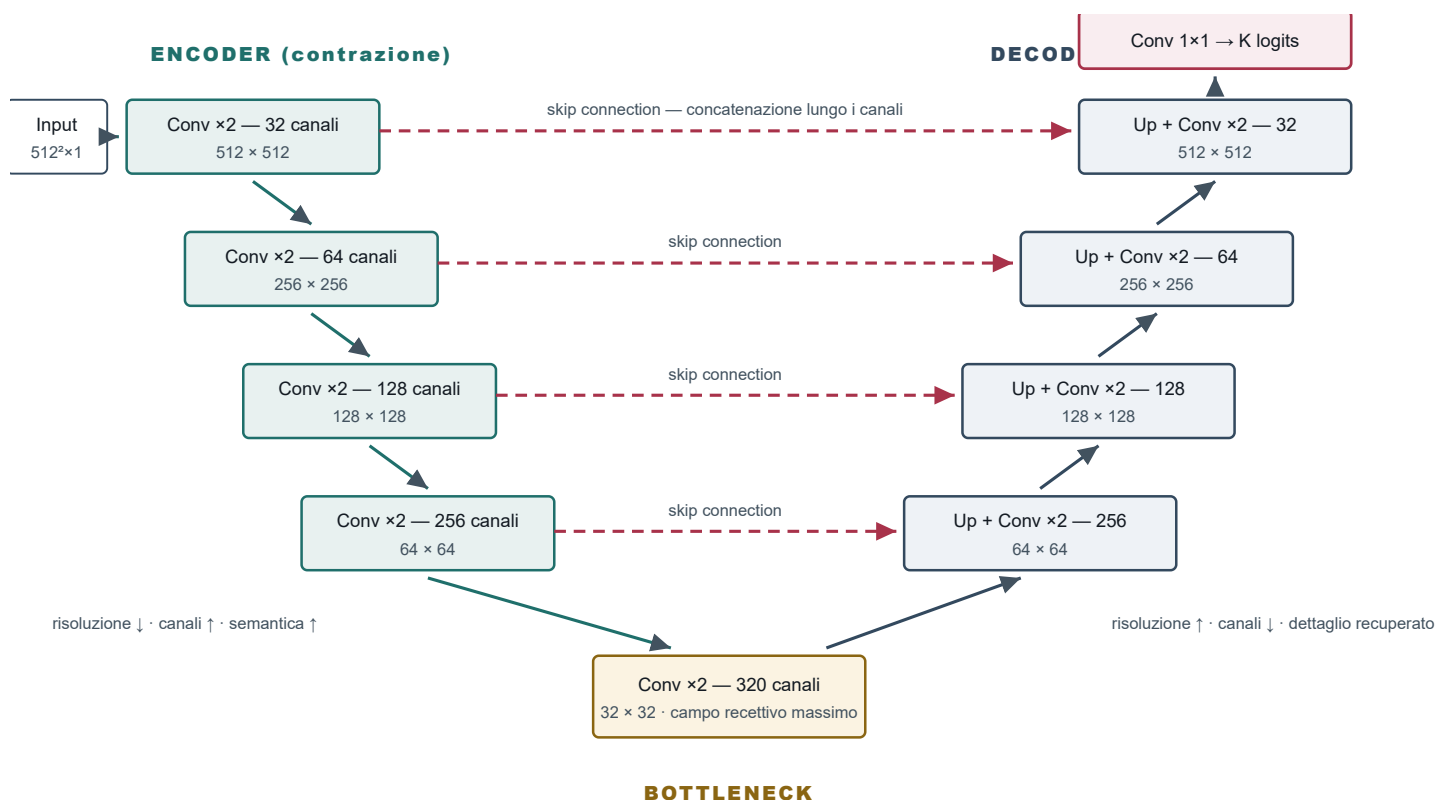


Figura 33.1 — Architettura U-Net. Il ramo sinistro riduce la risoluzione costruendo contesto; il ramo destro la ricostruisce. Le connessioni orizzontali trasferiscono le feature ad alta risoluzione dall'encoder al decoder, dove vengono concatenate lungo l'asse dei canali.

33.1 Encoder

È esattamente la CNN dei capitoli 6–13: blocchi di due convoluzioni 3×3 con normalizzazione e non linearità, seguiti da un dimezzamento della risoluzione (pooling o convoluzione con stride 2). A ogni livello i canali raddoppiano. Al fondo, il **bottleneck** contiene la rappresentazione più astratta e con il campo recettivo più ampio: "qui c'è tessuto renale, in una configurazione retroperitoneale, adiacente a fegato e colonna". Ciò che manca è la precisione: quella posizione a 32×32 copre un blocco di 16×16 pixel dell'immagine originale.

33.2 Decoder e convoluzione trasposta

Il decoder inverte il percorso. A ogni livello: si raddoppia la risoluzione, si concatena la skip connection corrispondente, si applicano due convoluzioni. L'upsampling avviene in due modi possibili:

- **Interpolazione** (bilineare o nearest) seguita da una convoluzione 3×3 . Semplice, senza artefatti, con i parametri concentrati nella convoluzione.
- **Convoluzione trasposta** (impropriamente detta "deconvoluzione"): un'operazione appresa che si può descrivere come inserimento di zeri fra i pixel seguito da una convoluzione ordinaria. La dimensione dell'uscita è

$$H_{out} = (H_{in} - 1) \cdot s - 2p + k$$

Con $k = 2, s = 2, p = 0$: $H_{out} = 2H_{in}$, il raddoppio esatto. La convoluzione trasposta è più espressiva ma può produrre il caratteristico artefatto a scacchiera quando k non è multiplo di s .

ESEMPIO NUMERICO • VERIFICA DELLE DIMENSIONI NEL DECODER

Livello del bottleneck: $32 \times 32 \times 320$.

1. **Upsample** $\times 2$ con conv trasposta 2×2 , 256 filtri $\rightarrow 64 \times 64 \times 256$.
2. **Skip** dall'encoder di livello 4: $64 \times 64 \times 256$.
3. **Concatenazione** lungo i canali $\rightarrow 64 \times 64 \times 512$. *Si noti: i canali si sommano, la risoluzione deve coincidere.*
4. **Due conv** 3×3 , 256 filtri $\rightarrow 64 \times 64 \times 256$.

Si ripete fino a $512 \times 512 \times 32$, poi conv 1×1 con K filtri $\rightarrow 512 \times 512 \times K$, e infine softmax lungo i canali. L'output ha la stessa risoluzione dell'input: una predizione per pixel.

33.3 Skip connection: perché sono la vera idea

DEFINIZIONE

Una **skip connection** collega direttamente l'uscita di un blocco dell'encoder all'ingresso del blocco del decoder alla stessa risoluzione, concatenandola lungo l'asse dei canali (nella U-Net originale; in altre architetture si somma).

Tre funzioni distinte, tutte importanti:

1. **Recupero del dettaglio spaziale.** Le feature dell'encoder a 512×512 contengono la posizione esatta dei bordi, informazione che il bottleneck ha necessariamente perso. Il decoder combina il "che cosa" (dal basso) con il "dove esattamente" (dalla skip).
2. **Percorso diretto per il gradiente.** In backward il gradiente può risalire agli strati iniziali senza attraversare tutto il bottleneck, attenuando il problema del gradiente evanescente e rendendo addestrabili reti profonde.
3. **Fusione multi-scala.** Ogni livello del decoder vede simultaneamente informazione a due granularità diverse, e la convoluzione successiva impara come pesarle.

INTUIZIONE - IL RADIOLOGO E LALENTE

Il percorso discendente equivale ad allontanarsi dall'immagine per capire quale organo si sta guardando; il percorso ascendente equivale a riavvicinarsi per tracciare il contorno. Senza skip connection sarebbe come dover ridisegnare il bordo a memoria dopo essersi allontanati. Con le skip, si torna ad avere sotto gli occhi l'immagine ad alta risoluzione mentre si conosce già l'interpretazione anatomica.

33.4 Da U-Net a nnU-Net

U-Net è una *architettura*. nnU-Net ("no new net") è un **framework di auto-configurazione** costruito attorno a essa: partendo da un'analisi automatica del dataset — spacing, dimensioni, intensità, numero di classi, modalità — deriva con regole esplicite tutte le scelte progettuali che di norma richiedono esperienza e tentativi.

SCELTA	COME VIENE DERIVATA
Spacing target	Mediana degli spacing del dataset (con regole specifiche per l'anisotropia)
Normalizzazione	CT → clipping ai percentili e z-score globale; altre modalità → z-score per caso
Patch size e batch size	Massimizzate compatibilmente con un budget di memoria GPU fissato
Numero di downsampling	Aumentato finché la dimensione della feature map più piccola non scende sotto una soglia
Normalizzazione dei layer	Instance Norm, per compatibilità con batch molto piccoli
Loss	Dice + Cross Entropy, con deep supervision sui livelli del decoder
Ottimizzatore	SGD, momentum di Nesterov 0.99, $\eta_0 = 0.01$, decadimento polinomiale
Addestramento	1000 epoche da 250 iterazioni, 5-fold cross-validation
Inference	Predizione a finestra scorrevole con pesatura gaussiana + test-time mirroring

Tabella 33.1 — L'insieme delle "regole fisse ed euristiche" di nnU-Net. Ogni riga è una decisione che un ricercatore prenderebbe manualmente; il valore del framework è averle codificate e validate su decine di dataset eterogenei.

Il risultato metodologicamente interessante è che questa configurazione automatica, applicata a una U-Net del tutto standard, batte regolarmente architetture molto più elaborate. La lezione — utile ben oltre il caso specifico — è che in segmentazione medica **la pipeline conta più dell'architettura**.

APPROFONDIMENTO • DOVE SI INNESTANO CALIBRAZIONE E INCERTEZZA

Tutto ciò che è stato costruito in questa dispensa converge qui. L'uscita di U-Net è, per ogni voxel, un vettore di logits (→ Cap. 18) trasformato da softmax in probabilità (→ Cap. 19). Quelle probabilità sono l'oggetto su cui si lavora:

- **Calibrazione:** le probabilità per voxel sono affidabili? Il temperature scaling agisce sui logits *prima* di softmax; metriche come ECE e Brier score misurano lo scarto fra confidenza dichiarata e accuratezza osservata, e vanno calcolate su un insieme non usato per stimare T .
- **Incerteza:** MC dropout (→ Cap. 15) o deep ensemble producono una distribuzione di predizioni per voxel; la sua entropia o varianza costituisce la mappa di incerteza. Tipicamente alta sui bordi dell'organo, sulle lesioni ambigue e sui casi fuori distribuzione — che è esattamente l'informazione di cui un clinico ha bisogno per decidere dove verificare manualmente.
- **Soglie:** la scelta della soglia di binarizzazione, o la sua calibrazione per ottimizzare una metrica specifica, opera anch'essa sulle probabilità, e va compiuta su validazione — mai sul test (→ Cap. 27).

RIEPILOGO DEL CAPITOLO 33

- U-Net = encoder (contesto) + decoder (risoluzione) + skip connection (dettaglio e cammino per il gradiente).
- L'upsampling avviene per interpolazione + conv oppure per convoluzione trasposta appresa.
- Le skip connection concatenano lungo i canali: la risoluzione deve coincidere, i canali si sommano.
- nnU-Net auto-configura preprocessing, architettura e training; la pipeline conta più dell'architettura.
- Calibrazione e quantificazione dell'incerteza operano sui logits e sulle probabilità per voxel prodotti da questa architettura.

34 Glossario

Definizioni compatte di tutta la terminologia introdotta, in ordine alfabetico. Ogni voce rimanda al capitolo in cui l'argomento è trattato per esteso.

Attivazione

Valore in uscita da un neurone o da una posizione di una feature map, dopo l'applicazione della funzione di attivazione. Non è un parametro: è un dato che scorre nella rete. (Cap. 12)

Augmentation (data)

Trasformazione casuale applicata agli esempi di training che ne preserva l'etichetta, per aumentare la varietà effettiva del dataset e insegnare invarianze. (Cap. 30)

Backpropagation

Algoritmo di calcolo dei gradienti basato sull'applicazione a ritroso della regola della catena, con riuso dei risultati intermedi. Calcola le derivate; non aggiorna i pesi. (Cap. 24)

Batch (mini-batch)

Gruppo di m esempi processati insieme, che produce un singolo aggiornamento dei parametri. (Cap. 26)

Batch Normalization

Normalizzazione delle attivazioni per canale usando media e varianza del mini-batch, seguita da riscalatura appresa γ , β . Comportamento diverso in training e inference. (Cap. 14)

Bias (parametro)

Termine additivo di una trasformazione affine; uno per filtro convoluzionale o per neurone denso. (Cap. 6, 17)

Bias (statistico)

Componente dell'errore dovuta a un modello troppo semplice per rappresentare il fenomeno; è il regime dell'underfitting. (Cap. 29)

Bottleneck

Punto di minima risoluzione spaziale e massima profondità semantica di un'architettura encoder-decoder. (Cap. 33)

Calibrazione

Proprietà per cui le probabilità predette corrispondono alle frequenze empiriche di correttezza. Misurata con ECE e Brier score, corretta a posteriori con temperature scaling. (Cap. 18)

Campo recettivo

Regione dell'immagine di input che può influenzare il valore di una specifica posizione di una feature map. Cresce con la profondità ed è moltiplicato dal downsampling. (Cap. 10)

Canale

Asse del tensore che indicizza le diverse mappe: 3 per un'immagine RGB, 1 per una CT, C_{out} dopo uno strato convoluzionale. Non ha struttura di vicinanza. (Cap. 4)

Convoluzione

Operazione che applica lo stesso kernel in tutte le posizioni dell'input, calcolando in ciascuna un prodotto scalare fra kernel e patch, sommando su tutti i canali di ingresso e aggiungendo un bias. È lineare. (Cap. 7)

Convoluzione 1×1

Convoluzione senza estensione spaziale: mescola i canali nella stessa posizione. Equivale a uno strato denso applicato pixel per pixel; è il ponte fra classificazione e segmentazione. (Cap. 17, 32)

Convoluzione trasposta

Operazione appresa di upsampling, descrivibile come inserimento di zeri seguito da convoluzione.

$$H_{out} = (H_{in} - 1)s - 2p + k. \text{ (Cap. 33)}$$

Cross entropy

Loss di classificazione $-\log \hat{y}_c$, pari al logaritmo negativo della probabilità assegnata alla classe corretta. Deriva dalla massima verosimiglianza; gradiente sui logits $\hat{\mathbf{y}} - \mathbf{y}$. (Cap. 21)

Cross-validation (K-fold)

Protocollo che divide il dataset in K parti, addestra K modelli con validazione a rotazione e media le metriche. Fornisce anche un ensemble. (Cap. 27)

Decoder

Ramo ascendente di un'architettura di segmentazione: ricostruisce progressivamente la risoluzione spaziale integrando le skip connection. (Cap. 33)

Deep supervision

Applicazione della loss anche a livelli intermedi del decoder, per migliorare il flusso del gradiente. Usata da nnU-Net. (Cap. 33)

Dice (coefficiente)

Metrica di sovrapposizione $2|A \cap B|/(|A| + |B|)$, insensibile alla dimensione dello sfondo. Nella versione differenziabile diventa Dice loss. (Cap. 20)

Dropout

Azzeramento casuale di unità durante il training, con riscalatura $1/(1 - p)$; regolarizzatore ed ensemble implicito. Se lasciato attivo in inference genera il MC dropout. (Cap. 15)

Early stopping

Interruzione dell'addestramento al minimo della loss di validazione, conservando quel modello. (Cap. 27, 28)

Encoder

Ramo discendente: riduce la risoluzione spaziale e aumenta il numero di canali, costruendo rappresentazioni sempre più astratte. (Cap. 3, 33)

Epoca

Un passaggio completo su tutto il training set, pari a $\lceil N/m \rceil$ iterazioni. (Cap. 25)

Equivarianza

Proprietà per cui una traslazione dell'input produce la stessa traslazione dell'output. È la proprietà della convoluzione, ed è indispensabile in segmentazione (da distinguere dall'invarianza). (Cap. 2)

Feature map

Matrice delle risposte di un filtro in tutte le posizioni dell'input. I suoi valori misurano allineamento con il pattern del filtro, non colori né grandezze assolute. (Cap. 10)

Filtro

Sinonimo di kernel: tensore di pesi appresi di forma (k_h, k_w, C_{in}) più un bias. (Cap. 6)

Flatten

Reshape da tensore a vettore. Non altera i valori; rende inaccessibile la struttura spaziale. Assente nelle reti di segmentazione. (Cap. 16)

Fully connected (denso)

Strato in cui ogni uscita dipende da tutti gli ingressi: $\mathbf{z} = W\mathbf{x} + \mathbf{b}$. Nessuna condivisione di pesi. (Cap. 17)

Fully convolutional network

Rete priva di strati densi, che produce un output con struttura spaziale; accetta input di dimensione variabile. (Cap. 32)

Gradient accumulation

Tecnica che somma i gradienti di più micro-batch prima di un unico aggiornamento, per simulare un batch grande con poca memoria. (Cap. 26)

Gradiente

Vettore delle derivate parziali della loss rispetto ai parametri; indica la direzione di massima crescita della loss. (Cap. 22)

Ground truth

Annotazione di riferimento fornita da un esperto. Interviene una sola volta, nel calcolo della loss sull'output finale; mai sulle feature map. (Cap. 20)

Hounsfield (unità)

Scala di attenuazione dei raggi X con significato fisico assoluto: -1000 aria, 0 acqua, +1000 osso corticale. Consente clipping anatomico e normalizzazione globale. (Cap. 5)

Inizializzazione

Assegnazione dei valori iniziali dei pesi. He per ReLU ($\sigma^2 = 2/n_{in}$), Xavier per attivazioni simmetriche; rompe la simmetria e preserva la varianza del segnale. (Cap. 6)

Instance Normalization

Normalizzazione con statistiche calcolate dentro il singolo esempio, indipendente dalla dimensione del batch. Standard in segmentazione 3D. (Cap. 14)

Invarianza

Proprietà per cui una trasformazione dell'input non modifica l'output. Ottenuta con pooling e aggregazione globale; desiderabile in classificazione, dannosa se totale in segmentazione. (Cap. 2, 13)

Iperparametro

Parametro non appreso dal gradiente ma scelto dall'esterno: learning rate, batch size, numero di filtri, probabilità di dropout. (Cap. 23)

Iterazione (step)

Ciclo forward → loss → backward → update su un mini-batch. (Cap. 25)

Kernel

Vedi Filtro. (Cap. 6)

Learning rate

Fattore η che scala il gradiente nell'aggiornamento. L'iperparametro più influente; tipicamente ridotto nel tempo con uno schedule. (Cap. 23)

Logits

Uscita lineare non normalizzata dell'ultimo strato, un valore per classe. Invarianti per traslazione additiva; la loro scala determina la confidenza. (Cap. 18)

Loss

Funzione scalare differenziabile che misura la discrepanza fra predizione e ground truth. Definisce l'obiettivo dell'ottimizzazione. (Cap. 20)

MAC

Moltiplicazione-accumulazione; unità di misura del costo computazionale. Per una conv: $H_{out}W_{out}C_{out}k^2C_{in}$. (Cap. 7)

MC dropout

Uso del dropout in inference con T passaggi ripetuti; media e varianza delle predizioni forniscono stima e incertezza. (Cap. 15)

Momentum

Media mobile dei gradienti passati che introduce inerzia nell'aggiornamento, smorzando le oscillazioni e accelerando lungo direzioni coerenti. (Cap. 22)

Neurone morto

Unità la cui pre-attivazione è sempre negativa: ReLU la azzera sempre, il gradiente è sempre nullo e i suoi pesi non cambiano più. (Cap. 12)

nnU-Net

Framework che auto-configura preprocessing, architettura U-Net, training e inference a partire da un'analisi automatica del dataset. (Cap. 33)

One-hot

Codifica di un'etichetta categorica come vettore di zeri con un solo 1 nella posizione della classe corretta. (Cap. 21)

Ottimizzatore

Algoritmo che usa i gradienti per aggiornare i parametri: SGD, SGD con momentum, Adam. (Cap. 22)

Overfitting

Adattamento al rumore del training set: la loss di training scende mentre quella di validazione risale. (Cap. 28)

Padding

Cornice artificiale aggiunta all'input per consentire l'applicazione del kernel ai pixel di bordo. Con $p = (k - 1)/2$ e stride 1 preserva la dimensione. (Cap. 8)

Parametro

Valore appreso dal gradiente: pesi dei kernel, bias, γ e β della normalizzazione, matrici degli strati densi. (Cap. 3)

Patch

(1) Finestra locale dell'input coperta dal kernel; (2) ritaglio di un volume usato come esempio di training quando il volume intero non entra in memoria. (Cap. 5, 7)

Pooling

Aggregazione di finestre di attivazioni (massimo o media) che riduce la risoluzione spaziale senza modificare il numero di canali e senza parametri. (Cap. 13)

Regolarizzazione

Insieme di tecniche che limitano l'overfitting: weight decay, dropout, augmentation, early stopping. (Cap. 28)

ReLU

$\max(0, z)$, applicata elemento per elemento. Introduce non linearità, produce sparsità, ha gradiente 1 o 0. Varianti: Leaky ReLU, PReLU, GELU. (Cap. 12)

Ricampionamento (resampling)

Conversione di un volume a uno spacing comune, necessaria affinché il campo recettivo del kernel abbia dimensione fisica comparabile fra pazienti. (Cap. 5)

Segmentazione semantica

Assegnazione di una classe a ogni pixel o voxel. Output $H \times W \times K$; loss mediata su tutti i pixel. (Cap. 32)

Skip connection

Collegamento diretto fra un livello dell'encoder e il corrispondente livello del decoder, tipicamente per concatenazione lungo i canali. Recupera il dettaglio spaziale e facilita il flusso del gradiente. (Cap. 33)

Softmax

$p_k = e^{z_k} / \sum_j e^{z_j}$: normalizza i logits in una distribuzione di probabilità. In segmentazione si applica lungo l'asse dei canali. (Cap. 19)

Spacing

Dimensione fisica del voxel in millimetri; metadato indispensabile per rendere confrontabili volumi diversi. (Cap. 5)

Sparsità

Frazione di attivazioni nulle; conseguenza tipica di ReLU. (Cap. 12)

Stride

Passo di traslazione della finestra convoluzionale o di pooling. $s = 2$ dimezza la risoluzione; non altera il numero di parametri. (Cap. 9)

Temperature scaling

Divisione dei logits per uno scalare T appreso su validazione: corregge la calibrazione senza modificare la classe predetta. (Cap. 18)

Tensore

Array multidimensionale; la forma è la tupla delle dimensioni. Convenzione PyTorch: NCHW (o NCDHW in 3D). (Cap. 4)

Test set

Insieme usato una sola volta, alla fine, per stimare la prestazione su dati nuovi. Ogni decisione presa guardandolo lo invalida. (Cap. 27)

Test-time augmentation (TTA)

Media delle predizioni su più trasformazioni dell'input in inference; migliora leggermente le metriche e fornisce un'ulteriore stima di incertezza. (Cap. 30)

Transfer learning

Riutilizzo di pesi pre-addestrati su un compito diverso come punto di partenza. (Cap. 28)

Underfitting

Errore alto sia in training sia in validazione: capacità insufficiente, regolarizzazione eccessiva o — più spesso — un bug di pipeline. (Cap. 29)

Validation set

Insieme usato per prendere decisioni sul modello senza aggiornarne i pesi: iperparametri, epoca di arresto, soglie. (Cap. 27)

Vanishing gradient

Attenuazione progressiva del gradiente lungo la retropropagazione, causata dal prodotto di molte derivate minori di 1. Mitigato da ReLU, normalizzazione e skip connection. (Cap. 12, 33)

Weight decay

Penalizzazione $\lambda \|\theta\|^2$ aggiunta alla loss, che scoraggia pesi di modulo elevato. (Cap. 28)

Weight sharing

Uso dello stesso kernel in tutte le posizioni spaziali: riduce drasticamente i parametri e produce l'equivarianza traslazionale. (*Cap. 2, 7*)

CHIUSURA

Il percorso è completo. Una CNN è una composizione di operazioni lineari locali con pesi condivisi, intervallate da non linearità e riduzioni di risoluzione, la cui uscita viene confrontata con un'annotazione tramite una loss differenziabile; i gradienti di quella loss, calcolati per regola della catena, modificano tutti i parametri, kernel compresi.

La segmentazione non richiede nuova matematica: richiede di eliminare il flatten, sostituire la testa densa con una convoluzione 1×1 , applicare la loss per pixel e — per non pagare in precisione il downsampling necessario a costruire il contesto — aggiungere un decoder e le skip connection. Da qui in avanti, U-Net e nnU-Net sono leggibili come conseguenze, non come oggetti nuovi.